دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده‌ی مهندسی کامپیوتر

عنوان:

تمرین سری اول

درس:

یادگیری ماشین قابل اعتماد

دانشجو:

امید طرب آور (۴۰۴۱۳۱۰۹۷)

استاد:

دکتر امیرمزلقانی

فهرست مطالب

عنوان **صفحه**

۱- انتخاب ویژگی ۵

۱-۱- تحلیل تأثیر انتخاب ویژگی بر Bias و Variance و تعمیم‌پذیری مدل ۵

۱-۲- ارزیابی تجربی تأثیر انتخاب ویژگی و نرمال‌سازی داده‌ها ۶

۱-۲-۱- نتایج و تحلیل در حالت بدون نرمال‌سازی ۶

۱-۲-۲- نتایج و تحلیل در حالت با نرمال‌سازی ۸

۱-۳- نکات پیاده‌سازی و ملاحظات عملی ۹

۱-۴- نتیجه‌گیری ۹

۲- داده‌افزایی برای بهبود تعمیم‌پذیری ۱۰

۲-۱- تحلیل تأثیر داده‌افزایی بر بایاس مدل ۱۰

۲-۲- مقایسه داده‌افزایی کلاسیک و رویکردهای مولد در تولید نمونه‌های جدید ۱۱

۲-۳- تأثیر داده‌افزایی بیش‌ازحد بر بایاس مدل ۱۳

۲-۴- ارزیابی تجربی روش‌های داده‌افزایی بر روی مجموعه‌داده MNIST ۱۴

۲-۴-۱- عملکرد مدل پایه ۱۴

۲-۴-۲- داده‌افزایی کلاسیک ۱۷

۲-۴-۳- داده‌افزایی مبتنی بر Autoencoder ۲۱

۲-۴-۴- داده‌افزایی مبتنی بر MixUp ۲۶

۲-۵- نکات پیاده‌سازی و ملاحظات عملی ۲۹

۲-۵-۱- تولید داده در MixUp ۲۹

۲-۵-۲- تعریف تابع هزینه در MixUp ۳۰

۲-۶- نتیجه‌گیری ۳۱

۳- تطبیق دامنه و تعمیم دامنه ۳۳

۳-۱- تحلیل اثر Domain Shift بر بایاس، واریانس و تعمیم‌پذیری مدل ۳۳

۳-۲- مقایسه تطبیق دامنه و تعمیم دامنه و بررسی فرضیات مربوط به دامنه هدف ۳۴

۳-۳- تطبیق دامنه در مسئله تشخیص ارقام دست‌نویس MNIST ۳۵
۳-۳-۱- نمایش نمونه داده‌ها ۳۵
۳-۳-۲- ارزیابی مدل پایه بدون تطبیق دامنه ۳۷
۳-۳-۳- کران بالای عملکرد با تنظیم نظارت‌شده ۴۲
۳-۳-۴- تطبیق دامنه با استفاده از Reweighting ۵۰
۳-۳-۵- تطبیق دامنه با استفاده از هم‌ترازی ویژگی‌ها (DANN) ۵۷
۳-۳-۶- تطبیق دامنه با استفاده از ترجمه دامنه مبتنی بر CycleGAN ۶۴
۳-۳-۷- تبیین نقش Domain Classifier Loss در هم‌ترازسازی ویژگی‌ها ۷۱
۳-۳-۸- تحلیل الگوی خطا و بهبود کلاس‌ها پس از تطبیق دامنه ۷۷
۳-۴- نکات پیاده‌سازی و ملاحظات عملی ۸۳
۳-۴-۱- تفاوت پیش‌پردازش در مدل‌های طبقه‌بندی و مدل ترجمه دامنه ۸۳
۳-۴-۲- برآورد وزن نمونه‌ها در روش Reweighting ۸۴
۳-۴-۳- لایه معکوس‌سازی گرادیان در روش DANN ۸۵
۳-۴-۴- طراحی Discriminator به‌صورت PatchGAN در CycleGAN ۸۷
۳-۵- نتیجه‌گیری ۸۹
۴- یادگیری انتقالی ۹۴
۴-۱- تأثیر یادگیری انتقالی بر تعمیم‌پذیری و تحلیل آن در چارچوب Bias–Variance ۹۴
۴-۲- ارزیابی تجربی یادگیری انتقالی با استفاده از معماری LeNet ۹۵
۴-۲-۱- بررسی داده‌ها و تحلیل تغییرات ناشی از چرخش ۹۵
۴-۲-۲- پیش‌آموزش مدل LeNet بر روی مجموعه MNIST ۹۶
۴-۲-۳- پیکربندی آزمایش‌ها و تنظیمات انتقال دانش ۹۷
۴-۲-۴- مقایسه عملکرد حالت‌های مختلف یادگیری انتقالی ۹۸
۴-۳- تحلیل قابلیت انتقال لایه‌ها و مقایسه با Domain Adaptation ۱۰۷
۴-۴- نکات پیاده‌سازی و ملاحظات عملی ۱۱۰
۴-۴-۱- کنترل لایه‌های قابل آموزش در سناریوهای مختلف ۱۱۰

۴-۵- نتیجه‌گیری..... ۱۱۲

فصل اول

انتخاب ویژگی[1]

در این فصل، تأثیر انتخاب ویژگی و پیش‌پردازش داده‌ها بر عملکرد و تعمیم‌پذیری مدل‌های یادگیری ماشین مورد بررسی قرار می‌گیرد. این تحلیل در چارچوب مفهوم Bias-Variance Tradeoff انجام شده و هدف آن ارزیابی این موضوع است که چگونه کاهش یا تغییر فضای ویژگی‌ها می‌تواند بر پیچیدگی مدل و دقت پیش‌بینی تأثیر بگذارد. در بخش نخست، اثر انتخاب ویژگی بر بایاس و واریانس به‌صورت مفهومی تحلیل می‌شود، و در ادامه، این مفاهیم به‌صورت تجربی با استفاده از یک مدل رگرسیون مبتنی بر درخت تصمیم بررسی می‌گردد، به‌طوری‌که تأثیر حذف ویژگی‌های کم‌اهمیت و همچنین نرمال‌سازی داده‌ها بر عملکرد مدل مورد ارزیابی و مقایسه قرار می‌گیرد.

۱-۱- تحلیل تأثیر انتخاب ویژگی بر Bias و Variance و تعمیم‌پذیری مدل

انتخاب ویژگی با کاهش ابعاد فضای ورودی، به‌طور مستقیم بر ظرفیت مدل برای تقریب تابع هدف تأثیر می‌گذارد. با حذف برخی ویژگی‌ها، مدل در فضای ساده‌تری آموزش داده می‌شود و در نتیجه درجه آزادی آن برای انطباق با داده‌های آموزشی کاهش می‌یابد. این کاهش ظرفیت معمولاً منجر به کاهش واریانس می‌شود، زیرا مدل کمتر قادر خواهد بود نوسانات و الگوهای تصادفی موجود در داده‌های آموزشی را یاد بگیرد. در نتیجه، حساسیت مدل نسبت به تغییرات داده کاهش یافته و پایداری آن روی داده‌های جدید افزایش می‌یابد.

با این حال، اگر در فرآیند انتخاب ویژگی، متغیرهایی حذف شوند که اطلاعات معناداری درباره‌ی متغیر هدف دارند، مدل بخشی از توان خود برای نمایش رابطه‌ی واقعی میان ورودی و خروجی را از دست می‌دهد. در این حالت، فضای ساده‌شده‌ی ورودی دیگر برای تقریب صحیح تابع هدف کافی نیست و این موضوع به افزایش بایاس و بروز کم‌برازش منجر می‌شود.

بنابراین، اثر انتخاب ویژگی بر تعمیم‌پذیری را باید در چارچوب Bias-Variance Tradeoff تحلیل کرد. در صورتی که حذف ویژگی‌ها عمدتاً شامل متغیرهای نویزی یا نامرتبط باشد، کاهش واریانس بر افزایش احتمالی بایاس غلبه کرده و عملکرد مدل بهبود می‌یابد. در مقابل، حذف ویژگی‌های مهم باعث افزایش بایاس شده و می‌تواند منجر به افت دقت و کاهش توان تعمیم گردد. از این رو، انتخاب ویژگی زمانی مؤثر است که با حفظ اطلاعات اصلی مسئله، از پیچیدگی غیرضروری مدل کاسته شود.

[1] Feature Selection

۱-۲- ارزیابی تجربی تأثیر انتخاب ویژگی و نرمال‌سازی داده‌ها

در این بخش، به‌منظور بررسی تجربی اثر انتخاب ویژگی، یک مدل Decision Tree Regressor بر روی مجموعه‌داده قیمت خانه‌ها (House_Prices.csv) آموزش داده شد. پس از آموزش مدل، اهمیت هر یک از ویژگی‌ها با استفاده از معیار Feature Importance استخراج گردید. این معیار نشان می‌دهد که هر ویژگی تا چه میزان در کاهش خطای مدل و تصمیم‌گیری در ساختار درخت نقش داشته است.

نتایج به‌دست‌آمده برای اهمیت ویژگی‌ها در جدول زیر ارائه شده است:

جدول ۱–۱: میزان اهمیت ویژگی‌ها

ویژگی	اهمیت FI
Area (decimeter^2)	۰.۹۹۹۶۲۵
Year	۰.۰۰۰۳۲۱
Parking	۰.۰۰۰۰۳۱
Elevator	۰.۰۰۰۰۲۳

بر اساس این نتایج، مشاهده می‌شود که ویژگی مساحت[۲] به‌صورت غالب در فرآیند تصمیم‌گیری مدل نقش دارد و سهم سایر ویژگی‌ها در مقایسه با آن بسیار ناچیز است. این موضوع نشان می‌دهد که مدل درخت تصمیم عمدتاً بر اساس این ویژگی اقدام به یادگیری رابطه‌ی بین ورودی‌ها و متغیر هدف نموده است.

۱-۲-۱- نتایج و تحلیل در حالت بدون نرمال‌سازی

پس از استخراج اهمیت ویژگی‌ها، عملکرد مدل در دو حالت بررسی شد: نخست، حالتی که در آن مدل با استفاده از تمامی ویژگی‌ها آموزش داده شد، و دوم، حالتی که تنها مهم‌ترین ویژگی یعنی Area به‌عنوان ورودی مدل در نظر گرفته شد. نتایج این دو آزمایش بر اساس معیارهای R^2، RMSE و MAE در جدول زیر ارائه شده است:

جدول ۱–۲: نتایج حاصل از اجرای آزمایش‌ها

مدل	ویژگی‌های مورد استفاده	R^2	RMSE	MAE
مدل ۱	تمامی ویژگی‌ها	۰.۹۹۹۶	۳۵.۳۵۵۳	۱۲.۵۰۰۰
مدل ۲	فقط Area	۰.۹۹۸۶	۶۳.۴۳۷۹	۴۲.۲۱۰۰

[2] Area

مطابق نتایج جدول، هرچند مدل تک‌ویژگی همچنان عملکرد بسیار بالایی دارد، اما استفاده از تمامی ویژگی‌ها به دقت بیشتری منجر شده است. مقدار بسیار بالای اهمیت ویژگی Area نشان می‌دهد که بخش اصلی ساختار رابطه‌ی میان ورودی و متغیر هدف توسط این ویژگی تعیین می‌شود؛ بنابراین انتظار می‌رود که مدل تنها با همین ویژگی نیز بخش عمده‌ای از رفتار داده را بازسازی کند. با این حال، یکسان نبودن نتایج دو مدل نشان می‌دهد که ویژگی‌های دیگر، با وجود اهمیت بسیار کم، کاملاً بی‌اثر نیستند.

توجیه این اختلاف آن است که معیار Feature Importance تنها سهم نسبی هر ویژگی را در کاهش خطا در ساختار نهایی درخت نشان می‌دهد و کوچک بودن این مقدار لزوماً به معنای بی‌فایده بودن کامل آن ویژگی نیست. درخت تصمیم، فضای ورودی را به‌صورت سلسله‌مراتبی تقسیم می‌کند و ممکن است برخی ویژگی‌ها فقط در تعداد محدودی از گره‌ها، یا در نواحی خاصی از فضای داده، نقش اصلاحی داشته باشند. در چنین شرایطی، این ویژگی‌ها سهم کلی کوچکی در ساختار درخت دارند، اما همان سهم محدود می‌تواند برای کاهش خطا در نقاط مرزی، نمونه‌های خاص یا حالات نزدیک به هم مؤثر باشد. به همین دلیل، حذف آن‌ها اگرچه ساختار اصلی مدل را از بین نمی‌برد، اما دقت نهایی پیش‌بینی را کاهش می‌دهد.

این موضوع در معیارهای خطا به‌خوبی مشاهده می‌شود. کاهش جزئی R^2 نشان می‌دهد که ویژگی Area به‌تنهایی قادر است تقریب بسیار خوبی از روند کلی قیمت ارائه دهد، اما افزایش قابل توجه RMSE و MAE بیانگر آن است که مدل تک‌ویژگی در برآورد دقیق‌تر مقادیر واقعی خطای بیشتری دارد. به بیان دیگر، Area اطلاعات غالب مسئله را حمل می‌کند، اما ویژگی‌هایی مانند Year، Parking و Elevator نقش تنظیم‌کننده و تکمیلی دارند و می‌توانند در تفکیک نمونه‌هایی که از نظر مساحت مشابه‌اند ولی از نظر قیمت تفاوت دارند مؤثر باشند.

علاوه بر تحلیل دقت، می‌توان این مسئله را از منظر مهندسی و عملیاتی نیز مورد توجه قرار داد. کاهش تعداد ویژگی‌ها، حتی در صورتی که با افت جزئی عملکرد همراه باشد، در بسیاری از کاربردها قابل‌قبول و حتی مطلوب است. استفاده از یک مدل تک‌ویژگی منجر به سادگی بیشتر، کاهش هزینه محاسباتی، افزایش سرعت آموزش و استنتاج، و همچنین بهبود قابلیت تفسیر مدل می‌شود. در سناریوهایی که محدودیت‌هایی نظیر زمان، منابع محاسباتی یا دسترسی به داده‌های کامل وجود دارد، پذیرش یک مصالحه[3] میان دقت و سادگی مدل می‌تواند تصمیمی منطقی باشد. در این حالت، مدلی که تنها از ویژگی Area استفاده می‌کند، با وجود افت نسبی در دقت، همچنان عملکرد بسیار قابل‌قبولی ارائه می‌دهد.

در جمع‌بندی، نتایج نشان می‌دهد که اگرچه ویژگی Area نقش غالب را در پیش‌بینی قیمت ایفا می‌کند، اما ویژگی‌های دیگر نیز به‌عنوان عوامل تکمیلی در بهبود دقت نهایی مدل مؤثر هستند. حذف این ویژگی‌ها منجر به کاهش دقت پیش‌بینی می‌شود، هرچند این کاهش در مقایسه با سادگی حاصل‌شده از مدل، در برخی کاربردها قابل‌چشم‌پوشی است. بنابراین، انتخاب بین استفاده از تمامی ویژگی‌ها یا تنها مهم‌ترین ویژگی باید بر اساس نیاز مسئله، میزان حساسیت به خطا و محدودیت‌های موجود انجام شود، به‌گونه‌ای که تعادل مناسبی میان دقت و سادگی مدل برقرار گردد.

[3] Trade-off

۱-۲-۲- نتایج و تحلیل در حالت با نرمال‌سازی

در این بخش، همان آزمایش بخش قبل پس از اعمال نرمال‌سازی داده‌ها تکرار شد. به این منظور، مقادیر ویژگی‌ها با استفاده از روش Min-Max به بازه‌ی [0,1] نگاشت شدند و سپس مدل Decision Tree Regressor در دو حالت (استفاده از تمامی ویژگی‌ها و استفاده از تنها مهم‌ترین ویژگی) آموزش داده شد. نتایج به‌دست‌آمده در جدول زیر ارائه شده است:

جدول ۱-۳: نتایج حاصل از اجرای آزمایش‌ها پس از نرمال‌سازی

مدل	ویژگی‌های مورد استفاده	R^2	RMSE	MAE
مدل ۱	تمامی ویژگی‌ها	۰.۹۹۹۶	۳۵.۳۵۵۳	۱۲.۵۰۰۰
مدل ۲	فقط Area	۰.۹۹۸۶	۶۳.۴۳۷۹	۴۲.۲۱۰۰

مطابق جدول، مشاهده می‌شود که نتایج به‌دست‌آمده در حالت با نرمال‌سازی کاملاً مشابه با حالت بدون نرمال‌سازی است. این موضوع نشان می‌دهد که نرمال‌سازی داده‌ها تأثیری بر عملکرد مدل در این مسئله نداشته است. دلیل این رفتار را می‌توان در ماهیت الگوریتم درخت تصمیم جستجو کرد. درخت تصمیم بر اساس مقایسه‌ی مقادیر ویژگی‌ها با آستانه‌های مشخص[۴] عمل می‌کند و تصمیم‌گیری آن مبتنی بر ترتیب نسبی داده‌ها است، نه مقیاس مطلق آن‌ها. به عبارت دیگر، اگرچه نرمال‌سازی بازه‌ی مقادیر ویژگی‌ها را تغییر می‌دهد، اما ترتیب نسبی نمونه‌ها حفظ می‌شود؛ در نتیجه، نقاط برش[۵] در درخت تنها به‌صورت عددی تغییر مقیاس می‌دهند، بدون آن‌که ساختار تصمیم‌گیری مدل دچار تغییر شود.

از دیدگاه مهندسی، این خاصیت به این معناست که درخت تصمیم نسبت به تبدیل‌های یکنوا[۶] نظیر نرمال‌سازی خطی ناوردا[۷] یا تغییر ناپذیر است. بنابراین، فرآیند نرمال‌سازی نه‌تنها بر ساختار درخت تأثیری ندارد، بلکه مقادیر Feature Importance نیز بدون تغییر باقی می‌مانند، زیرا سهم نسبی هر ویژگی در کاهش خطا وابسته به نحوه‌ی تقسیم‌بندی داده‌ها است، نه مقیاس عددی آن‌ها.

در نتیجه، در این مسئله خاص، نرمال‌سازی داده‌ها تأثیر معناداری بر عملکرد مدل نداشته و انتخاب یا عدم انتخاب آن تغییری در دقت نهایی ایجاد نکرده است. این رفتار نشان می‌دهد که برای مدل‌های مبتنی بر درخت، برخلاف بسیاری از الگوریتم‌های دیگر، نرمال‌سازی یک مرحله‌ی ضروری محسوب نمی‌شود و بیشتر در مدل‌هایی اهمیت دارد که وابسته به فاصله یا مقیاس داده‌ها هستند.

[4] Threshold
[5] Split points
[6] Monotonic transformations
[7] Invariant

۱-۳- نکات پیاده‌سازی و ملاحظات عملی

پیاده‌سازی این بخش از تمرین با توجه به استفاده از مدل Decision Tree Regressor نسبتاً ساده بوده و شامل مراحل استانداردی نظیر بارگذاری داده، جداسازی ویژگی‌ها و متغیر هدف، آموزش مدل و استخراج معیارهای ارزیابی است. فرآیند انتخاب ویژگی نیز مستقیماً بر اساس مقادیر Feature Importance انجام شده و نیاز به پیاده‌سازی پیچیده‌ای نداشته است.

تنها نکته قابل توجه در این بخش، استفاده از روش Min-Max Normalization در مرحله پیش‌پردازش داده‌ها است. در این روش، مقادیر هر ویژگی به بازه‌ی [0,1] نگاشت می‌شوند. انتخاب این روش عمدتاً به دلیل سادگی، رایج بودن در بسیاری از کاربردها، و حفظ ترتیب نسبی داده‌ها صورت گرفته است. به‌عبارت دیگر، این تبدیل یک نگاشت خطی یکنوا است که بدون تغییر در ساختار داده، مقیاس آن را استاندارد می‌کند.

با این حال، همان‌طور که در نتایج مشاهده شد، استفاده از نرمال‌سازی تأثیر معناداری بر عملکرد مدل نداشته است. این موضوع به ماهیت الگوریتم درخت تصمیم بازمی‌گردد که تصمیم‌گیری در آن بر اساس مقایسه‌های ترتیبی انجام می‌شود و به مقیاس مطلق داده‌ها وابسته نیست. بنابراین، انتخاب Min-Max در این تمرین بیشتر به‌منظور رعایت چارچوب مسئله و بررسی تجربی اثر نرمال‌سازی انجام شده است، نه به دلیل نیاز ذاتی مدل به این پیش‌پردازش.

۱-۴- نتیجه‌گیری

در این فصل، تأثیر انتخاب ویژگی و نرمال‌سازی داده‌ها بر عملکرد مدل مورد بررسی قرار گرفت. نتایج نشان داد که ویژگی Area نقش غالب در پیش‌بینی دارد، اما سایر ویژگی‌ها نیز به‌صورت تکمیلی در کاهش خطا مؤثر هستند. همچنین مشاهده شد که حذف ویژگی‌ها، هرچند منجر به ساده‌تر شدن مدل می‌شود، می‌تواند با افت دقت همراه باشد و لذا انتخاب آن نیازمند در نظر گرفتن مصالحه میان سادگی و عملکرد است. در نهایت، مشخص شد که نرمال‌سازی داده‌ها تأثیر معناداری بر مدل درخت تصمیم ندارد، که این موضوع با ماهیت این الگوریتم سازگار است.

فصل دوم

داده‌افزایی[8] برای بهبود تعمیم‌پذیری[9]

در این فصل، تأثیر داده‌افزایی به‌عنوان یکی از روش‌های مؤثر در بهبود تعمیم‌پذیری مدل‌های یادگیری ماشین مورد بررسی قرار می‌گیرد. ابتدا از منظر تئوری، ارتباط داده‌افزایی با مؤلفه بایاس تحلیل می‌شود و سپس در قالب آزمایش‌های عملی بر روی مجموعه‌داده MNIST، عملکرد مدل در حضور روش‌های مختلف داده‌افزایی شامل رویکردهای کلاسیک، مبتنی بر مدل‌های مولد، و روش‌های پیشرفته‌تر مورد ارزیابی و مقایسه قرار می‌گیرد.

۲-۱- تحلیل تأثیر داده‌افزایی بر بایاس مدل

داده‌افزایی به‌طور کلی به مجموعه‌ای از روش‌ها اطلاق می‌شود که با افزایش تنوع داده‌های آموزشی –چه از طریق اعمال تبدیلات روی داده‌های موجود و چه با تولید نمونه‌های جدید– سعی در بهبود فرآیند یادگیری مدل دارند. این افزایش تنوع، اگرچه در ظاهر تنها به گسترش داده‌ها مربوط است، اما در سطح عمیق‌تر به معنای وارد کردن دانش پیشین[10] درباره ساختار مسئله به مدل است.

برای تحلیل تأثیر داده‌افزایی بر بایاس، باید توجه داشت که بایاس به میزان محدود بودن فضای فرضیات مدل اشاره دارد. هرچه مدل به مجموعه محدودتری از توابع ممکن مقید شود، بایاس آن افزایش می‌یابد. داده‌افزایی دقیقاً از همین طریق عمل می‌کند: با اضافه کردن نمونه‌هایی که بیانگر برخی ویژگی‌های مطلوب مسئله هستند، مدل را به سمت یادگیری توابعی سوق می‌دهد که با این ویژگی‌ها سازگار باشند.

برای مثال، در مجموعه‌داده MNIST اگر فرض کنیم که چرخش‌های کوچک نباید بر برچسب رقم تأثیر بگذارند، می‌توانیم با اعمال چنین تبدیلاتی یا تولید نمونه‌های متناظر، این خاصیت را به داده‌ها اضافه کنیم. در این صورت، مدل در طول آموزش با نمونه‌هایی مواجه می‌شود که نشان می‌دهند یک رقم در زوایای مختلف همچنان به همان کلاس تعلق دارد. این موضوع به‌صورت ضمنی این قید را به مدل تحمیل می‌کند که نسبت به این نوع تغییرات ناوردا باشد. در نتیجه، مدل دیگر نمی‌تواند توابعی را انتخاب کند که به تغییرات جزئی در زاویه حساس هستند، بلکه به سمت توابعی هدایت می‌شود که این ناوردایی را رعایت می‌کنند. این محدودسازی فضای فرضیات، مصداق افزایش بایاس است.

[8] Data Augmentation
[9] Generalization
[10] Inductive bias

نکته مهم آن است که این افزایش بایاس لزوماً منفی نیست، بلکه در بسیاری از موارد بیانگر وارد کردن فرض‌های درست درباره مسئله به مدل است. به بیان دیگر، داده‌افزایی با اعمال قیود معنادار، مدل را از یادگیری الگوهای ناپایدار یا تصادفی بازمی‌دارد و آن را به سمت ساختارهای کلی‌تر هدایت می‌کند. بنابراین، می‌توان گفت داده‌افزایی از طریق جهت‌دهی فضای یادگیری، بایاس مدل را افزایش می‌دهد، اما این افزایش در صورتی که مبتنی بر فرض‌های صحیح درباره داده باشد، به بهبود رفتار مدل منجر خواهد شد.

۲-۲- مقایسه داده‌افزایی کلاسیک و رویکردهای مولد در تولید نمونه‌های جدید

داده‌افزایی را می‌توان از منظر نحوه‌ی تولید نمونه‌های جدید به دو رویکرد متمایز تقسیم کرد: رویکردهای مبتنی بر اعمال تبدیلات صریح بر داده‌های موجود و رویکردهایی که با استفاده از مدل‌های مولد، نمونه‌های جدیدی را بر پایه‌ی ساختار آماری داده تولید می‌کنند. این دو رویکرد، اگرچه هدف مشترکی در افزایش تنوع داده‌های آموزشی دارند، اما از نظر سازوکار تولید داده، میزان کنترل بر نمونه‌ها، پیچیدگی محاسباتی و شرایط کاربرد، تفاوت‌های بنیادینی دارند.

در داده‌افزایی کلاسیک، نمونه‌ی جدید مستقیماً از یک نمونه‌ی موجود و با اعمال یک تبدیل مشخص تولید می‌شود. به‌صورت صوری، اگر x یک نمونه از داده باشد، نمونه‌ی جدید را می‌توان به‌صورت

$$x' = T(x) \quad (۱)$$

نوشت که در آن T یک تبدیل از پیش تعریف‌شده مانند چرخش[11]، وارونگی[12]، جابه‌جایی[13]، برش، تغییر مقیاس، یا افزودن نویز کنترل‌شده است. در این چارچوب، منبع تنوع داده به مجموعه‌ی تبدیلاتی محدود است که طراح سیستم آن‌ها را انتخاب می‌کند. در نتیجه، رفتار داده‌های تولیدی تا حد زیادی قابل پیش‌بینی و کنترل‌پذیر است. اگر تبدیل انتخاب‌شده با ماهیت مسئله سازگار باشد، می‌توان با اطمینان نسبتاً بالایی انتظار داشت که برچسب نمونه حفظ شود. برای مثال، در مجموعه‌داده MNIST، اعمال چرخش‌های کوچک یا جابه‌جایی‌های جزئی معمولاً باعث تغییر کلاس رقم نمی‌شود و بنابراین چنین تبدیلاتی می‌توانند به‌صورت ایمن برای افزایش تنوع داده به‌کار روند.

در مقابل، در رویکردهای مولد مانند Autoencoder یا GAN، داده‌ی جدید الزاماً از طریق یک تبدیل صریح بر روی یک نمونه‌ی خاص به‌دست نمی‌آید. در اینجا، مدل ابتدا تلاش می‌کند توزیع داده‌ها یا نمایش نهفته‌ی آن‌ها را یاد بگیرد و سپس بر اساس این دانش، نمونه‌های جدید تولید کند. به‌عنوان مثال، در GAN یک بردار نهفته‌ی تصادفی z از یک توزیع ساده نمونه‌گیری شده و از طریق تابع مولد به یک نمونه‌ی داده تبدیل می‌شود:

$$x' = G(z) \quad z \sim p(z) \quad (۲)$$

[11] Rotation
[12] Flip
[13] Translation

در این حالت، نمونه‌ی تولیدشده لزوماً به یک نمونه‌ی خاص از داده‌ی اولیه وابسته نیست، بلکه باید از نظر آماری به توزیع داده‌های واقعی شباهت داشته باشد. در Autoencoder نیز، اگرچه در حالت پایه خروجی نزدیک به بازسازی ورودی است، اما با یادگیری یک نمایش فشرده، مدل می‌تواند ساختارهای مهم داده را استخراج کرده و نمونه‌هایی نزدیک به داده‌های واقعی تولید کند.

یکی از تفاوت‌های کلیدی این دو رویکرد در منشأ تنوع داده است. در داده‌افزایی کلاسیک، تنوع به‌صورت صریح و بر اساس تبدیلات از پیش تعیین‌شده ایجاد می‌شود؛ در حالی که در روش‌های مولد، تنوع نتیجه‌ی یادگیری الگوهای آماری داده و بازتولید آن‌ها است. این تفاوت باعث می‌شود که داده‌افزایی کلاسیک معمولاً قابل پیش‌بینی‌تر و پایدارتر باشد، در حالی که روش‌های مولد ظرفیت بیشتری برای تولید تنوع‌های پیچیده دارند.

تفاوت مهم دیگر به قابلیت کنترل بر کیفیت نمونه‌های تولیدی مربوط می‌شود. در روش‌های کلاسیک، چون نوع تبدیل مشخص است، رابطه‌ی بین نمونه‌ی اصلی و نمونه‌ی جدید روشن بوده و حفظ برچسب قابل توجیه است. اما در روش‌های مولد، کیفیت داده‌های تولیدی به میزان موفقیت مدل در یادگیری توزیع داده بستگی دارد. در صورتی که مدل مولد به‌خوبی آموزش نبیند، ممکن است نمونه‌هایی تولید شود که مبهم، غیرواقعی یا خارج از توزیع واقعی داده باشند. بنابراین، اگرچه روش‌های مولد از نظر تئوری انعطاف‌پذیرتر هستند، اما در عمل نیازمند دقت بیشتری در ارزیابی کیفیت خروجی‌ها هستند.

از نظر هزینه‌ی محاسباتی و پیچیدگی پیاده‌سازی نیز تفاوت قابل توجهی وجود دارد. داده‌افزایی کلاسیک معمولاً بسیار ساده، سریع و کم‌هزینه است و نیاز به آموزش مدل جداگانه ندارد. در مقابل، استفاده از Autoencoder یا GAN مستلزم آموزش یک مدل اضافی است که هم از نظر زمانی و هم از نظر محاسباتی هزینه‌بر است. علاوه بر این، آموزش برخی مدل‌های مولد مانند GAN می‌تواند با چالش‌هایی مانند ناپایداری همراه باشد.

از منظر کاربرد، داده‌افزایی کلاسیک در شرایطی مؤثرتر است که ناوردایی‌های مسئله به‌خوبی شناخته شده و بتوان آن‌ها را با تبدیلات ساده مدل کرد. در چنین حالتی، این روش بدون نیاز به پیچیدگی اضافی، می‌تواند تنوع مفیدی به داده‌ها اضافه کند. برای مثال، در مسائلی مانند تشخیص ارقام دست‌نویس یا بسیاری از کاربردهای پایه در بینایی ماشین، تبدیلاتی مانند چرخش‌های کوچک یا جابه‌جایی‌های جزئی می‌توانند به‌طور مؤثری تعمیم‌پذیری مدل را بهبود دهند.

در مقابل، روش‌های مولد در سناریوهایی مؤثرتر هستند که ساختار تغییرات در داده پیچیده بوده و با تبدیلات ساده قابل توصیف نباشد. برای مثال، در داده‌هایی مانند تصاویر پزشکی، چهره، یا داده‌های با ساختار پیچیده، تغییرات معنادار ممکن است شامل الگوهای ظریفی باشند که نمی‌توان آن‌ها را صرفاً با چرخش یا نویز مدل کرد. در این شرایط، اگر مدل مولد بتواند توزیع داده را به‌درستی یاد بگیرد، قادر خواهد بود نمونه‌هایی تولید کند که تنوع واقعی‌تری را بازتاب دهند. همچنین در شرایطی که داده‌ی برچسب‌دار محدود است، استفاده از مدل‌های مولد می‌تواند به غنی‌تر شدن مجموعه‌ی آموزشی کمک کند، به شرط آنکه کیفیت نمونه‌های تولیدی مناسب باشد.

در جمع‌بندی، داده‌افزایی کلاسیک با اتکا به تبدیلات صریح و کنترل‌شده، روشی ساده، قابل اعتماد و کم‌هزینه برای افزایش تنوع داده‌ها است، در حالی که رویکردهای مولد با یادگیری ساختار آماری داده، امکان تولید نمونه‌های متنوع‌تر و پیچیده‌تر را

فراهم می‌کنند، اما در مقابل به هزینه‌ی محاسباتی بیشتر و دقت بالاتر در ارزیابی کیفیت نیاز دارند. انتخاب میان این دو رویکرد باید بر اساس ماهیت مسئله، میزان شناخت از ناوردایی‌های داده، و محدودیت‌های محاسباتی انجام شود.

۲-۳- تأثیر داده‌افزایی بیش‌ازحد[14] بر بایاس مدل

داده‌افزایی زمانی مؤثر است که نمونه‌های تولیدشده همچنان نماینده‌ی معتبری از توزیع واقعی داده‌ها باشند و ویژگی‌های معنادار مسئله را حفظ کنند. در مقابل، اگر شدت یا ماهیت تغییرات به‌گونه‌ای باشد که این شرط نقض شود، پدیده‌ای رخ می‌دهد که می‌توان آن را داده‌افزایی بیش‌ازحد نامید. در این حالت، داده‌های تولیدی دیگر با ساختار واقعی مسئله سازگار نیستند و در نتیجه فرآیند یادگیری مدل دچار اختلال می‌شود.

از منظر بایاس، داده‌افزایی با اعمال قیود ناوردایی از جنس

$$f(x) = f(T(x)) \tag{۳}$$

فضای فرضیات مدل را محدود می‌کند. در حالت ایده‌آل، این ناوردایی‌ها بازتاب‌دهنده‌ی ویژگی‌های واقعی مسئله هستند. اما در شرایط over-augmentation، تبدیلاتی اعمال می‌شوند که این فرض ناوردایی در آن‌ها معتبر نیست. به بیان دیگر، رابطه‌ی فوق در واقعیت برقرار نیست، اما همچنان به مدل تحمیل می‌شود. این موضوع باعث می‌شود که مدل به سمت یادگیری توابعی سوق داده شود که با داده‌های واقعی ناسازگار هستند و در نتیجه، بایاس مدل در جهت نامطلوب افزایش یابد.

برای مثال، در مجموعه‌داده MNIST، اعمال چرخش‌های کوچک (مثلاً تا $10°$) معمولاً ساختار رقم را حفظ می‌کند و فرض ناوردایی نسبت به چرخش قابل توجیه است. اما در صورت اعمال چرخش‌های بزرگ (مانند $180°$)، این فرض دیگر معتبر نیست. به‌عنوان نمونه، رقم «۶» پس از چنین چرخشی به شکلی بسیار شبیه «۹» تبدیل می‌شود، یا برخی ارقام مانند «۲» و «۵» به اشکالی تبدیل می‌شوند که دیگر نماینده‌ی همان کلاس نیستند. در چنین شرایطی، اگر همچنان برچسب اولیه حفظ شود، داده‌های آموزشی حاوی تناقض خواهند بود و مدل نمی‌تواند مرز تصمیم معناداری بیاموزد. این وضعیت منجر به افزایش بایاس و بروز underfitting می‌شود.

پدیده‌ای مشابه در رویکردهای مولد مانند Autoencoder یا GAN نیز قابل مشاهده است، با این تفاوت که منشأ آن متفاوت است. در این روش‌ها، داده‌های جدید از طریق یادگیری و نمونه‌گیری از یک توزیع تخمینی $p_{model}(x)$ تولید می‌شوند. اگر این توزیع به‌خوبی با توزیع واقعی داده‌ها $p_{data}(x)$ منطبق نباشد، نمونه‌های تولیدشده ممکن است کیفیت پایین، ساختار غیرواقعی، یا تنوع نامعتبر داشته باشند. برای مثال، در MNIST ممکن است یک Autoencoder تصاویر بازسازی‌شده‌ای با لبه‌های محو یا شکل‌های تغییریافته تولید کند، یا یک GAN نمونه‌هایی بسازد که به‌وضوح بین چند کلاس مختلف قرار دارند. استفاده از چنین داده‌هایی در آموزش، به این معناست که مدل با الگوهایی مواجه می‌شود که در داده‌های واقعی وجود ندارند یا به‌درستی تعریف نشده‌اند. در نتیجه، فضای فرضیات به‌صورت نادرست محدود شده و بایاس در جهت نامطلوب افزایش می‌یابد.

[14] Over-Augmentation

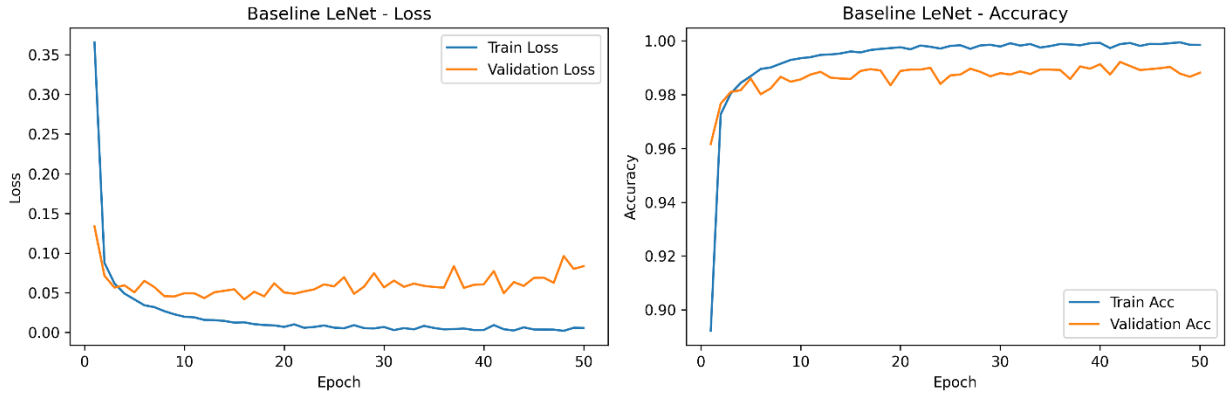
در مجموع، داده‌افزایی بیش‌ازحد -چه در قالب تبدیلات نامناسب در روش‌های کلاسیک و چه در قالب تولید نمونه‌های غیرواقعی در روش‌های مولد- منجر به انحراف داده‌های آموزشی از توزیع واقعی و تحمیل قیود نادرست به مدل می‌شود. این امر باعث افزایش بایاس به‌صورت نامطلوب شده و توانایی مدل در یادگیری الگوهای صحیح را کاهش می‌دهد. بنابراین، انتخاب نوع و شدت مناسب داده‌افزایی نقشی اساسی در حفظ سازگاری میان داده‌های آموزشی و واقعیت مسئله و در نتیجه دستیابی به تعمیم‌پذیری مطلوب ایفا می‌کند.

۲-۴- ارزیابی تجربی روش‌های داده‌افزایی بر روی مجموعه‌داده MNIST

در این بخش، عملکرد مدل در سناریوهای مختلف داده‌افزایی به‌صورت تجربی مورد بررسی قرار می‌گیرد. ابتدا یک مدل پایه بدون استفاده از داده‌افزایی آموزش داده می‌شود تا به‌عنوان مبنای مقایسه در نظر گرفته شود. سپس، روش‌های مختلف داده‌افزایی اعمال شده و تأثیر آن‌ها بر عملکرد مدل از جنبه‌های مختلفی مانند دقت، خطا، و الگوی اشتباهات تحلیل می‌شود.

۲-۴-۱- عملکرد مدل پایه

مدل LeNet بدون استفاده از هیچ‌گونه داده‌افزایی بر روی مجموعه‌داده MNIST آموزش داده شد تا عملکرد پایه‌ای آن به‌دست آید. نتایج آموزش این مدل در شکل زیر نشان داده شده است.



*شکل ۲–۱: تغییرات مقدار **Loss** و **Accuracy** در داده‌های آموزش و اعتبارسنجی برای مدل پایه **LeNet** بر روی مجموعه‌داده **MNIST***

همان‌طور که در شکل مشاهده می‌شود، مقدار Train Loss به‌سرعت کاهش یافته و به مقادیر بسیار کوچک همگرا شده است، در حالی که Validation Loss پس از کاهش اولیه، به مقدار نسبتاً ثابتی رسیده و در ادامه نوسانات جزئی همراه با یک روند افزایشی محسوس‌تر در انتهای آموزش نشان می‌دهد. این رفتار بیانگر بروز overfitting در مدل است که در اواخر epochها کمی پررنگ‌تر شده است. از سوی دیگر، مقدار Accuracy در هر دو مجموعه‌ی آموزش و اعتبارسنجی به مقادیر بالایی نزدیک به ۱ رسیده و روندی نسبتاً پایدار دارد، هرچند دقت اعتبارسنجی اندکی کمتر و با نوسان بیشتری همراه است. این اختلاف نشان می‌دهد که مدل تا حدی به داده‌های آموزشی متمایل شده، اما همچنان از سطح مناسبی از تعمیم‌پذیری برخوردار است.

در ادامه، نتایج کمی نهایی مدل بر روی داده‌های آموزش، اعتبارسنجی و آزمون در جدول زیر ارائه شده است.

*جدول ۲–۱: مقادیر نهایی دقت و خطا برای مدل پایه **LeNet** بر روی داده‌های آموزش، اعتبارسنجی و آزمون*

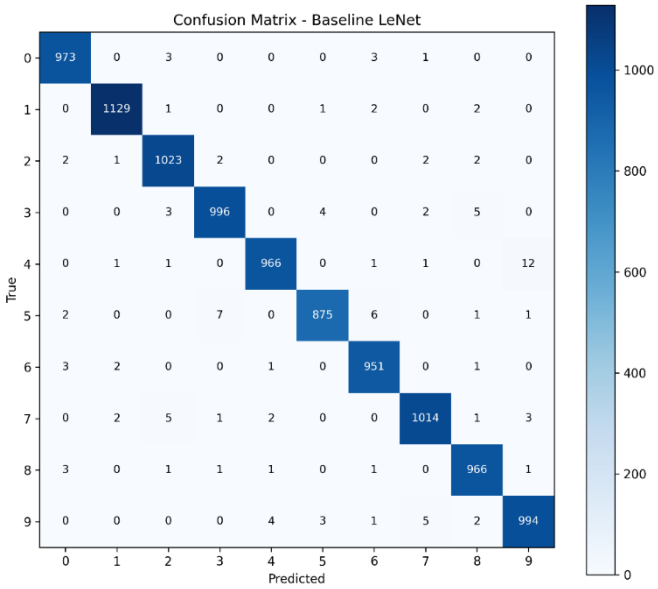
معیار	مقدار
Final Train Accuracy	۰.۹۹۸۵
Final Train Loss	۰.۰۰۵۳
Final Validation Accuracy	۰.۹۸۸۲
Final Validation Loss	۰.۰۸۳۳
Test Accuracy	۰.۹۸۸۷
Test Loss	۰.۰۶۶۸

دقت حدود ۹۸.۹٪ بر روی داده‌های آزمون نشان می‌دهد که مدل پایه حتی بدون استفاده از داده‌افزایی نیز عملکرد بسیار مطلوبی دارد. همچنین مقدار بسیار پایین Train Loss در کنار دقت نزدیک به ۱ در داده‌های آموزش بیانگر این است که مدل به‌خوبی داده‌های آموزشی را یاد گرفته است.

با این حال، مقایسه مقادیر بین مجموعه‌های آموزش و اعتبارسنجی نشان می‌دهد که مقدار Loss در داده‌های اعتبارسنجی و آزمون به‌طور محسوسی بیشتر از داده‌های آموزش است، در حالی که اختلاف دقت‌ها نسبتاً اندک باقی مانده است. این موضوع بیانگر بروز overfitting خفیف در مدل است، به این معنا که مدل تا حدی به داده‌های آموزشی وابسته شده اما همچنان توانایی تعمیم‌پذیری مناسبی دارد.

از سوی دیگر، اختلاف میان Validation و Test بسیار کم است، که نشان می‌دهد فرآیند آموزش و انتخاب مدل به‌درستی انجام شده و ارزیابی مدل روی داده‌های اعتبارسنجی توانسته تخمین قابل قبولی از عملکرد نهایی مدل ارائه دهد. در مجموع، می‌توان نتیجه گرفت که مدل پایه LeNet در این مسئله از تعادل مناسبی بین یادگیری و تعمیم‌پذیری برخوردار است، هرچند استفاده از تکنیک‌هایی مانند داده‌افزایی می‌تواند به کاهش بیشتر overfitting و بهبود عملکرد کمک کند.

برای تحلیل دقیق‌تر الگوی خطاها، ماتریس درهم‌ریختگی مدل در شکل زیر ارائه شده است.



*شکل ۲–۲: ماتریس درهم‌ریختگی برای مدل پایه **LeNet** بر روی داده‌های آزمون*

تمرکز مقادیر روی قطر اصلی ماتریس نشان‌دهنده‌ی عملکرد مناسب مدل در اکثر کلاس‌ها است و بیانگر دقت بالای مدل در تشخیص صحیح ارقام می‌باشد. با این حال، تعداد محدودی خطای ساختاری نیز مشاهده می‌شود؛ برای مثال، در برخی کلاس‌ها مانند رقم ۴ بیشترین خطا به‌صورت پیش‌بینی به‌عنوان ۹ دیده می‌شود. همچنین در کلاس‌هایی مانند ۵ و ۳ نیز اشتباهاتی وجود دارد (برای نمونه پیش‌بینی ۵ به‌صورت ۳ یا ۶)، و در برخی موارد دیگر مانند ۷ با ۲ یا ۹ نیز خطاهای پراکنده‌ای مشاهده می‌شود که می‌تواند ناشی از شباهت‌های ظاهری این ارقام در داده‌های دست‌نویس باشد.

به‌طور کلی، این الگو نشان می‌دهد که مدل در اغلب موارد عملکرد دقیقی دارد، اما همچنان در تمایز میان برخی کلاس‌های با شباهت بصری بالا با چالش‌های محدودی مواجه است.

در نهایت، گزارش کامل معیارهای ارزیابی برای هر کلاس در جدول زیر آورده شده است.

*جدول ۲–۲: گزارش کامل معیارهای **Precision**، **Recall** و **F1-score** برای هر کلاس در مدل پایه*

کلاس	Precision	Recall	F1-Score	Support
۰	۰.۹۸۹۸	۰.۹۹۲۹	۰.۹۹۱۳	۹۸۰
۱	۰.۹۹۴۷	۰.۹۹۴۷	۰.۹۹۴۷	۱۱۳۵
۲	۰.۹۸۶۵	۰.۹۹۱۳	۰.۹۸۸۹	۱۰۳۲
۳	۰.۹۸۹۱	۰.۹۸۶۱	۰.۹۸۷۶	۱۰۱۰
۴	۰.۹۹۱۸	۰.۹۸۳۷	۰.۹۸۷۷	۹۸۲
۵	۰.۹۹۰۹	۰.۹۸۰۹	۰.۹۸۵۹	۸۹۲
۶	۰.۹۸۵۵	۰.۹۹۲۷	۰.۹۸۹۱	۹۵۸
۷	۰.۹۸۹۳	۰.۹۸۶۴	۰.۹۸۷۸	۱۰۲۸
۸	۰.۹۸۵۷	۰.۹۹۱۸	۰.۹۸۸۷	۹۷۴
۹	۰.۹۸۳۲	۰.۹۸۵۱	۰.۹۸۴۲	۱۰۰۹
Macro Avg	۰.۹۸۸۷	۰.۹۸۸۶	۰.۹۸۸۶	۱۰۰۰۰
Weighted Avg	۰.۹۸۸۷	۰.۹۸۸۷	۰.۹۸۸۷	۱۰۰۰۰

- داده‌های تست: خطا=۰.۰۶۶۸ - دقت=۰.۹۸۸۷

مقادیر بالا و نسبتاً متعادل Precision و Recall در اکثر کلاس‌ها نشان می‌دهد که مدل در تشخیص اغلب ارقام عملکردی پایدار و قابل قبول دارد. با این حال، در برخی کلاس‌ها کاهش نسبی در این معیارها مشاهده می‌شود؛ به‌ویژه در کلاس ۵ که مقدار Recall پایین‌تری دارد، که نشان می‌دهد بخشی از نمونه‌های این کلاس به سایر کلاس‌ها نسبت داده شده‌اند. همچنین در کلاس‌هایی مانند ۹ و تا حدی ۳ نیز افت جزئی در Precision دیده می‌شود که می‌تواند ناشی از شباهت‌های ظاهری میان برخی ارقام دست‌نویس باشد.

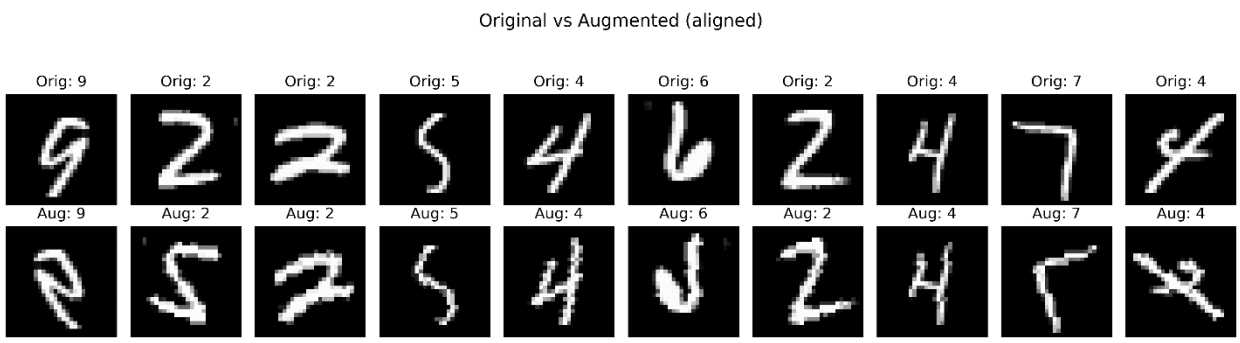
این نتایج نشان می‌دهند که اگرچه مدل پایه عملکرد کلی قوی‌ای دارد، اما همچنان در تمایز میان برخی کلاس‌های با شباهت بصری بالا با چالش‌هایی مواجه است. بنابراین، استفاده از روش‌هایی مانند داده‌افزایی می‌تواند به بهبود بیشتر تعمیم‌پذیری و کاهش این خطاها کمک کند.

۲-۴-۲- داده‌افزایی کلاسیک

در این بخش، با هدف افزایش تنوع داده‌های آموزشی و اعمال قیود ناوردایی مناسب، از تبدیلاتی نظیر چرخش تصادفی تا ۲۰ درجه و وارونگی افقی استفاده شده است. ایده‌ی اصلی در این نوع داده‌افزایی آن است که مدل نسبت به تغییرات جزئی در نمایش

ظاهری داده‌ها ناوردا شود و در نتیجه، به‌جای تکیه بر ویژگی‌های سطحی، ساختارهای پایدارتر را بیاموزد. با این حال، نکته‌ی مهم این است که اعتبار این فرض ناوردایی کاملاً وابسته به ماهیت مسئله است و هر تبدیل لزوماً انتخابی ایمن محسوب نمی‌شود.

نمونه‌هایی از تصاویر تولیدشده در اثر این تبدیلات در شکل زیر آورده شده‌اند.



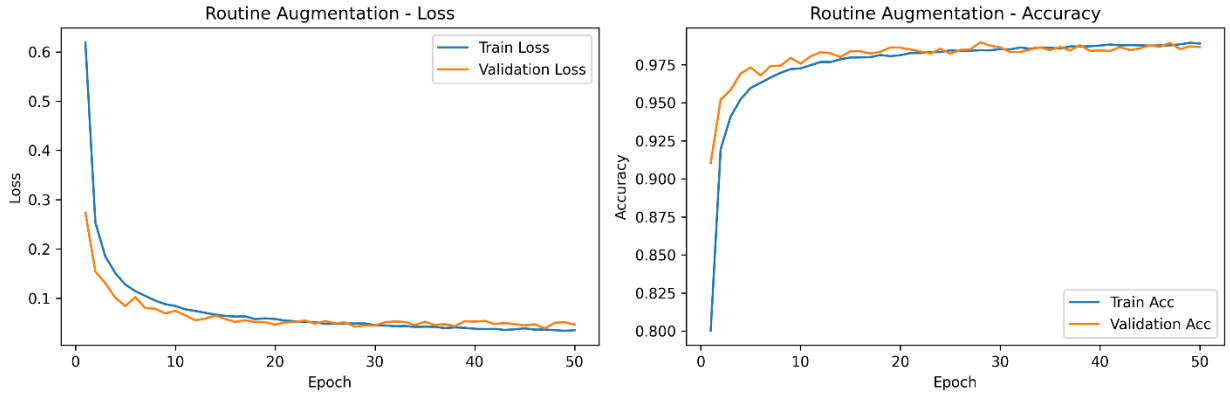
شکل ۲–۳: نمونه‌هایی از تصاویر اصلی و داده‌افزایی‌شده
ردیف بالا: تصاویر اصلی؛ ردیف پایین: نسخه‌های داده‌افزایی‌شده با اعمال چرخش و وارونگی

بررسی این نمونه‌ها نشان می‌دهد که چرخش‌های کوچک عموماً ساختار کلی ارقام را حفظ می‌کنند و تنها تغییراتی در جهت‌گیری آن‌ها ایجاد می‌شود، که با ماهیت داده‌های دست‌نویس سازگار است. اما در مورد وارونگی افقی، وضعیت متفاوت است. برخلاف اشیای طبیعی که معمولاً نسبت به بازتاب افقی ناوردا هستند، ارقام دست‌نویس لزوماً چنین خاصیتی ندارند. به‌عنوان مثال، برخی ارقام مانند «۲»، «۵» یا «۶» پس از وارونگی می‌توانند به اشکالی تبدیل شوند که یا به کلاس دیگری شباهت پیدا می‌کنند یا حتی از نظر بصری مبهم می‌شوند. در چنین شرایطی، فرض ناوردایی

$$f(x) = f(T(x)) \tag{۴}$$

برای تبدیل flip لزوماً برقرار نیست. بنابراین، این تبدیل می‌تواند به‌جای کمک به مدل، نوعی ابهام ساختاری در داده‌ها ایجاد کند.

اثر اعمال این تبدیلات بر رفتار مدل را می‌توان با بررسی روند آموزش بهتر تحلیل و تبیین کرد.

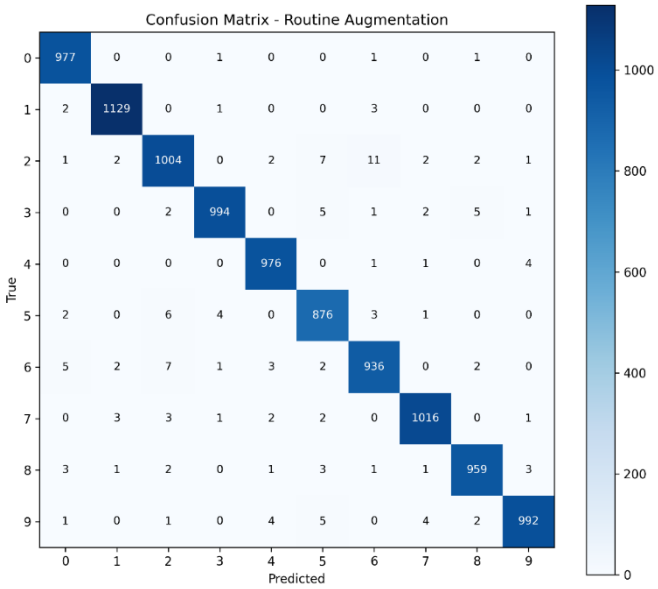


*شکل ۲–۴: تغییرات مقدار **Loss** و **Accuracy** در داده‌های آموزش و اعتبارسنجی در حضور داده‌افزایی کلاسیک*

در این نمودار، مدل نسبت به حالت پایه با سرعت کمتری به مقادیر پایین در Train Loss همگرا می‌شود و مقدار نهایی آن نیز اندکی بالاتر باقی می‌ماند، در حالی که فاصله‌ی میان Train و Validation کاهش یافته و این دو روند به یکدیگر نزدیک‌تر هستند. این رفتار نشان می‌دهد که داده‌افزایی باعث شده مسئله‌ی یادگیری برای مدل پیچیده‌تر شود و در نتیجه، مدل کمتر به سمت حفظ کردن داده‌ها میل کند. به بیان دیگر، داده‌افزایی نوعی regularization ضمنی ایجاد کرده که شدت overfitting را کاهش داده است. این اثر را می‌توان به‌صورت افزایش بایاس مفید در مدل تفسیر کرد؛ به این معنا که مدل به سمت کلاس محدودتری از توابع هدایت شده که با ناوردایی‌های اعمال‌شده سازگار هستند.

در عین حال، باید توجه داشت که این تغییر رفتار لزوماً به معنای بهبود مطلق عملکرد نیست. به‌ویژه، وجود تبدیل flip می‌تواند در برخی نمونه‌ها داده‌هایی تولید کند که از توزیع واقعی فاصله دارند. با این حال، به‌نظر می‌رسد مدل در حضور این تغییرات، به تعادل نسبی میان یادگیری الگوهای اصلی و مقابله با نویز ساختاری دست یافته است.

در ادامه، برای بررسی دقیق‌تر الگوی خطاها، ماتریس درهم‌ریختگی ارائه می‌شود.



شکل ۲–۵: ماتریس درهم‌ریختگی مدل آموزش‌دیده با داده‌افزایی کلاسیک

تحلیل ماتریس درهم‌ریختگی نشان می‌دهد که اگرچه خطاهای میان برخی کلاس‌های مشابه همچنان وجود دارند، اما این خطاها پراکنده‌تر شده‌اند و در چندین جهت مختلف توزیع شده‌اند. به‌عنوان مثال، در کلاس‌هایی مانند ۲ و ۶ یا ۵ و ۲ میزان خطا نسبتاً بیشتر و در چند کلاس مختلف پخش شده است. این الگو نشان می‌دهد که مدل کمتر به الگوهای خاص و محدود داده‌های آموزشی وابسته است و در عوض، نمایش‌های عمومی‌تری از کلاس‌ها آموخته است، هرچند این موضوع با افزایش پراکندگی خطاها نیز همراه بوده است.

در نهایت، گزارش طبقه‌بندی مدل در جدول زیر ارائه شده است.

جدول ۲–۳: گزارش کامل معیارهای **Precision**، **Recall** و **F1-score** برای مدل آموزش‌دیده با داده‌افزایی کلاسیک

کلاس	Precision	Recall	F1-Score	Support
۰	۰.۹۸۵۹	۰.۹۹۶۹	۰.۹۹۱۴	۹۸۰
۱	۰.۹۹۳۰	۰.۹۹۴۷	۰.۹۹۳۸	۱۱۳۵
۲	۰.۹۷۹۵	۰.۹۷۲۹	۰.۹۷۶۲	۱۰۳۲
۳	۰.۹۹۲۰	۰.۹۸۴۲	۰.۹۸۸۱	۱۰۱۰
۴	۰.۹۸۷۹	۰.۹۹۳۹	۰.۹۹۰۹	۹۸۲
۵	۰.۹۷۳۳	۰.۹۸۲۱	۰.۹۷۷۷	۸۹۲
۶	۰.۹۷۸۱	۰.۹۷۷۰	۰.۹۷۷۵	۹۵۸
۷	۰.۹۸۹۳	۰.۹۸۸۳	۰.۹۸۸۸	۱۰۲۸
۸	۰.۹۸۷۶	۰.۹۸۴۶	۰.۹۸۶۱	۹۷۴
۹	۰.۹۹۰۰	۰.۹۸۳۲	۰.۹۸۶۶	۱۰۰۹
Macro Avg	۰.۹۸۵۷	۰.۹۸۵۸	۰.۹۸۵۷	۱۰۰۰۰
Weighted Avg	۰.۹۸۵۹	۰.۹۸۵۹	۰.۹۸۵۹	۱۰۰۰۰

- داده‌های تست: خطا=۰.۰۴۸۷ – دقت=۰.۹۸۵۹

به‌طور کلی، نتایج نشان می‌دهند که داده‌افزایی کلاسیک، حتی با وجود برخی تبدیلات بالقوه پرریسک مانند flip، منجر به بهبود قابل توجهی در عملکرد نهایی مدل نشده و در مقایسه با حالت پایه (با دقت ۰.۹۸۸۷)، دقت مدل اندکی کاهش یافته و به حدود ۰.۹۸۵۹ رسیده است. این موضوع نشان می‌دهد که اگرچه داده‌افزایی باعث افزایش تنوع داده‌ها و اعمال نوعی regularization شده، اما هم‌زمان برخی تبدیلات نامتناسب می‌توانند اطلاعات گمراه‌کننده‌ای به مدل وارد کنند.

با این حال، این تحلیل تأکید می‌کند که انتخاب تبدیلات باید با دقت و بر اساس ماهیت مسئله انجام شود، زیرا اعمال ناوردایی‌های نامعتبر می‌تواند به‌جای بهبود، منجر به افت عملکرد مدل شود.

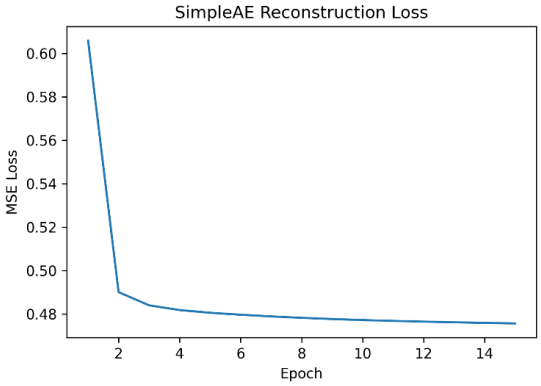
۲-۴-۳- داده‌افزایی مبتنی بر Autoencoder

در این بخش، از یک Autoencoder ساده کانولوشنی برای تولید نمونه‌های جدید استفاده شده است. به‌طور خلاصه، Autoencoder مدلی است که ورودی را به یک نمایش فشرده[15] نگاشت کرده و سپس تلاش می‌کند همان ورودی را بازسازی

[15] Latent

کند. در این فرآیند، مدل ناچار است ساختارهای اصلی داده را یاد بگیرد. برای داده‌افزایی، از خروجی این مدل (یعنی تصاویر بازسازی‌شده) در کنار داده‌های اصلی استفاده می‌شود تا مجموعه‌ی آموزشی گسترش یابد.

ابتدا کیفیت یادگیری Autoencoder از طریق خطای بازسازی بررسی می‌شود.

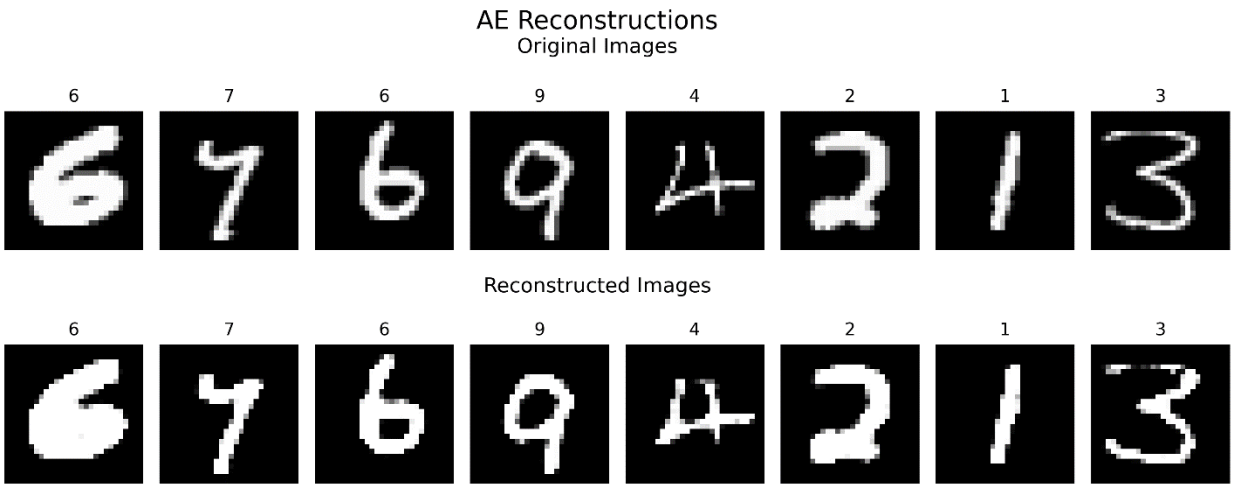


*شکل ۲–۶: روند کاهش خطای بازسازی (**MSE**) در طول آموزش **Autoencoder***

کاهش یکنواخت خطای بازسازی نشان می‌دهد که مدل توانسته است ساختار کلی داده‌ها را یاد بگیرد. با این حال، همگرایی به یک مقدار غیرصفر نسبتاً قابل توجه نشان می‌دهد که بازسازی‌ها کامل نیستند و بخشی از جزئیات از دست می‌رود. این نکته از منظر داده‌افزایی بسیار مهم است، زیرا نشان می‌دهد نمونه‌های تولیدشده دقیقاً از توزیع اصلی پیروی نمی‌کنند، بلکه نسخه‌ای هموارشده[۱۶] از آن هستند.

برای درک بهتر این موضوع، نمونه‌هایی از تصاویر بازسازی‌شده در شکل زیر آورده شده‌اند.

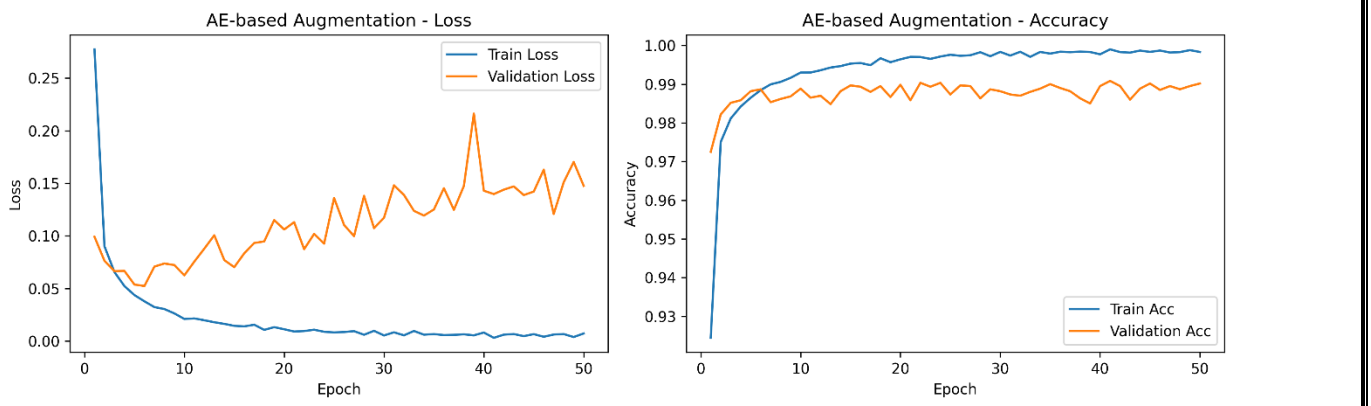
[16] Smoothed



شکل ۲–۷: تصاویر اصلی و بازسازی‌شده توسط ***Autoencoder***
ردیف بالا: تصاویر اصلی؛ ردیف پایین: تصاویر بازسازی‌شده توسط ***Autoencoder***

در این نمونه‌ها مشاهده می‌شود که Autoencoder توانسته ساختار کلی ارقام را به‌خوبی حفظ کند و بازسازی‌ها تا حد زیادی به نمونه‌های اصلی شباهت دارند، اما در عین حال، جزئیات ظریف مانند تیزی لبه‌ها و تغییرات دقیق در ضخامت خطوط تا حدی هموار شده‌اند. این رفتار تا حدی شبیه اعمال یک فیلتر low-pass بر روی داده‌ها است. در نتیجه، داده‌های بازسازی‌شده دارای نویز کمتر اما اطلاعات تمایزبخش نرم‌تر هستند. این موضوع یک مصالحه مهم ایجاد می‌کند: از یک سو، کاهش نویز می‌تواند به پایداری و تعمیم‌پذیری مدل کمک کند، اما از سوی دیگر، هموار شدن جزئیات ممکن است تمایز میان برخی کلاس‌های مشابه را اندکی دشوارتر کند.

در ادامه، رفتار مدل طبقه‌بندی که با این داده‌های افزوده‌شده آموزش دیده است بررسی می‌شود.

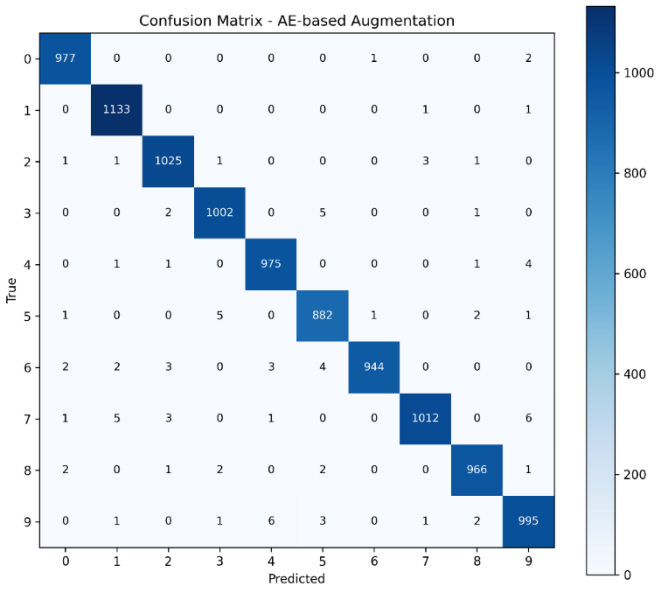


شکل ۲–۸: تغییرات مقدار ***Loss*** *و* ***Accuracy*** *در داده‌های آموزش و اعتبارسنجی در حضور داده‌افزایی مبتنی بر* ***Autoencoder***

یکی از نکات قابل توجه در این نمودار، افزایش تدریجی Validation Loss پس از چند epoch ابتدایی در حالی است که Accuracy تقریباً در سطح بالایی ثابت باقی می‌ماند. این رفتار نشان می‌دهد که مدل همچنان قادر به پیش‌بینی صحیح کلاس‌ها است، اما در پیش‌بینی‌های خود از اطمینان کمتری برخوردار شده است. این پدیده را می‌توان به تغییر توزیع داده‌های آموزشی نسبت داد؛ زیرا ترکیب داده‌های اصلی با نسخه‌های بازسازی‌شده و هموارشده باعث می‌شود مرزهای تصمیم‌گیری مدل کمتر شارپ باشند. به بیان دیگر، مدل به‌جای یادگیری مرزهای دقیق، به سمت نمایش‌های محافظه‌کارانه‌تر حرکت می‌کند که این امر مستقیماً در افزایش CrossEntropy Loss منعکس می‌شود.

این رفتار را می‌توان به‌عنوان نوعی کاهش کیفیت کالیبراسیون مدل نیز تفسیر کرد. یعنی اگرچه پیش‌بینی نهایی (argmax) همچنان در بسیاری از موارد صحیح است، اما توزیع احتمالات خروجی اطمینان کمتری را نشان می‌دهد. این موضوع بیان می‌کند که داده‌افزایی مبتنی بر Autoencoder می‌تواند دقت را حفظ کند، اما کیفیت اطمینان پیش‌بینی‌ها را تا حدی تضعیف کند.

برای بررسی الگوی خطاها، ماتریس درهم‌ریختگی در شکل زیر ارائه شده است.



شکل ۲–۹: ماتریس درهم‌ریختگی مدل آموزش‌دیده با داده‌افزایی مبتنی بر ***Autoencoder***

تحلیل این ماتریس نشان می‌دهد که الگوی کلی خطاها تا حد زیادی مشابه حالت پایه باقی مانده است، اما در برخی موارد، اشتباهات میان کلاس‌های مشابه اندکی افزایش یافته یا پراکندگی بیشتری پیدا کرده‌اند. به‌عنوان مثال، در برخی کلاس‌ها مانند ۴ و ۹ یا ۵ و ۳ و همچنین ۷ با ۹، خطاهای بیشتری نسبت به حالت پایه مشاهده می‌شود. این موضوع با تحلیل قبلی هم‌خوانی دارد؛ زیرا هموار شدن و کاهش جزئیات در تصاویر بازسازی‌شده می‌تواند تمایز میان این کلاس‌ها را دشوارتر کرده و منجر به افزایش این نوع خطاها شود.

در نهایت، گزارش طبقه‌بندی مدل در جدول زیر آورده شده است.

*جدول ۲–۴: گزارش کامل معیارهای **Precision**، **Recall** و **F1-score** برای مدل آموزش‌دیده با داده‌افزایی مبتنی بر **Autoencoder***

کلاس	Precision	Recall	F1-Score	Support
۰	۰.۹۹۲۹	۰.۹۹۶۹	۰.۹۹۴۹	۹۸۰
۱	۰.۹۹۱۳	۰.۹۹۸۲	۰.۹۹۴۷	۱۱۳۵
۲	۰.۹۹۰۳	۰.۹۹۳۲	۰.۹۹۱۸	۱۰۳۲
۳	۰.۹۹۱۱	۰.۹۹۲۱	۰.۹۹۱۶	۱۰۱۰
۴	۰.۹۸۹۸	۰.۹۹۲۹	۰.۹۹۱۴	۹۸۲
۵	۰.۹۸۴۴	۰.۹۸۸۸	۰.۹۸۶۶	۸۹۲
۶	۰.۹۹۷۹	۰.۹۸۵۴	۰.۹۹۱۶	۹۵۸
۷	۰.۹۹۵۱	۰.۹۸۴۴	۰.۹۸۹۷	۱۰۲۸
۸	۰.۹۹۲۸	۰.۹۹۱۸	۰.۹۹۲۳	۹۷۴
۹	۰.۹۸۵۱	۰.۹۸۶۱	۰.۹۸۵۶	۱۰۰۹
Macro Avg	۰.۹۹۱۱	۰.۹۹۱۰	۰.۹۹۱۰	۱۰۰۰۰
Weighted Avg	۰.۹۹۱۱	۰.۹۹۱۱	۰.۹۹۱۱	۱۰۰۰۰

- داده‌های تست: خطا=۰.۱۱۴۱ – دقت=۰.۹۹۱۱

به‌طور کلی، نتایج نشان می‌دهند که داده‌افزایی مبتنی بر Autoencoder، در این آزمایش نه‌تنها باعث افت عملکرد نشده، بلکه نسبت به مدل پایه (با دقت ۰.۹۸۸۷) بهبود جزئی‌ای نیز ایجاد کرده و دقت را به حدود ۰.۹۹۱۱ رسانده است، هرچند مقدار Test Loss افزایش یافته است. این رفتار نشان می‌دهد که داده‌های بازسازی‌شده اگرچه تنوع کاملاً جدیدی ایجاد نمی‌کنند و بیشتر نسخه‌های هموارشده‌ای از داده‌های موجود هستند، اما می‌توانند با کاهش نویز و ایجاد نوعی regularization، به بهبود تصمیم‌گیری مدل کمک کنند.

با این حال، همچنان این نکته برقرار است که این نوع داده‌افزایی لزوماً اطلاعات جدیدی به مدل اضافه نمی‌کند و حتی ممکن است برخی ویژگی‌های تمایزبخش را تضعیف کند. بنابراین، این تحلیل نشان می‌دهد که در استفاده از مدل‌های مولد برای داده‌افزایی، کیفیت نمونه‌های تولیدی و میزان حفظ یا از دست رفتن جزئیات، نقش تعیین‌کننده‌ای در نتیجه نهایی دارند..

۲-۴-۴- داده‌افزایی مبتنی بر MixUp

در این بخش، از روش MixUp به‌عنوان یک رویکرد پیشرفته داده‌افزایی استفاده شده است. برخلاف روش‌هایی که تغییراتی روی یک نمونه‌ی منفرد اعمال می‌کنند، MixUp با ترکیب دو نمونه‌ی متفاوت در فضای ورودی و برچسب، داده‌های جدید تولید می‌کند.

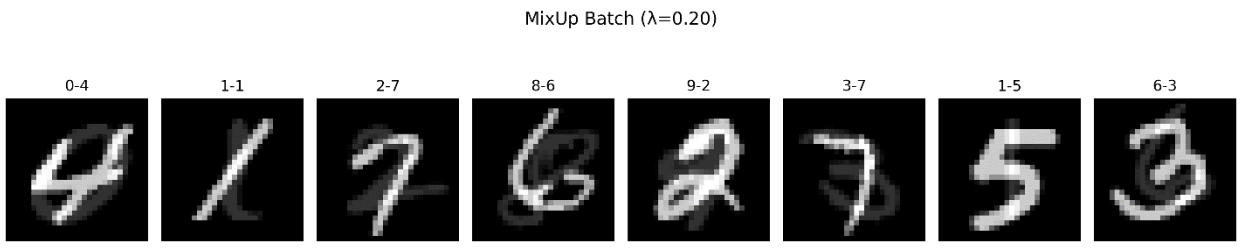
به‌صورت نظری، در MixUp دو نمونه (x_i, y_i) و (x_j, y_j) انتخاب شده و نمونه‌ی جدید به شکل زیر ساخته می‌شود:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j \quad (۵)$$

$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j \quad (۶)$$

که در آن λ از یک توزیع Beta نمونه‌برداری می‌شود. در این چارچوب، داده‌های جدید در واقع نقاطی در فضای بین نمونه‌های واقعی هستند و نه صرفاً تغییرات محلی روی یک داده. ایده‌ی کلیدی MixUp این است که مدل را وادار کند تا در این فضای بینابینی رفتار خطی و هموار داشته باشد. این قید، از شکل‌گیری مرزهای تصمیم‌گیری پیچیده و ناپایدار جلوگیری کرده و نقش یک regularization قوی را ایفا می‌کند.

نمونه‌هایی از داده‌های تولیدشده با این روش در شکل زیر آورده شده‌اند.



*شکل ۲–۱۰: نمونه‌هایی از تصاویر ترکیبی تولیدشده با **MixUp**؛ هر تصویر ترکیبی از دو کلاس مختلف با نسبت λ مشخص است.*

این تصاویر در نگاه اول ممکن است غیرطبیعی یا حتی مبهم به‌نظر برسند، زیرا ویژگی‌های دو کلاس متفاوت را به‌طور هم‌زمان در خود دارند و مرز مشخصی میان آن‌ها دیده نمی‌شود. با این حال، این «غیرطبیعی بودن» بخشی از مکانیزم اصلی MixUp است. هدف این روش تولید داده‌های واقع‌گرایانه نیست، بلکه کاهش وابستگی مدل به الگوهای بسیار خاص در داده‌های آموزشی است. در حالت معمول، مدل ممکن است یاد بگیرد که هر کلاس را تنها بر اساس چند ویژگی مشخص و ثابت تشخیص دهد، اما در MixUp، چون هر نمونه ترکیبی از دو کلاس است، مدل دیگر نمی‌تواند به این الگوهای محدود تکیه کند. در عوض، مجبور می‌شود ویژگی‌های کلی‌تر و مشترک‌تری را یاد بگیرد و تصمیم‌گیری خود را بر اساس میزان حضور هر کلاس در تصویر انجام دهد. این موضوع باعث می‌شود مرز بین کلاس‌ها به‌جای حالت تیز و قطعی، نرم‌تر و پیوسته‌تر شود.

رفتار مدل در حضور این نوع داده‌افزایی در شکل زیر قابل مشاهده است.



*شکل ۲–۱۱: تغییرات مقدار **Loss** و **Accuracy** در داده‌های آموزش و آزمون در حضور داده‌افزایی مبتنی بر **MixUp***

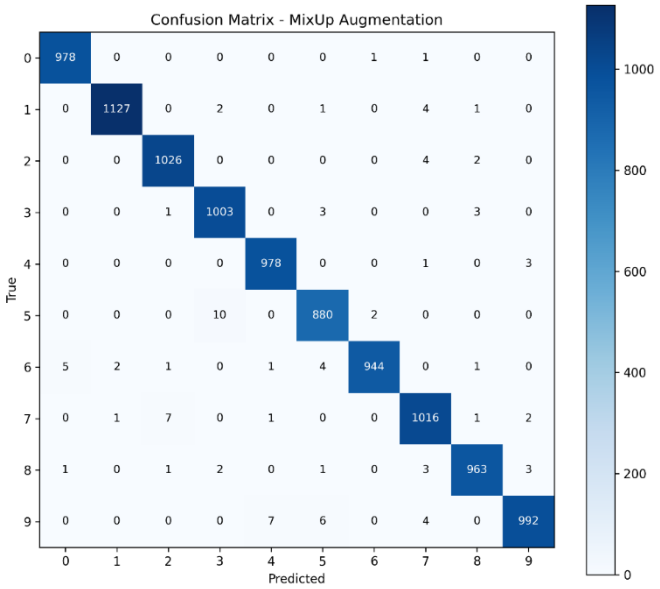
یکی از نکات مهم در این نمودار، عدم نمایش Train Accuracy است. دلیل این موضوع آن است که در MixUp، برچسب‌ها به‌صورت ترکیبی و نرم تعریف می‌شوند و دیگر یک برچسب قطعی برای هر نمونه وجود ندارد. در نتیجه، معیار دقت به شکل معمول (بر اساس مقایسه‌ی argmax با یک برچسب صحیح) تعریف‌پذیر نیست یا حداقل تفسیر مستقیمی ندارد. به همین دلیل، گزارش دقت آموزش در این حالت می‌تواند گمراه‌کننده باشد و از ارائه‌ی آن صرف‌نظر شده است.

همچنین باید توجه داشت که MixUp تنها در مرحله‌ی آموزش اعمال می‌شود و داده‌های آزمون به همان صورت اصلی و بدون تغییر باقی می‌مانند. این نکته اهمیت زیادی در تفسیر نتایج دارد، زیرا عملکرد مدل در داده‌های تست نشان‌دهنده‌ی تعمیم‌پذیری واقعی آن به داده‌های طبیعی است، نه داده‌های ترکیبی.

از منظر تحلیلی، مقدار نسبتاً بالای Train Loss در این روش کاملاً قابل انتظار است. از آنجا که مدل با برچسب‌های نرم آموزش می‌بیند، حتی در بهترین حالت نیز نمی‌تواند به پیش‌بینی‌های کاملاً قطعی برسد، و در نتیجه مقدار loss به‌طور ذاتی بالاتر باقی می‌ماند. بنابراین، مقایسه‌ی مستقیم مقدار loss در این حالت با سایر روش‌ها چندان معنادار نیست.

در مقابل، روند کاهشی Validation Loss و پایداری Validation Accuracy در مقادیر بالا نشان می‌دهد که مدل توانسته تعمیم‌پذیری مناسبی کسب کند. این رفتار را می‌توان به اثر ساختاری MixUp نسبت داد: با اعمال قید خطی بودن، مدل از یادگیری مرزهای تصمیم‌گیری بیش‌ازحد پیچیده یا وابسته به نویز بازداشته می‌شود و در نتیجه، روی داده‌های دیده‌نشده عملکرد پایدارتر و قابل‌اعتمادتری دارد. به بیان دیگر، MixUp با افزایش بایاس مفید، به کاهش واریانس مدل کمک می‌کند.

برای تحلیل دقیق‌تر الگوی خطاها، ماتریس درهم‌ریختگی در شکل زیر ارائه شده است.



شکل ۲–۱۲: ماتریس درهم‌ریختگی مدل آموزش‌دیده با داده‌افزایی مبتنی بر ***MixUp***

این ماتریس نشان می‌دهد که خطاها نسبت به حالت‌های قبلی همچنان در سطح پایینی قرار دارند و در چندین جهت به‌صورت پراکنده توزیع شده‌اند. در برخی کلاس‌ها مانند ۵ و ۹ هنوز اشتباهاتی به کلاس‌های نزدیک (مثلاً ۵ با ۳ یا ۹ با ۴ و ۵) دیده می‌شود، اما این خطاها ساختار متمرکز و شدیدی ندارند. این الگو نشان می‌دهد که مدل کمتر به الگوهای خاص وابسته است و توانسته نمایش‌های پایدارتری از ویژگی‌ها یاد بگیرد، هرچند همچنان در تمایز میان برخی کلاس‌های با شباهت بصری بالا با چالش‌های محدودی مواجه است.

در نهایت، گزارش طبقه‌بندی مدل در جدول زیر ارائه شده است.

جدول ۲–۵: گزارش کامل معیارهای **Precision**، **Recall** و **F1-score** برای مدل آموزش‌دیده با **MixUp**

کلاس	Precision	Recall	F1-Score	Support
۰	۰.۹۹۳۹	۰.۹۹۸۰	۰.۹۹۵۹	۹۸۰
۱	۰.۹۹۷۳	۰.۹۹۳۰	۰.۹۹۵۱	۱۱۳۵
۲	۰.۹۹۰۳	۰.۹۹۴۲	۰.۹۹۲۳	۱۰۳۲
۳	۰.۹۸۶۲	۰.۹۹۳۱	۰.۹۸۹۶	۱۰۱۰
۴	۰.۹۹۰۹	۰.۹۹۵۹	۰.۹۹۳۴	۹۸۲
۵	۰.۹۸۳۲	۰.۹۸۶۵	۰.۹۸۴۹	۸۹۲
۶	۰.۹۹۶۸	۰.۹۸۵۴	۰.۹۹۱۱	۹۵۸
۷	۰.۹۸۳۵	۰.۹۸۸۳	۰.۹۸۵۹	۱۰۲۸
۸	۰.۹۹۱۸	۰.۹۸۸۷	۰.۹۹۰۲	۹۷۴
۹	۰.۹۹۲۰	۰.۹۸۳۲	۰.۹۸۷۶	۱۰۰۹
Macro Avg	۰.۹۹۰۶	۰.۹۹۰۶	۰.۹۹۰۶	۱۰۰۰۰
Weighted Avg	۰.۹۹۰۷	۰.۹۹۰۷	۰.۹۹۰۷	۱۰۰۰۰

در مجموع، MixUp را می‌توان به‌عنوان یکی از روش‌های مؤثر داده‌افزایی در این آزمایش در نظر گرفت. این روش نسبت به مدل پایه (با دقت ۰.۹۸۸۷) بهبود ایجاد کرده و دقت را به حدود ۰.۹۹۰۷ رسانده و Test Loss را نیز کاهش داده است. برخلاف روش‌های مبتنی بر تبدیلات یا بازسازی، MixUp مستقیماً ساختار فضای داده را تغییر داده و با اعمال یک قید صریح بر رفتار مدل، آن را به سمت یادگیری نمایش‌های هموارتر و پایدارتر هدایت می‌کند.

۲-۵- نکات پیاده‌سازی و ملاحظات عملی

در این بخش، برخی از اجزای مهم پیاده‌سازی که نقش کلیدی در رفتار نهایی مدل دارند بررسی می‌شوند. هدف، تمرکز بر جزئیاتی است که ممکن است در نگاه اول ساده به نظر برسند، اما در عمل تأثیر قابل‌توجهی بر عملکرد و تفسیر نتایج دارند.

۲-۵-۱- تولید داده در MixUp

در این بخش، نحوه‌ی تولید نمونه‌های ترکیبی در روش MixUp بررسی می‌شود. این مرحله هسته‌ی اصلی این روش است و نحوه‌ی نمونه‌برداری و ترکیب داده‌ها می‌تواند مستقیماً بر کیفیت regularization اعمال‌شده تأثیر بگذارد.

```python
def mixup_data(x, y, alpha=0.4):
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1.0
    batch_size = x.size(0)
    index = torch.randperm(batch_size).to(device)
    mixed_x = lam * x + (1-lam) * x[index, :]
    y_a, y_b = y, y[index]
    return mixed_x, y_a, y_b, lam
```

*شکل ۲–۱۳: تابع تولید داده‌های ترکیبی در روش **MixUp** بر اساس نمونه‌برداری از توزیع **Beta** و جایگشت تصادفی در **batch***

در این پیاده‌سازی، ضریب ترکیب λ از توزیع Beta با پارامتر α نمونه‌برداری می‌شود. این انتخاب بسیار مهم است، زیرا شکل توزیع Beta تعیین می‌کند که داده‌های تولیدشده تا چه حد به یکی از نمونه‌ها نزدیک باشند یا ترکیبی متعادل از هر دو باشند. برای مثال، اگر α کوچک باشد، مقدار λ اغلب به ۰ یا ۱ نزدیک خواهد بود و در نتیجه نمونه‌های تولیدشده شباهت زیادی به یکی از داده‌های اصلی خواهند داشت. در مقابل، مقادیر بزرگ‌تر α باعث تولید نمونه‌های واقعاً ترکیبی می‌شوند که هر دو داده را به‌طور هم‌زمان منعکس می‌کنند.

استفاده از یک جایگشت تصادفی روی batch نیز نکته‌ی مهمی است. این کار باعث می‌شود هر نمونه با یک نمونه‌ی دیگر به‌صورت تصادفی ترکیب شود و از ایجاد الگوهای ثابت در ترکیب داده‌ها جلوگیری شود. در غیر این صورت، ممکن بود مدل به روابط خاصی بین نمونه‌های مشخص عادت کند که این امر با هدف اصلی MixUp در تضاد است.

نکته‌ی قابل توجه دیگر این است که ترکیب در فضای ورودی انجام می‌شود، نه در فضای ویژگی‌های استخراج‌شده. این موضوع باعث می‌شود که مدل از همان ابتدای شبکه با داده‌های بینابینی مواجه شود و در نتیجه، کل فرآیند یادگیری تحت تأثیر این قید خطی قرار گیرد.

همچنین، در این مرحله برچسب‌ها به‌صورت جفت (y_a, y_b) نگهداری می‌شوند و ترکیب آن‌ها به مرحله‌ی محاسبه‌ی loss موکول می‌شود. این طراحی باعث می‌شود انعطاف‌پذیری بیشتری در تعریف تابع هزینه وجود داشته باشد و از تبدیل زودهنگام برچسب‌ها به یک نمایش خاص جلوگیری شود.

۲-۵-۲- تعریف تابع هزینه در MixUp

در این بخش، نحوه‌ی اعمال MixUp در تابع هزینه بررسی می‌شود. برخلاف حالت استاندارد که در آن یک برچسب قطعی برای هر نمونه وجود دارد، در اینجا مدل باید هم‌زمان دو برچسب را با ضرایب متفاوت در نظر بگیرد.

```python
def mixup_criterion(criterion, pred, y_a, y_b, lam):
    return lam * criterion(pred, y_a) + (1-lam) * criterion(pred, y_b)
```

*شکل ۲-۱۴: تعریف تابع هزینه در **MixUp** به‌صورت ترکیب خطی از **loss** مربوط به دو برچسب*

در این پیاده‌سازی، مقدار loss به‌صورت ترکیب خطی از دو مقدار CrossEntropy محاسبه می‌شود:

$$\mathcal{L} = \lambda \mathcal{L}\big(f(x), y_a\big) + (1-\lambda)\mathcal{L}(f(x), y_b) \quad (۷)$$

این رابطه نشان می‌دهد که مدل به‌جای یادگیری یک نگاشت قطعی، در حال یادگیری توزیعی از برچسب‌ها است. به بیان دیگر، خروجی مدل باید به‌گونه‌ای باشد که هم‌زمان با هر دو برچسب سازگار باشد، متناسب با سهم آن‌ها در نمونه‌ی ورودی.

یکی از نکات مهم در این طراحی این است که نیازی به ساخت صریح برچسب‌های soft به‌صورت بردارهای one-hot ترکیب‌شده وجود ندارد. در عوض، ترکیب در سطح تابع هزینه انجام می‌شود که هم از نظر محاسباتی ساده‌تر است و هم از بروز خطاهای عددی احتمالی جلوگیری می‌کند.

از منظر تحلیلی، این نوع تعریف loss باعث می‌شود که مدل از تولید پیش‌بینی‌های بیش‌ازحد مطمئن فاصله بگیرد. زیرا برای هر نمونه، بیش از یک برچسب معتبر وجود دارد و مدل نمی‌تواند تمام احتمال را به یک کلاس اختصاص دهد. این ویژگی به‌طور مستقیم به بهبود کالیبراسیون مدل و کاهش overfitting کمک می‌کند.

همچنین، این ساختار توضیح می‌دهد که چرا مقدار loss در MixUp معمولاً بالاتر از حالت استاندارد باقی می‌ماند؛ زیرا حتی در بهترین حالت نیز مدل باید بین چند برچسب تعادل برقرار کند و نمی‌تواند به پیش‌بینی‌های کاملاً قطعی برسد.

۲-۶- نتیجه‌گیری

در این فصل، روش‌های مختلف داده‌افزایی از منظر تأثیر بر تعمیم‌پذیری مدل مورد بررسی قرار گرفتند. نتایج نهایی در جدول زیر به‌صورت یکپارچه آورده شده‌اند.

جدول ۲–۶: مقایسه عملکرد نهایی مدل تحت روش‌های مختلف داده‌افزایی

روش	Test Accuracy	Test Loss
Baseline LeNet	۰.۹۸۸۷	۰.۰۶۶۸
Routine Augmentation (Transformations)	۰.۹۸۵۹	۰.۰۴۸۷
AE-based Augmentation	۰.۹۹۱۱	۰.۱۱۴۱
MixUp Augmentation	۰.۹۹۰۷	۰.۰۴۲۶

به‌طور کلی، نتایج نشان می‌دهند که نوع داده‌افزایی اهمیت بیشتری از صرفاً افزایش حجم داده دارد. داده‌افزایی کلاسیک با اعمال تبدیلات ساده اگرچه باعث کاهش قابل توجه loss شده، اما با افت جزئی در دقت همراه بوده است. این موضوع نشان می‌دهد که ایجاد ناورداهای هندسی می‌تواند به منظم‌تر شدن یادگیری کمک کند، اما لزوماً به بهبود دقت منجر نمی‌شود.

در مقابل، داده‌افزایی مبتنی بر Autoencoder با هموارسازی داده‌ها و حذف نویز، توانسته دقت بالاتری ایجاد کند، اما این کار با افزایش loss همراه بوده است. این رفتار نشان می‌دهد که اگرچه تصمیم نهایی مدل در بسیاری از موارد صحیح است، اما اطمینان آن در پیش‌بینی‌ها کاهش یافته و توزیع احتمالات خروجی نرم‌تر شده است.

در نهایت، MixUp با اعمال یک قید خطی در فضای داده و برچسب، نوعی regularization مؤثر ایجاد می‌کند که منجر به کاهش loss و بهبود پایداری مدل می‌شود. این روش مدل را به یادگیری نمایش‌های هموارتر و کاهش وابستگی به الگوهای خاص وادار می‌کند.

در مجموع، نمی‌توان یک روش را به‌صورت مطلق برتر دانست، زیرا هرکدام جنبه‌ی متفاوتی از یادگیری را بهبود می‌دهند. اگر هدف بیشینه‌سازی دقت باشد، روش‌های مبتنی بر بازسازی مانند Autoencoder می‌توانند مناسب‌تر باشند، در حالی که برای دستیابی به پیش‌بینی‌های پایدارتر، کالیبره‌تر و با اطمینان بیشتر، روش‌هایی مانند MixUp انتخاب منطقی‌تری محسوب می‌شوند.

فصل سوم

تطبیق دامنه[17] و تعمیم دامنه[18]

در این فصل به بررسی اثر اختلاف توزیع بین داده‌های آموزش و آزمون[19] بر عملکرد مدل‌ها پرداخته و راهکارهای مقابله با آن را مطالعه می‌کنیم. ابتدا به تحلیل مفهومی این پدیده از منظر بایاس و واریانس می‌پردازیم و سپس با پیاده‌سازی و مقایسه چندین روش تطبیق دامنه، تلاش می‌کنیم عملکرد مدل را در شرایطی که داده‌های هدف بدون برچسب هستند بهبود دهیم. در نهایت، اثر این روش‌ها بر رفتار مدل و الگوی خطاها به‌صورت تجربی تحلیل می‌شود.

۳-۱- تحلیل اثر Domain Shift بر بایاس، واریانس و تعمیم‌پذیری مدل

در بسیاری از روش‌های یادگیری ماشین، فرض اساسی این است که داده‌های آموزش و آزمون از یک توزیع مشترک نمونه‌برداری شده‌اند. با این حال، در مسائل واقعی این فرض اغلب نقض شده و پدیده‌ای تحت عنوان Domain Shift رخ می‌دهد، که در آن توزیع داده‌های آزمون با داده‌های آموزش متفاوت است. در چنین شرایطی، حتی اگر مدل به‌خوبی روی داده‌های آموزش عمل کند، ممکن است در مواجهه با داده‌های جدید دچار افت عملکرد قابل توجهی شود.

کاهش تعمیم‌پذیری در اثر Domain Shift را نمی‌توان صرفاً به افزایش واریانس نسبت داد؛ بلکه عامل اصلی آن افزایش بایاس مؤثر مدل نسبت به توزیع جدید است. مدل در فرآیند آموزش، نگاشتی را برای تقریب تابع هدف تحت توزیع داده‌های آموزش یاد می‌گیرد. زمانی که این توزیع تغییر می‌کند، نگاشت یادگرفته‌شده دیگر نمایش مناسبی از رابطه‌ی واقعی بین ورودی و خروجی در دامنه جدید نیست، و در نتیجه خطای سیستماتیک (Bias) افزایش می‌یابد.

برای درک بهتر این موضوع، یک مثال کاربردی در نظر بگیرید. فرض کنید مدلی برای تشخیص اشیاء بر روی تصاویر گرفته‌شده در شرایط نوری مناسب و با دوربین‌های باکیفیت آموزش داده شده است. حال اگر این مدل را روی تصاویری که در شرایط نور کم یا با دوربین‌های ضعیف‌تر ثبت شده‌اند اعمال کنیم، عملکرد آن به‌طور محسوسی کاهش می‌یابد. در اینجا، ساختار کلی اشیاء تغییر نکرده است، اما ویژگی‌های آماری تصویر (مانند روشنایی، کنتراست و نویز) متفاوت شده‌اند. از آنجا که مدل ویژگی‌های خاصی را تحت توزیع اولیه یاد گرفته است، در دامنه جدید این ویژگی‌ها دیگر به‌درستی نمایانگر کلاس‌ها نیستند. بنابراین، مدل دچار خطای سیستماتیک می‌شود که نشان‌دهنده افزایش بایاس است.

[17] Domain Adaptation
[18] Domain Generalization
[19] Domain Shift

در مقابل، واریانس به میزان حساسیت مدل نسبت به نوسانات داده‌های آموزش اشاره دارد. اگر مدل دارای واریانس بالا باشد، تغییرات کوچک در داده‌های آموزش می‌تواند به تغییرات قابل توجهی در خروجی مدل منجر شود. با این حال، در سناریوی Domain Shift، حتی اگر مدل دارای واریانس پایین باشد و رفتار پایداری از خود نشان دهد، همچنان به دلیل عدم تطابق بین توزیع‌ها دچار افت عملکرد خواهد شد. به بیان دیگر، مشکل اصلی در اینجا ناپایداری مدل نیست، بلکه عدم انطباق بین توزیع داده‌های آموزش و آزمون است.

در نتیجه، می‌توان گفت که Domain Shift عمدتاً منجر به افزایش بایاس مدل نسبت به دامنه جدید شده و این افزایش بایاس عامل اصلی کاهش تعمیم‌پذیری است، در حالی که واریانس نقش ثانویه‌ای در این پدیده ایفا می‌کند.

۳-۲- مقایسه تطبیق دامنه و تعمیم دامنه و بررسی فرضیات مربوط به دامنه هدف

تطبیق دامنه و تعمیم دامنه هر دو با هدف مقابله با افت عملکرد ناشی از اختلاف توزیع میان داده‌های آموزش و آزمون مطرح می‌شوند، اما در فرضیات و میزان دشواری مسئله تفاوت اساسی دارند.

در تطبیق دامنه فرض می‌شود که در زمان آموزش، داده‌هایی از دامنه هدف در دسترس هستند، هرچند این داده‌ها معمولاً بدون برچسب‌اند. بنابراین مدل می‌تواند با مشاهده مستقیم ساختار آماری دامنه هدف، خود را با آن سازگار کند؛ برای مثال از طریق هم‌تراز کردن توزیع ویژگی‌ها یا یادگیری نمایش‌های مشترک بین دو دامنه. در مقابل، در تعمیم دامنه هیچ داده‌ای از دامنه هدف در اختیار مدل قرار ندارد و مدل باید صرفاً بر اساس دامنه‌های منبع، به‌گونه‌ای آموزش ببیند که بتواند روی دامنه‌های کاملاً ندیده نیز عملکرد مناسبی داشته باشد.

از این رو، تفاوت کلیدی این دو رویکرد در فرض دسترسی به داده‌های دامنه هدف است. تطبیق دامنه بر این فرض استوار است که می‌توان پیش از استقرار مدل، نمونه‌هایی از دامنه هدف را (حتی بدون برچسب) جمع‌آوری کرد. اما تعمیم دامنه بر فرضی سخت‌گیرانه‌تر متکی است: دامنه هدف در زمان آموزش کاملاً ناشناخته و غیرقابل دسترس است. همین موضوع باعث می‌شود که تعمیم دامنه ذاتاً مسئله‌ای دشوارتر باشد، زیرا مدل باید بدون مشاهده دامنه هدف، ویژگی‌هایی بیاموزد که نسبت به تغییرات مختلف دامنه پایدار باشند. به بیان دیگر، در تطبیق دامنه مدل برای یک دامنه مشخص سازگار[۲۰] می‌شود، در حالی که در تعمیم دامنه باید برای مجموعه‌ای از دامنه‌های ناشناخته مقاوم[۲۱] شود.

برای روشن‌تر شدن این تفاوت، یک سناریوی مناسب برای تطبیق دامنه در نظر بگیرید. فرض کنید یک مدل تشخیص شیء بر روی تصاویر باکیفیت (مثلاً تصاویر دیتاست‌های استاندارد) آموزش داده شده است، اما قرار است در یک سیستم واقعی با دوربین‌های ارزان‌قیمت و نویزدار استفاده شود. در این حالت، می‌توان پیش از استقرار سیستم، مجموعه‌ای از تصاویر واقعی (حتی بدون برچسب) از همان محیط عملیاتی جمع‌آوری کرد. از آنجا که دامنه هدف مشخص و قابل دسترس است، استفاده از تطبیق دامنه منطقی است، زیرا مدل می‌تواند با ویژگی‌های آماری این محیط (نویز، نور، اعوجاج) سازگار شود.

[20] Adapt
[21] Robust

در مقابل، سناریویی را در نظر بگیرید که در آن اساساً امکان دسترسی به دامنه هدف در زمان آموزش وجود ندارد. برای مثال، یک مدل بینایی ماشین برای استفاده در یک سامانه خودران عمومی طراحی شده است که قرار است در کشورهای مختلف، شرایط آب‌وهوایی گوناگون (باران، برف، مه)، ساعات مختلف شبانه‌روز و با انواع دوربین‌ها مورد استفاده قرار گیرد. در اینجا نه تنها دامنه هدف واحدی وجود ندارد، بلکه دامنه‌های آینده بسیار متنوع، ناشناخته و عملاً غیرقابل پیش‌بینی هستند. جمع‌آوری داده از «همه» شرایط ممکن پیش از آموزش عملاً غیرممکن است. در چنین سناریویی، حتی اگر بتوان نمونه‌هایی از برخی شرایط جمع‌آوری کرد، همچنان تضمینی وجود ندارد که این داده‌ها نماینده تمامی دامنه‌های آینده باشند. بنابراین، استفاده از تطبیق دامنه که مبتنی بر مشاهده یک دامنه هدف مشخص است، کافی نخواهد بود و باید از تعمیم دامنه استفاده کرد تا مدل بتواند به‌صورت ذاتی نسبت به تغییرات دامنه مقاوم باشد.

در جمع‌بندی، می‌توان گفت که تطبیق دامنه رویکردی مناسب برای شرایطی است که دامنه هدف مشخص و قابل دسترس باشد، در حالی که تعمیم دامنه برای سناریوهایی ضروری است که دامنه هدف ناشناخته، متغیر یا غیرقابل دسترس است. علاوه بر این، تعمیم دامنه به دلیل نبود اطلاعات مستقیم از دامنه هدف، مسئله‌ای پیچیده‌تر محسوب می‌شود و نیازمند یادگیری نمایش‌هایی عمومی‌تر و پایدارتر نسبت به تغییرات توزیع است.

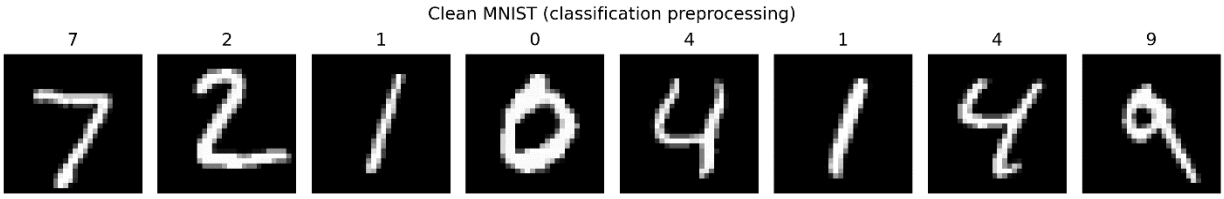
۳-۳- تطبیق دامنه در مسئله تشخیص ارقام دست‌نویس MNIST

در این بخش، هدف بررسی اثر تغییر دامنه بر عملکرد مدل و ارزیابی روش‌های مختلف تطبیق دامنه است. بدین منظور، ابتدا یک مدل پایه بر روی داده‌های MNIST آموزش داده شده و سپس عملکرد آن بر روی داده‌های تغییر یافته (چرخیده) مورد بررسی قرار می‌گیرد. در ادامه، با استفاده از سه رویکرد متفاوت شامل Reweighting، هم‌ترازی ویژگی‌ها (DANN[۲۲]) و انتقال دامنه (CycleGAN)، تلاش می‌شود تا افت عملکرد ناشی از تغییر دامنه کاهش یابد. در نهایت، نتایج این روش‌ها با یکدیگر و همچنین با یک کران بالای نظارت‌شده مقایسه می‌شوند.

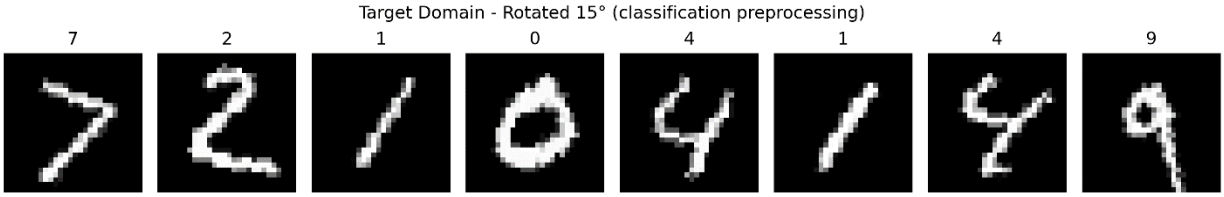
۳-۳-۱- نمایش نمونه داده‌ها

در این بخش، به‌منظور درک بهتر ماهیت مسئله و میزان تفاوت میان دامنه‌ها، نمونه‌هایی از داده‌های مورد استفاده در این تمرین نمایش داده شده‌اند. این نمونه‌ها شامل داده‌های دامنه منبع (MNIST اصلی) و داده‌های دامنه هدف با دو میزان چرخش (۱۵ و ۹۰ درجه) هستند. همچنین، برای هر یک از این حالت‌ها، دو نوع پیش‌پردازش متفاوت که در ادامه برای مدل‌های طبقه‌بندی و مدل‌های مولد مورد استفاده قرار می‌گیرند، در نظر گرفته شده است.

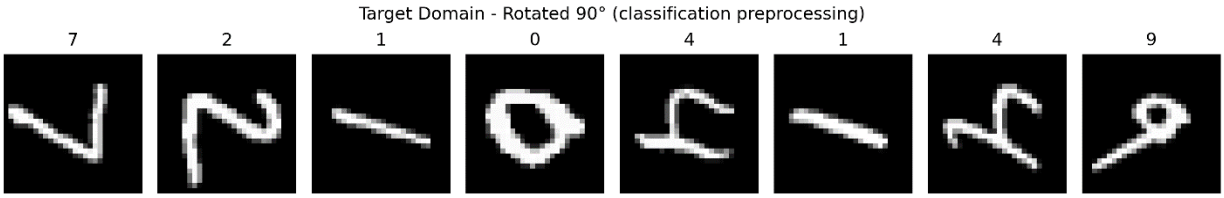
[22] Domain-Adversarial Neural Network



*شکل ۳–۱: نمونه‌هایی از داده‌های **MNIST** (دامنه منبع) پس از پیش‌پردازش مناسب برای مدل‌های طبقه‌بندی*



شکل ۳–۲: نمونه‌هایی از داده‌های دامنه هدف با چرخش ۱۵ درجه پس از پیش‌پردازش طبقه‌بندی

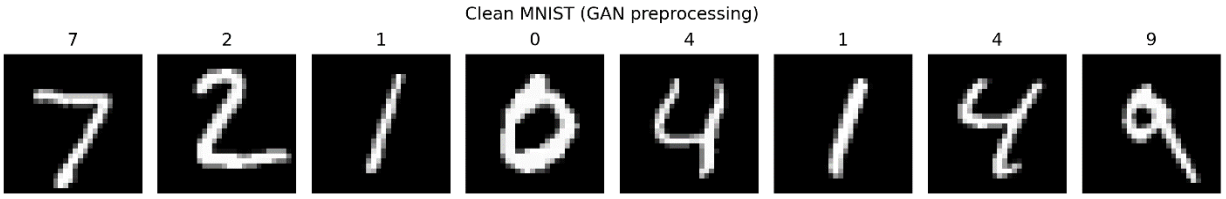


شکل ۳–۳: نمونه‌هایی از داده‌های دامنه هدف با چرخش ۹۰ درجه که بیانگر یک تغییر دامنه شدیدتر هستند.

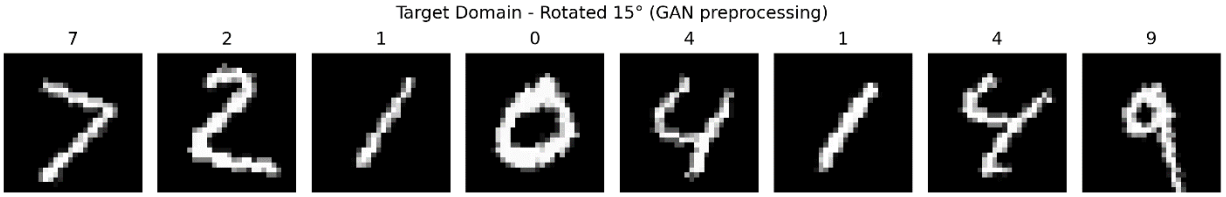
در شکل ۳–۱، نمونه‌هایی از داده‌های تمیز MNIST پس از پیش‌پردازش مورد استفاده در مدل‌های طبقه‌بندی نمایش داده شده‌اند. این داده‌ها همان دامنه منبع هستند که مدل پایه بر روی آن‌ها آموزش می‌بیند و به‌عنوان مرجع اصلی برای مقایسه با سایر دامنه‌ها در نظر گرفته می‌شوند.

در شکل ۳–۲، همین داده‌ها پس از اعمال چرخش ۱۵ درجه نمایش داده شده‌اند. در این حالت، اگرچه تغییراتی در جهت‌گیری ارقام ایجاد شده است، اما ساختار کلی آن‌ها همچنان تا حد زیادی قابل تشخیص باقی مانده است.

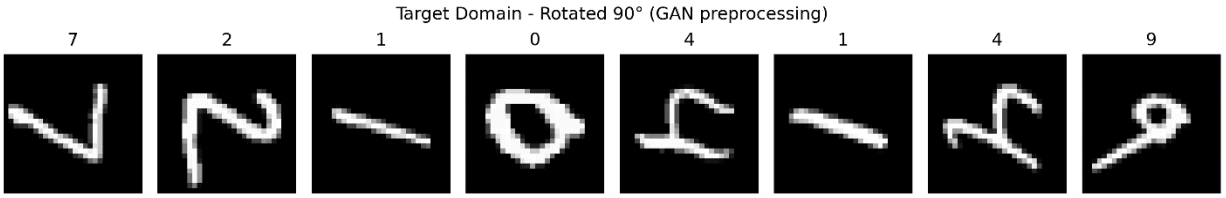
در شکل ۳–۳، داده‌ها با چرخش ۹۰ درجه نمایش داده شده‌اند که نشان‌دهنده یک تغییر دامنه شدیدتر است. در این حالت، ساختار هندسی بسیاری از ارقام به‌طور قابل توجهی تغییر کرده و شباهت بین برخی کلاس‌ها افزایش یافته است.



*شکل ۳–۴: داده‌های تمیز **MNIST** پس از پیش‌پردازش مورد استفاده در مدل‌های مولد*



*شکل ۳–۵: داده‌های چرخیده ۱۵ درجه پس از پیش‌پردازش **GAN***



*شکل ۳–۶: داده‌های چرخیده ۹۰ درجه پس از پیش‌پردازش **GAN***

در شکل ۳–۴ تا شکل ۳–۶، همان داده‌ها تحت پیش‌پردازش مورد استفاده در مدل‌های مولد نمایش داده شده‌اند. تفاوت اصلی در این حالت، نحوه نرمال‌سازی داده‌ها است که به‌گونه‌ای انجام می‌شود که مقادیر پیکسل در بازه‌ای متقارن حول صفر قرار گیرند. این نوع پیش‌پردازش برای پایداری فرآیند یادگیری در مدل‌های مولد ضروری است.

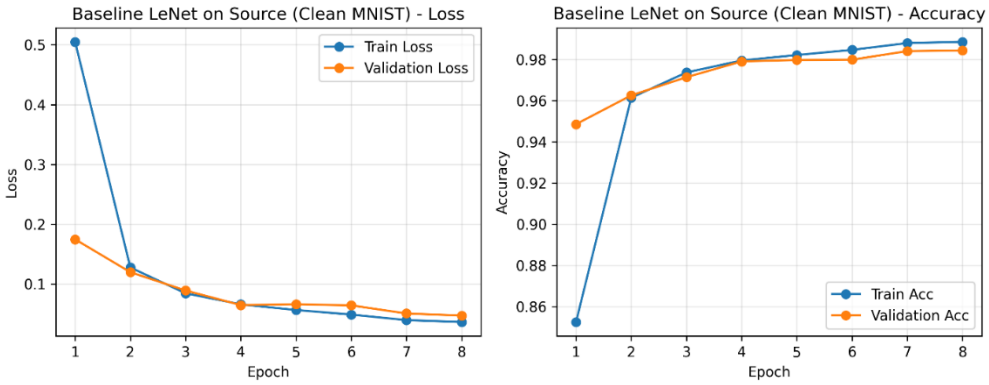
با مقایسه این تصاویر، می‌توان مشاهده کرد که هرچه میزان چرخش افزایش می‌یابد، فاصله بین دامنه منبع و دامنه هدف بیشتر شده و در نتیجه، مسئله تعمیم برای مدل دشوارتر می‌شود. به‌ویژه در حالت چرخش ۹۰ درجه، برخی ارقام از نظر بصری به کلاس‌های دیگر نزدیک می‌شوند که این موضوع می‌تواند منجر به افزایش خطای مدل در صورت عدم استفاده از روش‌های تطبیق دامنه شود. همچنین، تفاوت در پیش‌پردازش داده‌ها نشان می‌دهد که هر دسته از مدل‌ها (طبقه‌بندی و مولد) نیازمند نمایش مناسب‌تری از داده‌ها برای یادگیری مؤثر هستند، هرچند محتوای اصلی داده‌ها در هر دو حالت یکسان باقی می‌ماند.

۳-۳-۲- ارزیابی مدل پایه بدون تطبیق دامنه

در این بخش، عملکرد مدل پایه مورد بررسی قرار می‌گیرد. منظور از مدل پایه، همان مدل LeNet است که صرفاً بر روی داده‌های دامنه منبع (MNIST اصلی) آموزش داده شده و هیچ‌گونه اطلاعاتی از دامنه هدف (داده‌های چرخیده) در فرآیند آموزش آن استفاده نشده است. این مدل به‌عنوان مرجع اولیه در نظر گرفته می‌شود تا بتوان اثر تغییر دامنه و همچنین میزان بهبود حاصل از روش‌های تطبیق دامنه را در مراحل بعدی به‌درستی ارزیابی کرد.

۳-۳-۲-۱- تحلیل همگرایی مدل و عملکرد در حضور تغییر دامنه

نتایج حاصل از آموزش مدل در شکل ۳–۷ نشان می‌دهد که فرآیند یادگیری به‌صورت پایدار انجام شده و مدل به‌خوبی بر روی داده‌های دامنه منبع همگرا شده است. نزدیکی مقادیر عملکرد روی داده‌های آموزش و اعتبارسنجی بیانگر آن است که مدل دچار بیش‌برازش قابل توجهی نشده و توانایی تعمیم مناسبی در همان دامنه دارد.



*شکل ۳–۷: روند تغییرات مقدار خطا و دقت مدل پایه در طول آموزش بر روی داده‌های **MNIST** (دامنه منبع)*

با این حال، زمانی که همین مدل بدون هیچ‌گونه تطبیقی بر روی داده‌های دامنه هدف ارزیابی می‌شود، رفتار آن به‌طور قابل توجهی تغییر می‌کند که نتایج آن در جدول ۳–۱ آمده است.

جدول ۳–۱: عملکرد مدل پایه (بدون تطبیق دامنه) بر روی داده تست در دامنه‌های مختلف

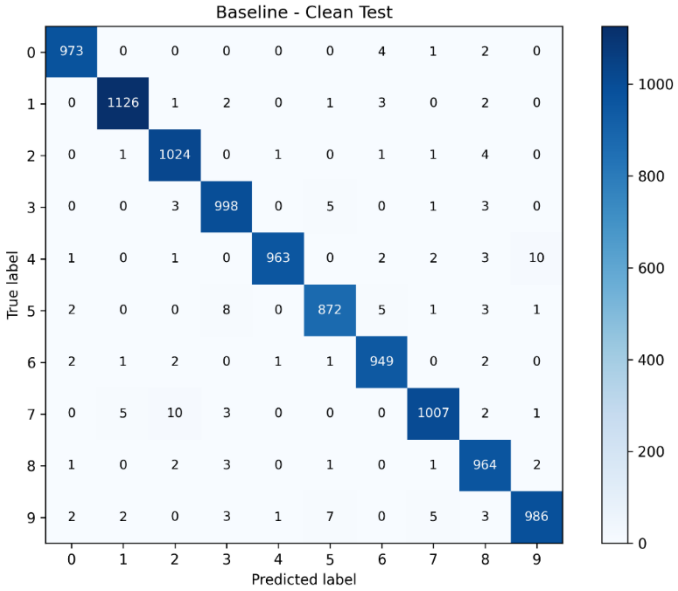
دامنه ارزیابی	**دقت تست**
Clean (MNIST)	۰.۹۸۶۲
Rot15	۰.۹۴۸۱
Rot90	۰.۱۶۹۲

تحلیل این نتایج نشان می‌دهد که اگرچه مدل در دامنه‌ای که بر روی آن آموزش دیده عملکرد بسیار مطلوبی دارد، اما با تغییر دامنه –حتی در حد یک چرخش ساده– عملکرد آن دچار افت می‌شود. این افت در حالت چرخش ۱۵ درجه نسبتاً محدود است، که نشان می‌دهد مدل تا حدی نسبت به تغییرات جزئی مقاوم است. اما در حالت چرخش ۹۰ درجه، کاهش شدید عملکرد بیانگر آن است که ویژگی‌های آموخته‌شده توسط مدل به‌شدت به ساختار فضایی داده‌های دامنه منبع وابسته هستند و در مواجهه با تغییرات اساسی، دیگر قابل اتکا نیستند.

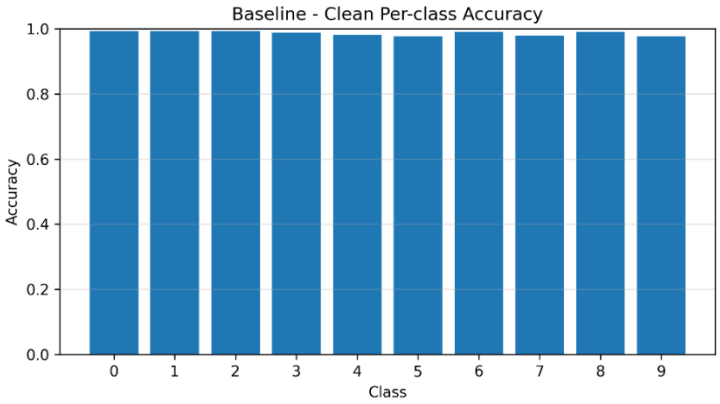
این مشاهده به‌روشنی ضرورت استفاده از روش‌های تطبیق دامنه را نشان می‌دهد، چرا که بدون در نظر گرفتن تفاوت میان توزیع داده‌های آموزش و آزمون، حتی مدل‌هایی که در دامنه منبع عملکرد بسیار خوبی دارند نیز ممکن است در دامنه هدف با شکست مواجه شوند.

۳-۳-۲-۲- تحلیل ماتریس درهم‌ریختگی و دقت کلاس‌به‌کلاس در حالت مدل پایه

در این بخش، عملکرد مدل پایه با جزئیات بیشتری و در سطح هر کلاس بررسی می‌شود. برای هر دامنه (داده‌های تمیز، چرخش ۱۵ درجه و چرخش ۹۰ درجه)، ماتریس درهم‌ریختگی و نمودار دقت کلاس‌به‌کلاس در کنار یکدیگر ارائه شده‌اند تا بتوان الگوی خطاهای مدل را به‌صورت دقیق‌تر تحلیل کرد.

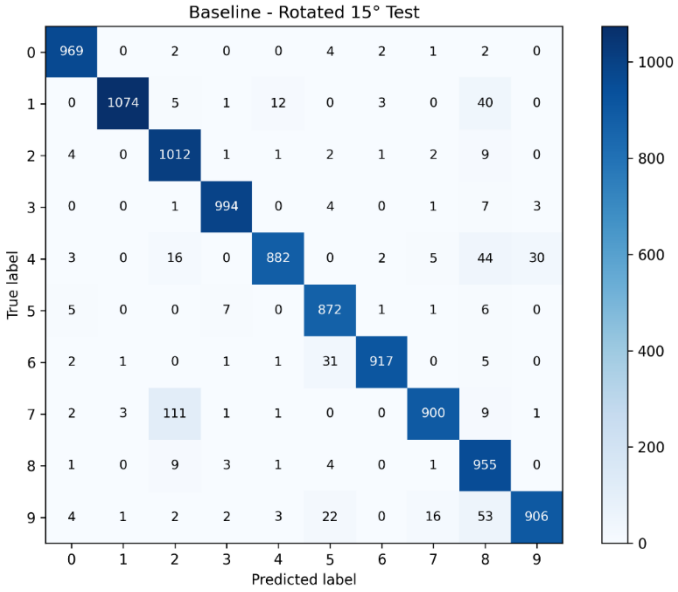


شکل ۳–۸: ماتریس درهم‌ریختگی مدل پایه بر روی داده‌های دامنه منبع

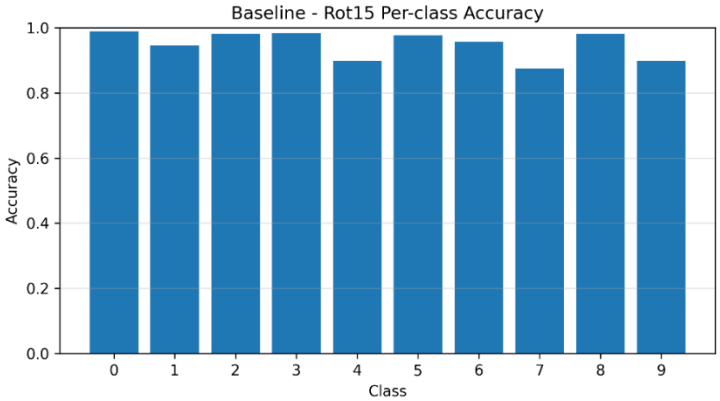


شکل ۳–۹: دقت کلاس‌به‌کلاس مدل پایه بر روی داده‌های دامنه منبع

در دامنه منبع، ماتریس درهم‌ریختگی نشان می‌دهد که مقادیر روی قطر اصلی به‌وضوح غالب هستند و خطاها بسیار محدود و پراکنده‌اند. دقت کلاس‌به‌کلاس نیز برای تمامی ارقام بسیار بالا و نزدیک به ۱ است که بیانگر یادگیری مؤثر ویژگی‌های متمایزکننده‌ی هر کلاس توسط مدل پایه است. با این حال، بررسی دقیق‌تر ماتریس نشان می‌دهد که خطاهای برجسته عمدتاً میان کلاس‌هایی رخ می‌دهند که از نظر ساختار نوشتاری شباهت بیشتری دارند؛ به‌طور مشخص، بیشترین اشتباهات مربوط به ۵ و ۳، همچنین ۹ و ۵، و نیز ۴ و ۹ است. علاوه بر این، در مورد رقم ۷ نیز تمایز آن از ۲ یکی از منابع اصلی خطاست. این الگو نشان می‌دهد که حتی در شرایط داده‌ی تمیز، مدل در تمایز برخی ویژگی‌های ریز هندسی (مانند انحناها یا نحوه‌ی اتصال خطوط) دچار چالش جزئی است. با این وجود، تعداد این خطاها همچنان کم بوده و تأثیر محسوسی بر عملکرد کلی مدل ندارند.

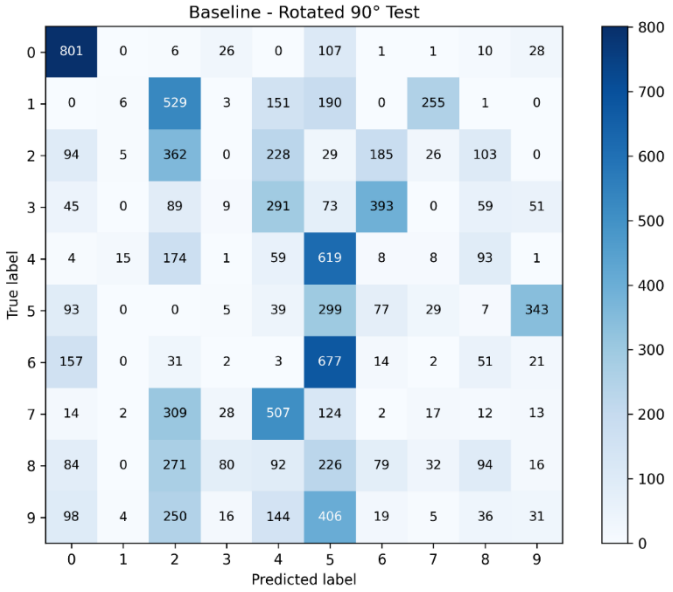


شکل ۳–۱۰: ماتریس درهم‌ریختگی مدل پایه بر روی داده‌های چرخیده ۱۵ درجه

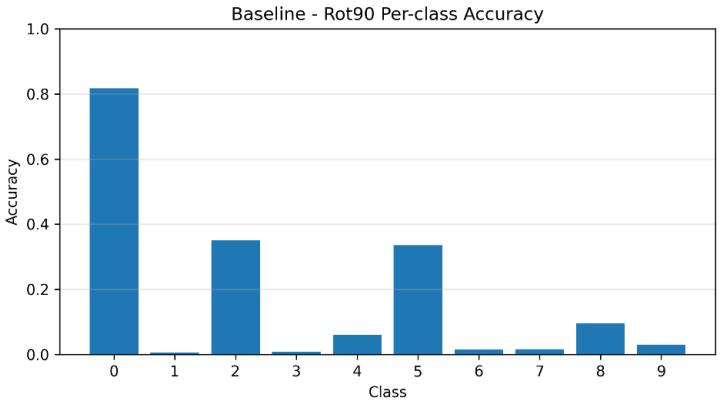


شکل ۳–۱۱: دقت کلاس‌به‌کلاس مدل پایه بر روی داده‌های چرخیده ۱۵ درجه

در حالت چرخش ۱۵ درجه، اگرچه ساختار کلی ماتریس همچنان تا حد زیادی قطرمحور باقی مانده است، اما میزان خطاها نسبت به حالت داده‌ی تمیز به‌طور محسوسی افزایش یافته است. دقت کلاس‌به‌کلاس نیز افت کرده و این کاهش در برخی کلاس‌ها پررنگ‌تر است. تحلیل دقیق‌تر نشان می‌دهد که رقم ۷ بیشترین تأثیر را از چرخش گرفته و تعداد قابل توجهی از نمونه‌های آن به‌صورت نادرست به کلاس ۲ پیش‌بینی شده‌اند. همچنین رقم ۴ نیز دچار افت عملکرد شده و بخش قابل توجهی از نمونه‌های آن با کلاس‌های ۸ و ۹ اشتباه گرفته می‌شوند. در کنار این موارد، برای رقم ۹ نیز افزایش خطا مشاهده می‌شود که بخشی از آن به اشتباه با کلاس‌های ۸ و ۵ مربوط است. این الگو نشان می‌دهد که چرخش -حتی در حد ۱۵ درجه- می‌تواند ساختار فضایی برخی ارقام را به‌گونه‌ای تغییر دهد که شباهت آن‌ها با کلاس‌های دیگر افزایش یابد و در نتیجه مرزهای تصمیم‌گیری مدل دچار اختلال شود، به‌ویژه برای کلاس‌هایی که از ابتدا نیز از نظر هندسی به یکدیگر نزدیک‌تر هستند.



شکل ۳–۱۲: ماتریس درهم‌ریختگی مدل پایه بر روی داده‌های چرخیده ۹۰ درجه



شکل ۳–۱۳: دقت کلاس‌به‌کلاس مدل پایه بر روی داده‌های چرخیده ۹۰ درجه

در حالت چرخش ۹۰ درجه، الگوی خطاها به‌طور اساسی تغییر کرده و ماتریس درهم‌ریختگی عملاً ساختار قطرمحور خود را از دست داده است. با بررسی کلاس‌به‌کلاس مشاهده می‌شود که رقم ۰ همچنان عملکرد نسبتاً خوبی دارد و بخش قابل توجهی از نمونه‌های آن به‌درستی تشخیص داده می‌شوند. رقم ۱ تقریباً به‌طور کامل از کلاس صحیح خارج شده و عمدتاً به کلاس‌های ۲، ۷ و ۵ نگاشت می‌شود. رقم ۲ در مقایسه با سایر کلاس‌ها عملکرد بهتری دارد، اما همچنان بخش قابل توجهی از نمونه‌های آن به کلاس‌های ۴ و ۶ اشتباه گرفته می‌شوند. رقم ۳ دچار افت شدید شده و بیشتر با کلاس‌های ۶ و ۴ اشتباه می‌شود. رقم ۴ عمدتاً به کلاس ۵ نگاشت شده و تمایز آن به‌شدت تضعیف شده است. رقم ۵ نیز اگرچه نسبت به بسیاری از کلاس‌ها عملکرد بهتری دارد، اما همچنان پراکندگی قابل توجهی به کلاس‌های ۹ و ۰ دارد. رقم ۶ تقریباً به‌صورت غالب به کلاس ۵ پیش‌بینی می‌شود. رقم ۷ بیشتر به کلاس‌های ۴ و ۲ نگاشت می‌شود و دقت پایینی دارد. رقم ۸ عملکرد ضعیفی داشته و بخش زیادی از نمونه‌های آن به کلاس‌های ۲ و ۵ پیش‌بینی می‌شوند. در نهایت، رقم ۹ نیز به‌طور گسترده با کلاس‌های ۵ و ۲ اشتباه گرفته می‌شود و تمرکز خود را روی کلاس صحیح از دست داده است.

در نمودار دقت کلاس‌به‌کلاس نیز مشاهده می‌شود که تنها کلاس ۰ دقت نسبتاً قابل قبولی دارد و حتی کلاس‌هایی مانند ۲ و ۵ نیز با وجود عملکرد بهتر نسبی، همچنان فاصله‌ی زیادی تا یک عملکرد مناسب دارند، در حالی که سایر کلاس‌ها با افت شدید مواجه شده‌اند. این رفتار نشان می‌دهد که مدل پایه به‌شدت به جهت‌گیری و ساختار فضایی داده‌های ورودی وابسته است و در برابر چرخش شدید، ویژگی‌های استخراج‌شده دیگر تفکیک‌پذیری لازم را ندارند. به بیان دیگر، نمایش‌های آموخته‌شده نسبت به تبدیلات هندسی بزرگ ناوردا نیستند و این امر منجر به افت شدید عملکرد در سطح کلاس‌ها می‌شود.

در مجموع، این تحلیل نشان می‌دهد که خطاهای مدل در دامنه‌های جدید به‌صورت یکنواخت توزیع نشده‌اند، بلکه برخی کلاس‌ها به‌طور خاص نسبت به تغییر دامنه حساس‌تر هستند. این موضوع اهمیت استفاده از روش‌های تطبیق دامنه را بیش از پیش آشکار می‌سازد، چرا که هدف این روش‌ها کاهش چنین ناهماهنگی‌هایی در سطح ویژگی‌ها و در نتیجه بهبود عملکرد مدل در تمامی کلاس‌ها است.

۳-۳-۳- کران بالای عملکرد با تنظیم نظارت‌شده[۲۳]

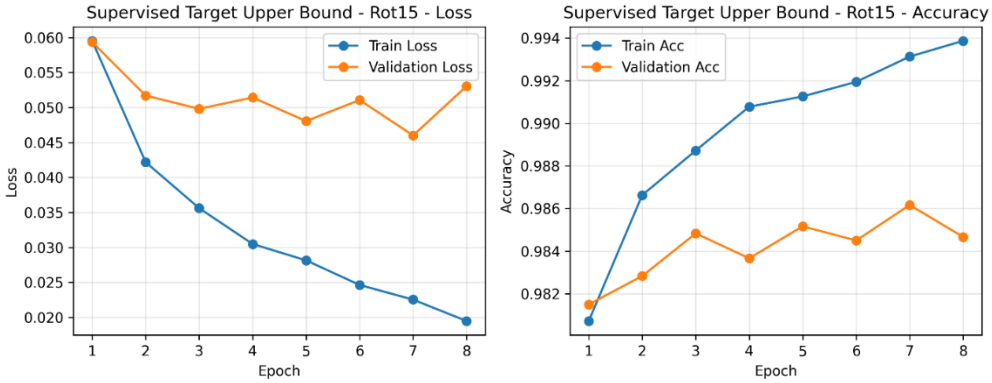
در این بخش، به‌منظور درک بهتر از سقف عملکرد قابل دستیابی در مسئله، حالتی در نظر گرفته می‌شود که در آن به برچسب‌های دامنه هدف دسترسی کامل داریم. در این سناریو، مدل پایه مستقیماً بر روی داده‌های دامنه هدف (تصاویر چرخیده‌شده) به‌صورت نظارت‌شده بازآموزی[۲۴] می‌شود. بدیهی است که این حالت با فرض مسئله‌ی تطبیق دامنه (که در آن برچسب‌های دامنه هدف در دسترس نیستند) سازگار نیست و صرفاً به‌عنوان یک مرجع برای ارزیابی عملکرد سایر روش‌ها مورد استفاده قرار می‌گیرد.

در واقع، نتایج این بخش یک کران بالا برای دقت مدل فراهم می‌کنند؛ به این معنا که هیچ‌یک از روش‌های تطبیق دامنه بدون نظارت نباید انتظار داشته باشند به‌صورت معنادار از این سطح عملکرد فراتر روند.

[23] Supervised Fine-Tuning
[24] Fine-tune

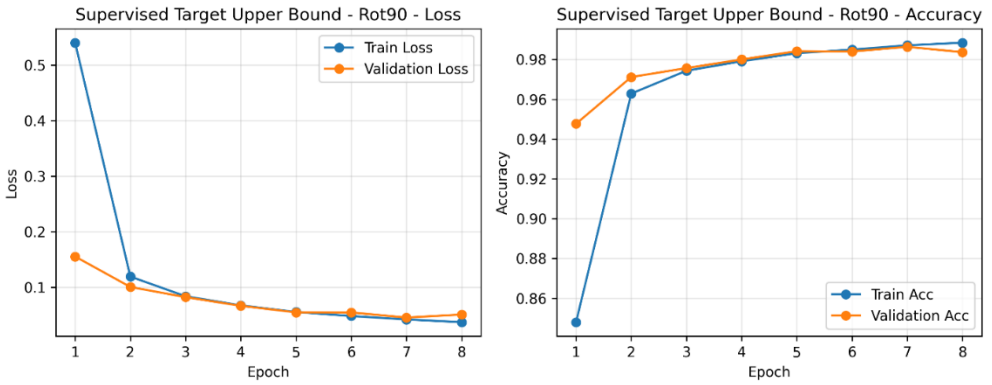
۳-۳-۳-۱- تحلیل روند یادگیری در حالت نظارت‌شده

در حالت چرخش ۱۵ درجه، مدل به‌سرعت با دامنه جدید سازگار می‌شود و روند کاهش خطای آموزش به‌صورت یکنواخت ادامه دارد. در مقابل، خطای اعتبارسنجی پس از یک کاهش اولیه، در ادامه نوسان‌های جزئی دارد و به‌صورت کاملاً یکنواخت کاهش پیدا نمی‌کند. با این حال، مقدار آن در سطح نسبتاً پایینی باقی می‌ماند که نشان‌دهنده‌ی یادگیری مناسب ویژگی‌های دامنه هدف است. از سوی دیگر، دقت اعتبارسنجی نیز با وجود نوسانات خفیف، در بازه‌ی بالایی (حدود ۰.۹۸ تا ۰.۹۸۶) حفظ شده و بیانگر عملکرد پایدار مدل است.



شکل ۳–۱۴: روند تغییرات خطا و دقت مدل پس از تنظیم نظارت‌شده بر روی داده‌های چرخیده ۱۵ درجه

فاصله بین دقت آموزش و اعتبارسنجی اندک اما قابل مشاهده است؛ به‌طوری‌که دقت آموزش به‌صورت یکنواخت افزایش می‌یابد، در حالی که دقت اعتبارسنجی روندی با نوسان دارد. این الگو نشان می‌دهد که نشانه‌های خفیفی از بیش‌برازش در حال شکل‌گیری است، اما شدت آن پایین بوده و هنوز تأثیر مخربی بر تعمیم مدل نگذاشته است. در مجموع، مدل توانسته تعمیم مناسبی روی دامنه هدف داشته باشد، هرچند پایداری کامل در معیارهای اعتبارسنجی مشاهده نمی‌شود.



شکل ۳–۱۵: روند تغییرات خطا و دقت مدل پس از تنظیم نظارت‌شده بر روی داده‌های چرخیده ۹۰ درجه

در حالت چرخش ۹۰ درجه، اگرچه در ابتدای آموزش مدل با خطای بالایی مواجه است، اما روند یادگیری نشان می‌دهد که مدل به‌سرعت خود را با این تغییر شدید در دامنه تطبیق می‌دهد. کاهش تند خطا در ایپاک‌های ابتدایی و افزایش همزمان دقت، بیانگر نقش کلیدی داده‌های برچسب‌دار دامنه هدف در اصلاح نمایش‌های آموخته‌شده است. در ادامه، روند بهبود با شیب ملایم‌تری ادامه یافته و خطاها به سطح پایینی می‌رسند.

همچنین فاصله بین معیارهای آموزش و اعتبارسنجی بسیار کم باقی می‌ماند و هر دو دقت به مقادیر بالایی همگرا می‌شوند که نشان‌دهنده‌ی تعمیم مناسب مدل و نبود بیش‌برازش قابل توجه است. در نهایت، عملکرد مدل به سطح بالایی از دقت می‌رسد که در تضاد آشکار با عملکرد ضعیف مدل پایه در این دامنه است.

در ادامه، برای تکمیل تحلیل، عملکرد نهایی مدل در حالت تنظیم نظارت‌شده بر روی دامنه هدف در قالب جدول زیر ارائه می‌شود.

جدول ۳–۲: عملکرد مدل در حالت ***Supervised Fine-Tuning*** *(کران بالا) بر روی دامنه‌های مختلف*

دامنه آموزش	**دقت روی داده تمیز (تست عادی)**	**دقت روی دامنه هدف**
Rot15	۰.۹۷۳۷	۰.۹۸۶۳
Rot90	۰.۳۳۲۶	۰.۹۸۵۳

همان‌طور که مشاهده می‌شود، در هر دو حالت، دقت مدل بر روی دامنه هدف (داده‌های چرخیده) به مقادیر بسیار بالا و نزدیک به ۰.۹۹ رسیده است. این نتایج در واقع یک کران بالا را نشان می‌دهند، زیرا مدل با استفاده از برچسب‌های دامنه هدف آموزش دیده و در نتیجه قادر است به‌خوبی با توزیع جدید سازگار شده و عملکرد بسیار مطلوبی ارائه دهد. لازم به ذکر است که این روش جزو روش‌های تطبیق دامنه محسوب نمی‌شود و صرفاً به‌عنوان یک مرجع برای مقایسه و درک حداکثر عملکرد قابل دستیابی ارائه شده است.

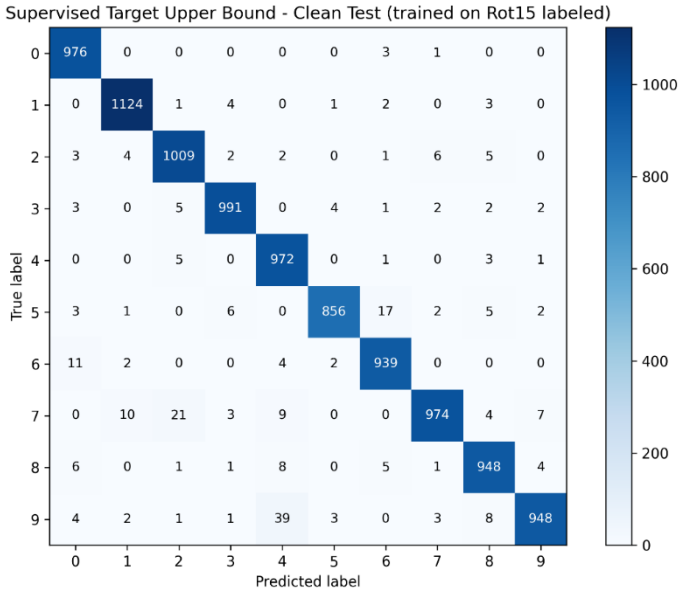
در مقابل، کاهش قابل توجه دقت بر روی داده‌های تمیز، به‌ویژه در حالت آموزش روی داده‌های چرخیده ۹۰ درجه، نشان‌دهنده‌ی پدیده‌ی over-specialization یا سازگاری بیش‌ازحد با دامنه هدف است. به بیان دیگر، مدل با تمرکز بر ویژگی‌های خاص دامنه جدید، بخشی از توانایی خود در تعمیم به دامنه اولیه را از دست می‌دهد.

در مجموع، این نتایج نشان می‌دهند که شکاف عملکرد در حالت بدون تطبیق عمدتاً ناشی از اختلاف توزیع بین دامنه‌ها است، نه محدودیت ذاتی مدل. همچنین، این جدول یک مرجع کمی مناسب برای مقایسه با روش‌های تطبیق دامنه بدون نظارت فراهم می‌کند و می‌توان سنجید هر روش تا چه حد به این کران بالا نزدیک می‌شود.

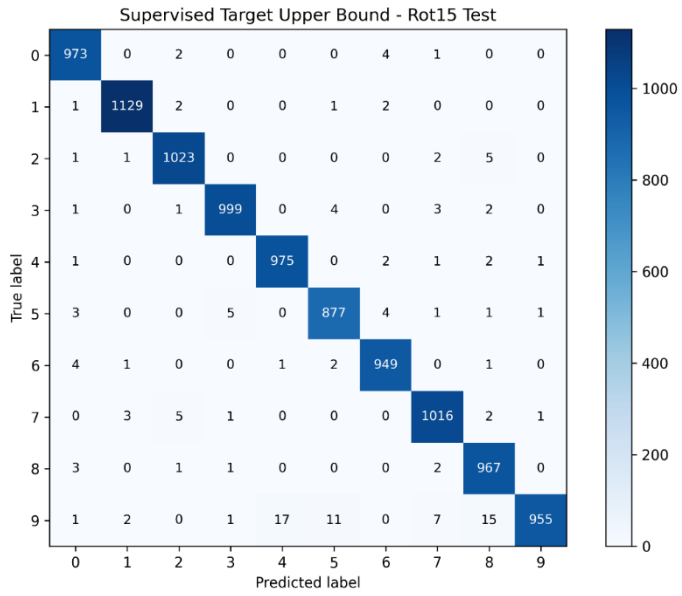
۳-۳-۳-۲- تحلیل ماتریس درهم‌ریختگی و دقت در هر کلاس در حالت Supervised Fine-Tuning

در این بخش، عملکرد مدل آموزش‌دیده به‌صورت نظارت‌شده بر روی دامنه هدف، با استفاده از ماتریس‌های درهم‌ریختگی و نمودار دقت در هر کلاس مورد بررسی دقیق‌تری قرار می‌گیرد. هدف از این تحلیل، شناسایی الگوهای خطا و بررسی این موضوع است که مدل در هر دامنه، کدام کلاس‌ها را بیشتر با یکدیگر اشتباه می‌گیرد.

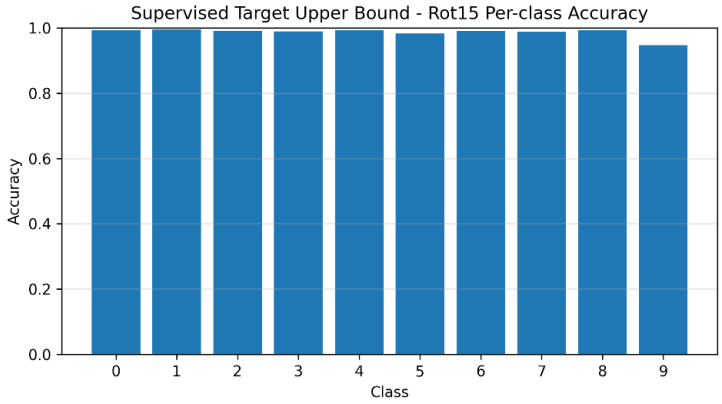
ابتدا نتایج مربوط به حالت آموزش روی داده‌های چرخیده ۱۵ درجه ارائه شده و سپس همین تحلیل برای حالت چرخش ۹۰ درجه انجام می‌شود.



شکل ۳–۱۶: ماتریس درهم‌ریختگی مدل آموزش‌دیده بر روی داده‌های ۱۵ درجه، ارزیابی‌شده بر روی داده‌های تمیز (تست عادی)



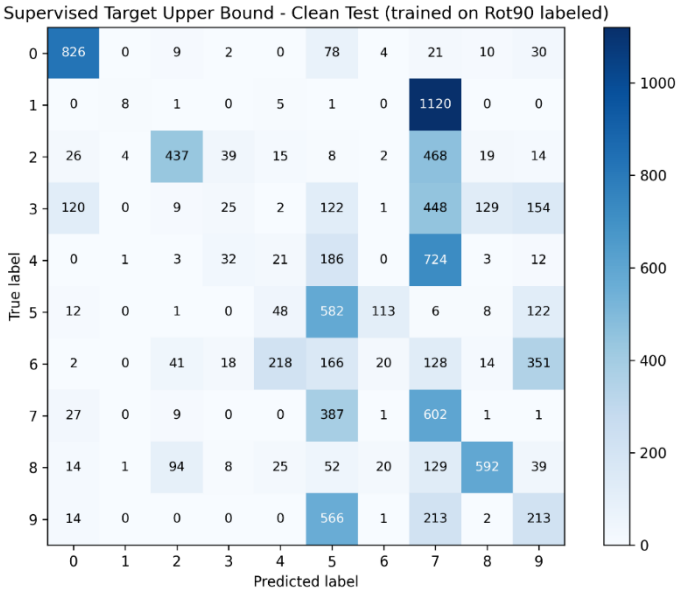
شکل ۳–۱۷: ماتریس درهم‌ریختگی مدل آموزش‌دیده بر روی داده‌های ۱۵ درجه، ارزیابی‌شده بر روی داده‌های چرخیده ۱۵ درجه



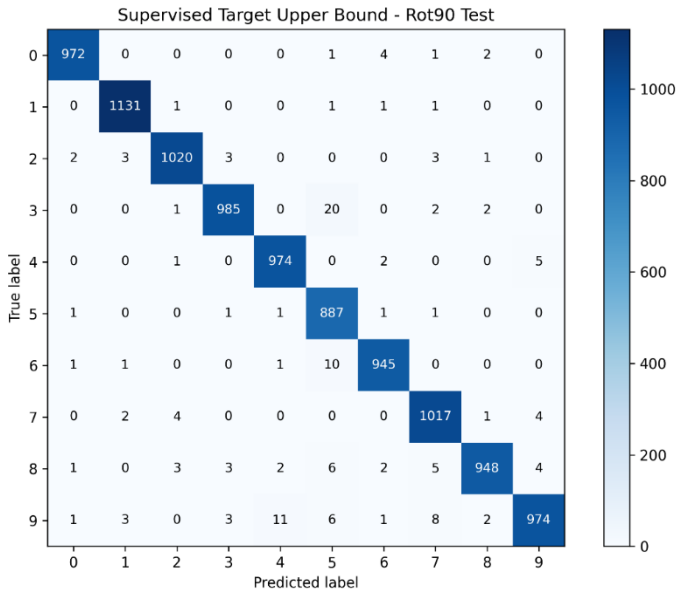
شکل ۳–۱۸: دقت مدل در هر کلاس برای داده‌های چرخیده ۱۵ درجه در حالت ***Supervised Fine-Tuning***

در حالت آموزش روی داده‌های ۱۵ درجه، مدل عملکردی بسیار پایدار و دقیق در تمامی کلاس‌ها از خود نشان می‌دهد. در ماتریس درهم‌ریختگی مربوط به داده‌های ۱۵ درجه، مقادیر قطری در تقریباً همه‌ی کلاس‌ها بسیار بالا هستند که نشان‌دهنده‌ی تشخیص صحیح اغلب نمونه‌هاست. خطاهای باقی‌مانده نیز محدود هستند و برخلاف حالت چرخش شدید، تمرکز قابل توجهی خارج از قطر اصلی دیده نمی‌شود.

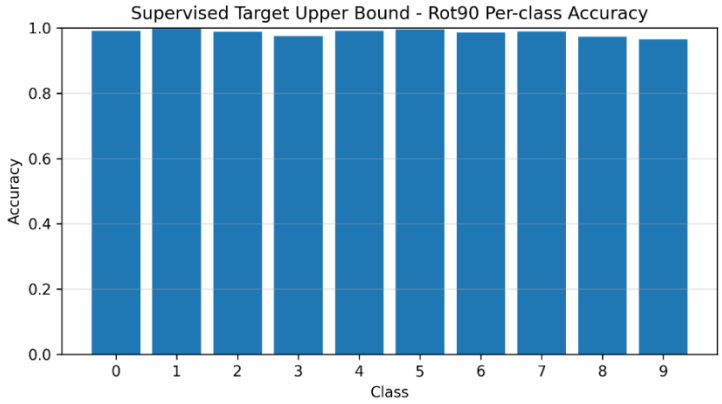
با این حال، چند خطای مشخص قابل ذکر است؛ برای مثال، در کلاس ۹ بخشی از نمونه‌ها با ۴، ۸ و ۵ اشتباه گرفته شده‌اند. همچنین در کلاس ۷ تعداد کمی از نمونه‌ها به‌صورت ۲ پیش‌بینی شده‌اند. در داده‌ی تمیز نیز پس از آموزش روی Rot15، مهم‌ترین خطای قابل مشاهده مربوط به اشتباه ۹ با ۴ است. با وجود این موارد، نمودار دقت کلاس‌به‌کلاس نشان می‌دهد که همه‌ی کلاس‌ها دقتی نزدیک به ۱ دارند و هیچ کلاسی دچار افت شدید عملکرد نشده است؛ بنابراین خطاها بیشتر محدود، پراکنده و کم‌اثر هستند..



شکل ۳–۱۹: ماتریس درهم‌ریختگی مدل آموزش‌دیده بر روی داده‌های ۹۰ درجه، ارزیابی‌شده بر روی داده‌های تمیز (تست عادی)



شکل ۳–۲۰: ماتریس درهم‌ریختگی مدل آموزش‌دیده بر روی داده‌های ۹۰ درجه، ارزیابی‌شده بر روی داده‌های چرخیده ۹۰ درجه



شکل ۳–۲۱: دقت مدل در هر کلاس برای داده‌های چرخیده ۹۰ درجه در حالت ***Supervised Fine-Tuning***

در حالت آموزش روی داده‌های ۹۰ درجه، رفتار مدل تفاوت معناداری با حالت قبل دارد. در ارزیابی روی داده‌های چرخیده ۹۰ درجه، همانند حالت ۱۵ درجه، عملکرد مدل بسیار دقیق و یکنواخت است و تقریباً تمامی کلاس‌ها با دقت بالا شناسایی می‌شوند؛ به‌طوری‌که مقادیر قطر اصلی در ماتریس درهم‌ریختگی غالب هستند و خطاها بسیار محدود باقی مانده‌اند. این موضوع نشان می‌دهد که مدل توانسته نمایش‌های مناسبی برای این دامنه جدید یاد بگیرد.

اما در ارزیابی روی داده‌های تمیز، الگوی خطا کاملاً ساختاریافته و قابل‌توجه است. به‌طور مشخص، رقم ۱ تقریباً به‌طور کامل به ۷ نگاشت شده است. همچنین رقم ۲ در بسیاری از موارد به ۷ پیش‌بینی می‌شود. برای رقم ۳ نیز اشتباهات گسترده‌ای با ۷، ۹ و ۸ دیده می‌شود. رقم ۴ عمدتاً به ۷ نگاشت شده و رقم ۵ نیز علاوه بر پیش‌بینی صحیح، بخش قابل توجهی از نمونه‌هایش به ۶ و ۹ اشتباه می‌شود. در مورد رقم ۶، نگاشت‌های قابل توجهی به ۴، ۵ و ۹ مشاهده می‌شود. رقم ۷ نیز علاوه بر پیش‌بینی صحیح، بخشی از نمونه‌هایش به ۵ نگاشت شده است. برای رقم ۸، اشتباهات عمده با ۲ و ۷ رخ می‌دهد و در نهایت رقم ۹ نیز بیشتر به ۵ و ۷ پیش‌بینی می‌شود. این الگو نشان می‌دهد که مدل پس از آموزش روی داده‌های ۹۰ درجه، ویژگی‌هایی وابسته به جهت‌گیری تصویر یاد گرفته و در دامنه اصلی دچار سردرگمی می‌شود.

نمودار دقت در هر کلاس برای داده‌های ۹۰ درجه نشان می‌دهد که در دامنه هدف، تمامی کلاس‌ها تقریباً به‌طور یکنواخت عملکرد بالایی دارند و افت محسوسی در هیچ کلاس خاصی دیده نمی‌شود. بنابراین، خطاهای اصلی مدل نه در دامنه هدف، بلکه در انتقال مجدد به دامنه اولیه ظاهر می‌شوند.

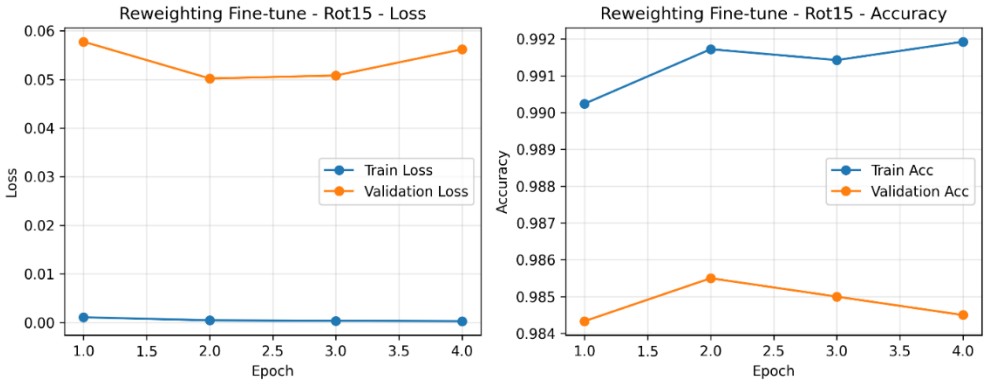
در جمع‌بندی این بخش، می‌توان گفت که در حالت Supervised Fine-Tuning، برای چرخش ۱۵ درجه خطاها بسیار محدود و غیرسیستماتیک هستند، اما برای چرخش ۹۰ درجه، با وجود عملکرد عالی در دامنه هدف، مدل در دامنه اولیه دچار اشتباهات ساختاریافته بین کلاس‌ها می‌شود، به‌ویژه در کلاس‌هایی که پس از چرخش شباهت بصری پیدا می‌کنند.

۳-۳-۴- تطبیق دامنه با استفاده از Reweighting

در این بخش، یکی از ساده‌ترین رویکردهای تطبیق دامنه یعنی Reweighting مورد بررسی قرار می‌گیرد. ایده‌ی اصلی این روش آن است که به نمونه‌های دامنه منبع وزن‌های متفاوتی اختصاص داده شود تا توزیع آن‌ها به توزیع دامنه هدف نزدیک‌تر شود. به بیان دیگر، مدل بدون تغییر معماری یا نمایش ویژگی‌ها، صرفاً با تغییر اهمیت نمونه‌ها در فرآیند آموزش تلاش می‌کند اثر شکاف دامنه را کاهش دهد.

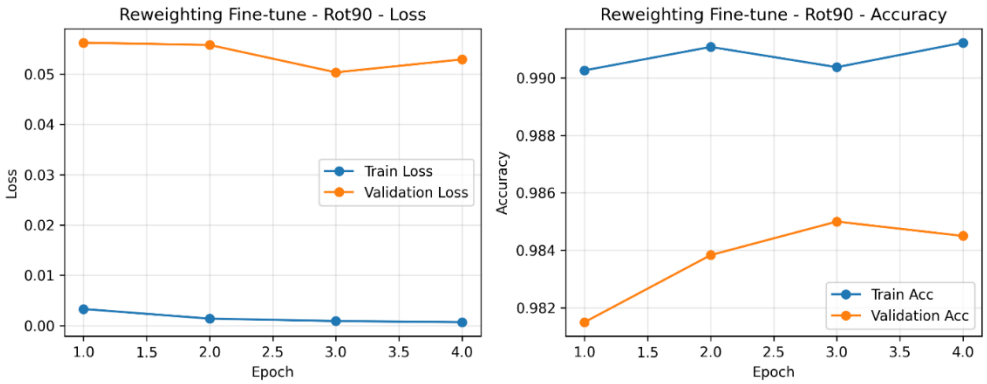
۳-۳-۴-۱- بررسی روند آموزش و عملکرد مدل در روش Reweighting

در این زیربخش، روند تغییرات خطا و دقت مدل در طول آموزش برای دو دامنه‌ی چرخیده ۱۵ و ۹۰ درجه بررسی می‌شود.



*شکل ۳–۲۲: روند تغییرات خطا و دقت در آموزش مدل با استفاده از **Reweighting** برای دامنه چرخیده ۱۵ درجه*

در حالت چرخش ۱۵ درجه، مقدار loss آموزش بسیار کوچک و نزدیک به صفر است که به ماهیت وزن‌دار بودن تابع هزینه در این روش مربوط می‌شود. در مقابل، loss اعتبارسنجی پس از یک کاهش اولیه، نوسان خفیفی داشته و در ادامه تقریباً در یک بازه‌ی ثابت باقی می‌ماند. از نظر دقت، مقدار آموزش در سطح بسیار بالایی (نزدیک به ۱) قرار دارد، در حالی که دقت اعتبارسنجی در بازه‌ی حدود ۰.۹۸۴ تا ۰.۹۸۶ نوسان می‌کند و فاصله‌ی کمی با دقت آموزش دارد. این رفتار نشان می‌دهد که مدل با وجود اعمال وزن‌دهی، توانسته عملکرد پایداری داشته باشد و تنها نشانه‌های خفیفی از بیش‌برازش یا ناپایداری در دقت اعتبارسنجی مشاهده می‌شود.



شکل ۳–۲۳: روند تغییرات خطا و دقت در آموزش مدل با استفاده از ***Reweighting*** *برای دامنه چرخیده ۹۰ درجه*

در حالت چرخش ۹۰ درجه، اگرچه اختلاف دامنه همچنان قابل توجه است، اما loss اعتبارسنجی از همان ابتدا در بازه‌ی نسبتاً ثابتی قرار دارد و پس از یک کاهش جزئی در میانه‌ی آموزش، مجدداً با نوسان خفیفی افزایش می‌یابد. این رفتار نشان می‌دهد که مدل بدون نوسانات شدید، به یک ناحیه‌ی پایدار از نظر خطا همگرا شده است. در مقابل، loss آموزش از ابتدا بسیار پایین بوده و به‌صورت یکنواخت به مقادیر نزدیک به صفر کاهش می‌یابد.

در نمودار دقت نیز مشاهده می‌شود که دقت اعتبارسنجی روندی افزایشی در اپیاک‌های ابتدایی دارد و سپس با نوسانات جزئی در یک بازه‌ی محدود تثبیت می‌شود. از سوی دیگر، دقت آموزش در سطح بالایی قرار داشته و با تغییرات بسیار کم، تقریباً ثابت باقی می‌ماند. این الگو نشان می‌دهد که وزن‌دهی مجدد باعث ایجاد ناپایداری در فرآیند یادگیری نشده و مدل توانسته به همگرایی نسبتاً پایداری دست یابد، هرچند فاصله‌ی اندکی بین عملکرد آموزش و اعتبارسنجی همچنان حفظ شده است.

جدول ۳–۳: عملکرد نهایی روش ***Reweighting*** *بر روی داده‌های تمیز و دامنه هدف*

حالت	**Clean Accuracy**	**Target Accuracy**
Reweighting (Rot15)	۰.۹۸۵۶	۰.۹۵۵۰
Reweighting (Rot90)	۰.۹۸۴۹	۰.۱۶۹۱

با توجه به نتایج نهایی، مشاهده می‌شود که:

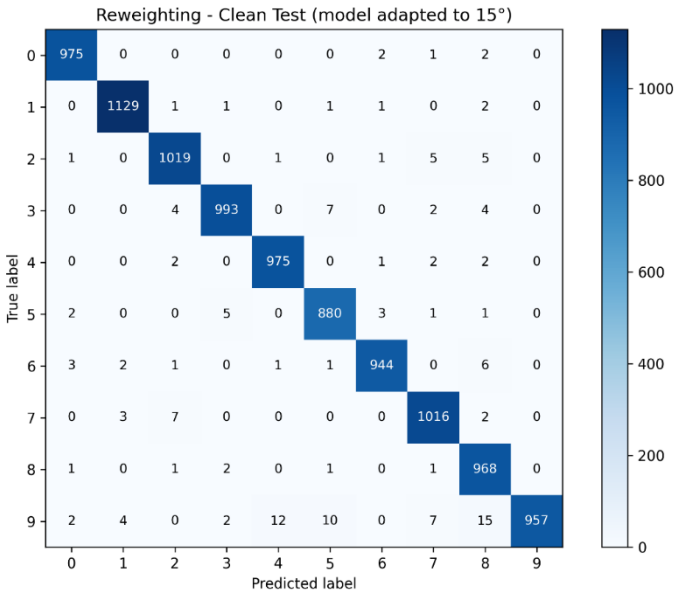
- در چرخش ۱۵ درجه، روش Reweighting نسبت به مدل پایه بهبود محدودی در دامنه هدف ایجاد کرده است؛ دقت از ۰.۹۴۸۱ به ۰.۹۵۵۰ رسیده است. در مقابل، دقت روی داده‌های تمیز اندکی کاهش یافته و از ۰.۹۸۶۲ به ۰.۹۸۵۶ رسیده است. این رفتار نشان می‌دهد که در تغییرات ملایم دامنه، وزن‌دهی مجدد می‌تواند تا حدی به بهبود عملکرد روی دامنه هدف کمک کند، اما این بهبود می‌تواند با افت بسیار جزئی در دامنه منبع همراه باشد.
- در چرخش ۹۰ درجه، روش Reweighting عملاً بهبود محسوسی در دامنه هدف ایجاد نکرده است؛ دقت از ۰.۱۶۹۲ به ۰.۱۶۹۱ رسیده و تقریباً بدون تغییر باقی مانده است. با این حال، دقت روی داده‌های تمیز نیز فقط کمی کاهش

یافته و از ۰.۹۸۶۲ به ۰.۹۸۴۹ رسیده است. این رفتار را می‌توان به محدودیت ذاتی Reweighting نسبت داد؛ زیرا این روش مبتنی بر فرض تفاوت در توزیع داده‌ها است، در حالی که در شیفت‌های شدید مانند چرخش ۹۰ درجه، تغییرات ایجادشده در نمایش ویژگی‌ها نیز قابل توجه است و صرف بازوزن‌دهی نمونه‌ها برای جبران این اختلاف کافی نخواهد بود.

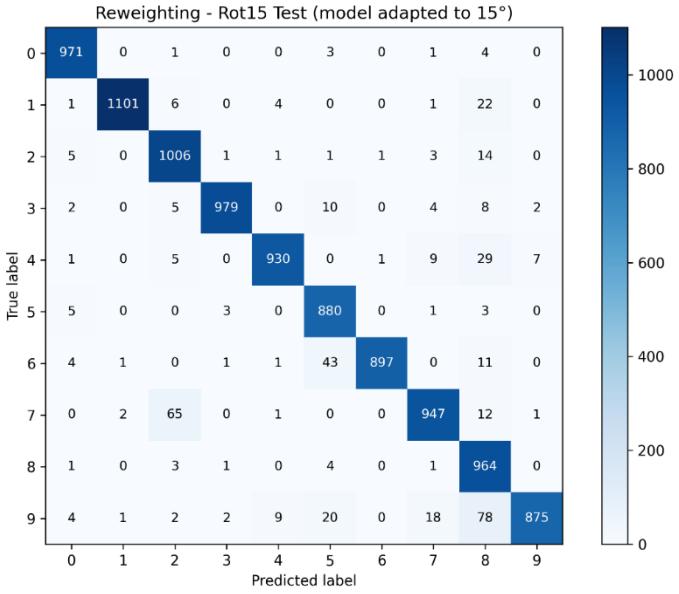
در مجموع، نتایج نشان می‌دهند که عملکرد مناسب روی داده‌های تمیز لزوماً به معنی موفقیت در دامنه هدف نیست. به‌ویژه در حالت چرخش ۹۰ درجه، مدل با وجود حفظ دقت بالا روی دامنه منبع، همچنان در دامنه هدف عملکرد بسیار ضعیفی دارد؛ بنابراین، ارزیابی هم‌زمان هر دو دامنه برای تحلیل روش‌های تطبیق دامنه ضروری است.

۳-۳-۴-۲- بررسی ماتریس درهم‌ریختگی و دقت در هر کلاس در روش Reweighting

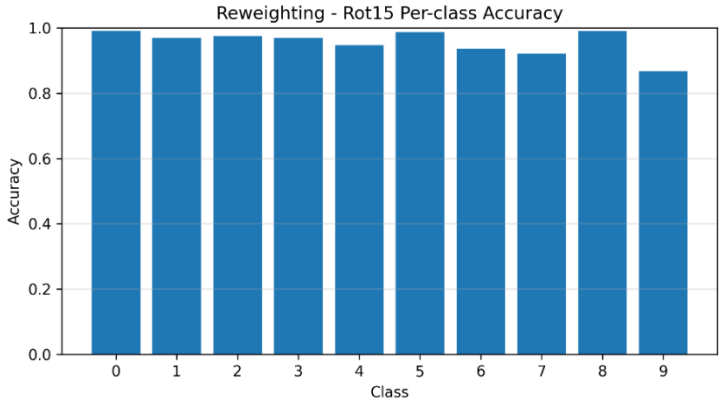
در این زیربخش، عملکرد مدلِ تطبیق‌یافته با روش Reweighting در سطح هر کلاس بررسی می‌شود. برای این منظور، ابتدا نتایج مربوط به حالتی ارائه می‌شود که مدل برای دامنه‌ی چرخیده‌ی ۱۵ درجه تطبیق داده شده است؛ در این بخش، ماتریس درهم‌ریختگی روی داده‌های Clean، سپس ماتریس درهم‌ریختگی روی داده‌های Rot15 و در ادامه نمودار دقت در هر کلاس برای داده‌های Rot15 آورده می‌شود. پس از آن، تحلیل کلی این سه خروجی ارائه خواهد شد. در ادامه نیز همین روند برای حالتی تکرار می‌شود که مدل برای دامنه‌ی چرخیده‌ی ۹۰ درجه تطبیق داده شده است.



*شکل ۳–۲۴: ماتریس درهم‌ریختگی مدل **Reweighting** روی داده‌های **Clean**، در حالتی که مدل برای دامنه ۱۵ درجه تطبیق داده شده است.*



*شکل ۳–۲۵: ماتریس درهم‌ریختگی مدل **Reweighting** روی داده‌های **Rot15**، در حالتی که مدل برای دامنه ۱۵ درجه تطبیق داده شده است.*

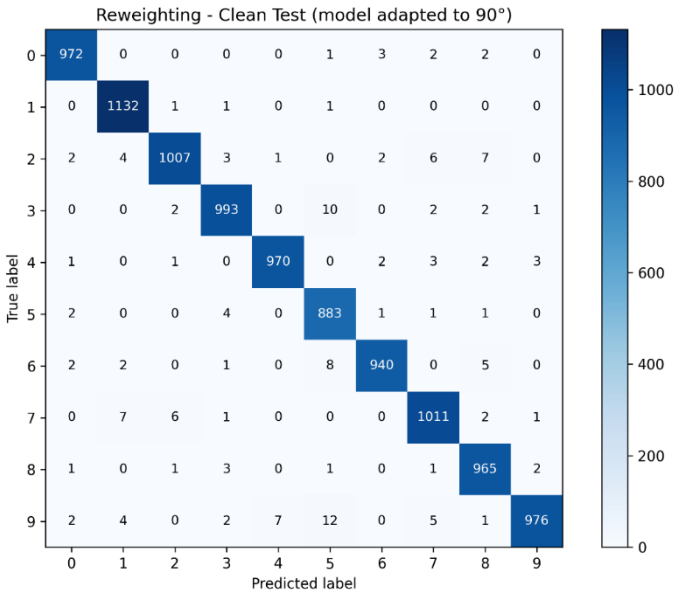


شکل ۳–۲۶: دقت مدل ***Reweighting*** *در هر کلاس برای داده‌های* ***Rot15***

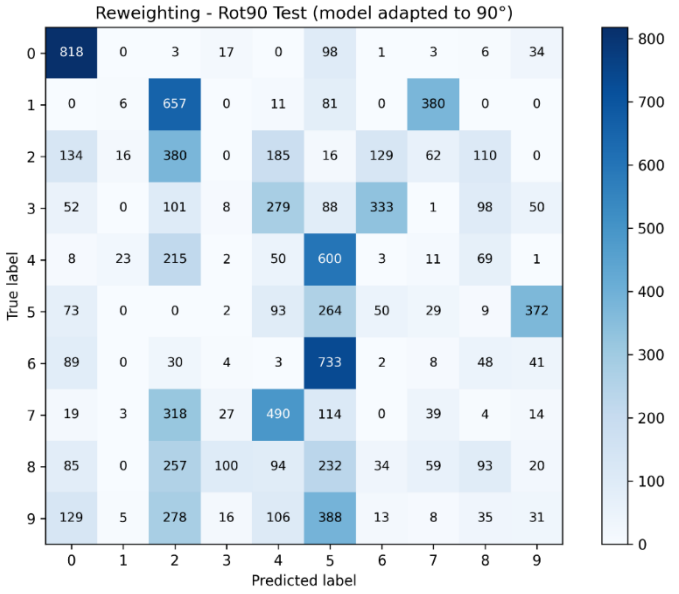
تحلیل این سه خروجی نشان می‌دهد که در حالت تطبیق با دامنه‌ی ۱۵ درجه، ساختار کلی تصمیم‌گیری مدل تا حد زیادی حفظ شده و تطبیق دامنه تأثیر مخربی بر عملکرد در دامنه‌ی اصلی نداشته است. در داده‌های Clean، ماتریس درهم‌ریختگی همچنان به‌شدت قطرمحور است و خطاها بسیار محدود و پراکنده‌اند؛ بنابراین مدل پس از Reweighting نیز توانایی بالایی در تشخیص صحیح نمونه‌های دامنه‌ی مرجع حفظ کرده است.

در داده‌های Rot15 نیز تمرکز اصلی پیش‌بینی‌ها روی قطر باقی مانده، اما چند الگوی خطای مشخص قابل مشاهده است. مهم‌ترین مورد، اشتباه قابل توجه کلاس ۷ با ۲ است. همچنین کلاس‌های ۱ و ۲ در برخی نمونه‌ها با ۸ اشتباه گرفته می‌شوند که نشان‌دهنده‌ی افزایش شباهت ساختاری این ارقام پس از چرخش است. علاوه بر این، کلاس ۶ در تعداد قابل توجهی از نمونه‌ها به‌صورت ۵ پیش‌بینی می‌شود و برای کلاس ۹ نیز پراکندگی خطا عمدتاً به سمت ۸، ۷ و تا حدی ۵ دیده می‌شود. کلاس ۴ نیز در برخی موارد به ۸ و ۷ اشتباه گرفته می‌شود. این الگوها نشان می‌دهند که چرخش باعث افزایش شباهت بصری میان برخی ارقام شده و مرزهای تصمیم‌گیری مدل را در این نواحی تضعیف کرده است.

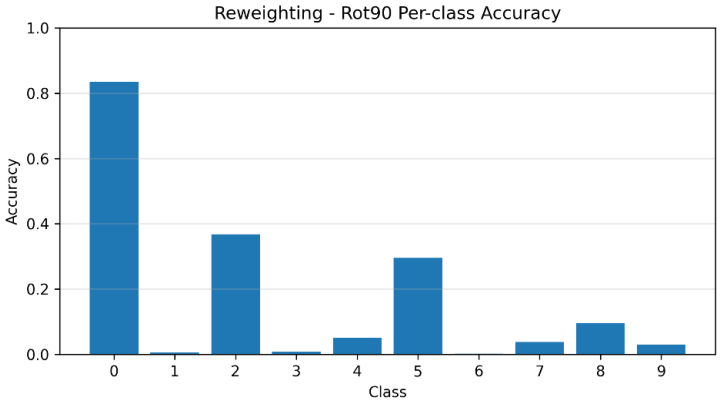
نمودار دقت کلاس‌به‌کلاس این تحلیل را تأیید می‌کند؛ به‌طوری‌که پایین‌ترین دقت مربوط به کلاس ۹ است و پس از آن کلاس‌های ۷ و ۶ قرار می‌گیرند، در حالی که سایر کلاس‌ها همچنان دقتی بالا و نزدیک به مقدار ایده‌آل دارند. در مجموع، می‌توان گفت که در چرخش ۱۵ درجه، خطاها به چند کلاس خاص متمرکز شده‌اند، اما عملکرد کلی مدل همچنان پایدار و قابل قبول باقی مانده است.



*شکل ۳–۲۷: ماتریس درهم‌ریختگی مدل **Reweighting** روی داده‌های **Clean**، در حالتی که مدل برای دامنه ۹۰ درجه تطبیق داده شده است.*



*شکل ۳–۲۸: ماتریس درهم‌ریختگی مدل **Reweighting** روی داده‌های **Rot90**، در حالتی که مدل برای دامنه ۹۰ درجه تطبیق داده شده است.*



*شکل ۳–۲۹: دقت مدل **Reweighting** در هر کلاس برای داده‌های **Rot90***

در حالت تطبیق با دامنه‌ی ۹۰ درجه، همچنان یک شکاف واضح میان عملکرد مدل روی داده‌های Clean و داده‌های هدف مشاهده می‌شود. در ماتریس درهم‌ریختگی مربوط به داده‌های Clean، ساختار تصمیم‌گیری مدل تا حد زیادی حفظ شده است؛ به‌طوری‌که بیشتر پیش‌بینی‌ها روی قطر اصلی متمرکز هستند و خطاها محدود و پراکنده‌اند. بنابراین، اعمال Reweighting موجب تخریب عملکرد مدل در دامنه‌ی مرجع نشده است.

اما در داده‌های Rot90 وضعیت به‌مراتب پیچیده‌تر است و ماتریس درهم‌ریختگی دیگر ساختار قطرمحور مشخصی ندارد. تنها کلاس ۰ همچنان با دقت بالا شناسایی می‌شود، در حالی که سایر کلاس‌ها دچار ابهام شدید هستند. به‌طور مشخص، کلاس ۱ عمدتاً به ۲ و سپس به ۷ پیش‌بینی می‌شود. کلاس ۲ علاوه بر پیش‌بینی صحیح محدود، به‌طور قابل توجهی با کلاس‌های ۴، ۰، ۶ و ۸ اشتباه گرفته می‌شود. کلاس ۳ به‌شدت با ۶ و ۴ دچار اختلاط است و دقت بسیار پایینی دارد. کلاس ۴ عمدتاً به ۵ و سپس ۲ نگاشت می‌شود. برای کلاس ۵، خطاها عمدتاً به سمت ۹ دیده می‌شوند. کلاس ۶ در بخش بزرگی از نمونه‌ها به ۵ پیش‌بینی می‌شود و همچنین با ۰ نیز اشتباه دارد. کلاس ۷ بیشتر با ۴ و ۲ دچار ابهام است. در کلاس ۸، خطاها به‌صورت گسترده به کلاس‌های ۲، ۵، ۳، ۴ و ۰ پخش شده‌اند. در نهایت، کلاس ۹ نیز عمدتاً با ۵، ۲ و ۰ اشتباه گرفته می‌شود.

نمودار دقت کلاس‌به‌کلاس این وضعیت را به‌خوبی تأیید می‌کند. کلاس ۰ تنها کلاسی است که دقت بالایی دارد. پس از آن، کلاس‌های ۲ و ۵ عملکرد نسبتاً بهتری نسبت به سایر کلاس‌ها نشان می‌دهند، اما همچنان فاصله‌ی قابل توجهی با حالت ایده‌آل دارند. کلاس‌های ۴، ۷، ۸ و ۹ در سطحی ضعیف قرار دارند و کلاس‌های ۱، ۳ و ۶ بدترین عملکرد را دارند؛ به‌ویژه کلاس‌های ۳ و ۶ که دقت آن‌ها بسیار پایین است.

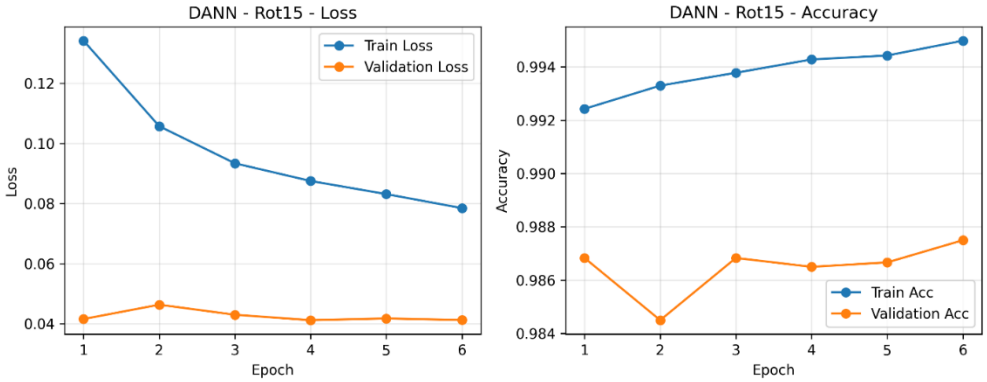
در مجموع، این نتایج نشان می‌دهند که در چرخش ۹۰ درجه، خطاهای مدل عمدتاً در کلاس‌هایی متمرکز می‌شوند که پس از چرخش شدید، شباهت ساختاری بالایی با چندین رقم دیگر پیدا می‌کنند. بنابراین، اگرچه Reweighting توانسته عملکرد مدل را روی دامنه‌ی مرجع حفظ کند، اما برای رفع ابهام‌های ساختاری ناشی از یک تغییر دامنه‌ی شدید کافی نیست.

۳-۳-۵- تطبیق دامنه با استفاده از هم‌ترازی ویژگی‌ها (DANN)

در این بخش، روش Feature Alignment با استفاده از DANN بررسی می‌شود. ایده اصلی این روش آن است که مدل، علاوه بر یادگیری ویژگی‌های مناسب برای طبقه‌بندی، ویژگی‌هایی بیاموزد که تا حد امکان بین دامنه منبع و دامنه هدف مشترک باشند. به بیان دیگر، DANN تلاش می‌کند نمایش‌های میانی مدل را به‌گونه‌ای شکل دهد که تشخیص کلاس همچنان ممکن باشد، اما تمایز میان منبع و هدف برای یک طبقه‌بند دامنه دشوار شود. در نتیجه، انتظار می‌رود مدل نسبت به تغییر دامنه مقاوم‌تر شود و در داده‌های هدف افت عملکرد کمتری داشته باشد.

۳-۳-۵-۱- بررسی روند آموزش و عملکرد مدل در روش هم‌ترازی ویژگی‌ها (DANN)

در این زیربخش، روند تغییرات خطا و دقت مدل در طول فرآیند آموزش بررسی می‌شود تا پایداری یادگیری و نحوه‌ی همگرایی مدل مورد تحلیل قرار گیرد. این نمودارها دید مناسبی از رفتار مدل در طول آموزش و کیفیت یادگیری ویژگی‌ها ارائه می‌دهند.

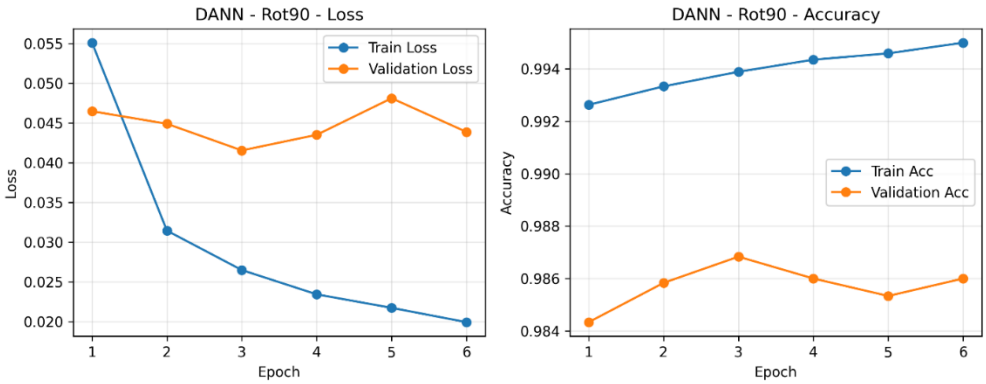


*شکل ۳–۳۰: روند تغییرات خطا و دقت مدل **DANN** در حالتی که مدل برای دامنه ۱۵ درجه تطبیق داده شده است. ارزیابی تست در طول آموزش بر روی داده‌های تمیز انجام شده است.*

در حالت تطبیق با دامنه‌ی ۱۵ درجه، نمودارها نشان می‌دهند که فرآیند آموزش از پایداری مناسبی برخوردار است. خطای آموزش روندی کاهشی و نسبتاً یکنواخت دارد و به‌تدریج کاهش می‌یابد. در مقابل، خطای اعتبارسنجی در سطح پایینی قرار دارد و با وجود نوسانات جزئی، در یک بازه‌ی محدود باقی می‌ماند و روند کلی آن پایدار است، به‌طوری‌که نشانه‌ای از واگرایی یا افزایش تدریجی خطا مشاهده نمی‌شود.

در نمودار دقت نیز مشاهده می‌شود که دقت آموزش در سطح بسیار بالایی قرار دارد و به‌صورت یکنواخت افزایش می‌یابد. در مقابل، دقت اعتبارسنجی دارای نوسانات خفیف در ایپاک‌های ابتدایی است، اما در ادامه در یک بازه‌ی فشرده و بالا تثبیت می‌شود. فاصله‌ی میان دقت آموزش و دقت اعتبارسنجی کم بوده و نشان‌دهنده‌ی تعمیم مناسب مدل است.

این رفتار نشان می‌دهد که افزودن مؤلفه‌ی هم‌ترازی دامنه در DANN باعث ناپایداری در فرآیند یادگیری نشده و مدل توانسته تعادل مناسبی میان یادگیری ویژگی‌های تفکیک‌پذیر و دامنه‌ناوابسته برقرار کند. همچنین مشاهده می‌شود که این روش بهبودهایی در عملکرد مدل بر روی داده‌های دامنه هدف (۱۵ درجه چرخیده) ایجاد کرده است.



*شکل ۳–۳۱: روند تغییرات خطا و دقت مدل **DANN** در حالتی که مدل برای دامنه ۹۰ درجه تطبیق داده شده است؛ ارزیابی تست در طول آموزش بر روی داده‌های تمیز انجام شده است.*

در حالت تطبیق با دامنه‌ی ۹۰ درجه نیز روند آموزش از نظر عددی پایدار است. خطای آموزش به‌صورت یکنواخت و قابل توجهی کاهش پیدا می‌کند، در حالی که خطای اعتبارسنجی در یک بازه‌ی محدود نوسان دارد و تغییرات آن بدون روند افزایشی یا واگرایی شدید است. این رفتار نشان می‌دهد که فرآیند یادگیری از نظر بهینه‌سازی پایدار بوده و نشانه‌ای از بیش‌برازش شدید مشاهده نمی‌شود.

در نمودار دقت نیز مشاهده می‌شود که دقت آموزش روندی صعودی و یکنواخت دارد و به مقادیر بسیار بالایی می‌رسد. در مقابل، دقت اعتبارسنجی با وجود نوسانات جزئی، در یک بازه‌ی فشرده باقی می‌ماند؛ به‌طوری‌که پس از یک افزایش اولیه، افت خفیفی را تجربه کرده و سپس مجدداً به مقدار قبلی نزدیک می‌شود. در مجموع، این الگو نشان‌دهنده‌ی همگرایی مناسب مدل و پایداری عملکرد آن در طول آموزش است.

با این حال، این پایداری در روند آموزش لزوماً به معنای تطبیق موفق با دامنه‌ی هدف نیست. در شرایط تغییر دامنه‌ی شدید، هم‌ترازی ویژگی‌ها در DANN اگرچه به یادگیری پایدار کمک می‌کند، اما به‌تنهایی برای ایجاد نمایش‌های کاملاً دامنه‌ناوابسته کافی نیست و در نتیجه چالش‌های تعمیم به دامنه‌ی هدف همچنان باقی می‌مانند.

*جدول ۳–۴: عملکرد نهایی روش هم‌ترازی ویژگی‌ها (**DANN**) بر روی داده‌های تمیز و دامنه هدف*

حالت	**Clean Accuracy**	**Target Accuracy**
DANN (Rot15)	۰.۹۸۸۸	۰.۹۵۹۱
DANN (Rot90)	۰.۹۸۸۷	۰.۱۶۷۶

تحلیل نتایج نهایی نشان می‌دهد که در چرخش ۱۵ درجه، روش هم‌ترازی ویژگی‌ها (DANN) نسبت به مدل پایه بهبود قابل‌توجهی روی دامنه هدف ایجاد کرده است (از ۰.۹۴۸۱ به ۰.۹۵۹۱)، در حالی که دقت روی داده‌های تمیز نیز افزایش یافته و از ۰.۹۸۶۲ به ۰.۹۸۸۸ رسیده است. این رفتار نشان می‌دهد که در تغییرات دامنه‌ی خفیف، هم‌ترازی ویژگی‌ها در DANN می‌تواند

به‌صورت مؤثری توزیع دو دامنه را به یکدیگر نزدیک کرده و بدون ایجاد trade-off محسوس، به بهبود تعمیم در دامنه هدف کمک کند.

در مقابل، در چرخش ۹۰ درجه، اگرچه دقت روی داده‌های تمیز همچنان بالا باقی مانده و اندکی نسبت به مدل پایه بهبود یافته است (از ۰.۹۸۶۲ به ۰.۹۸۸۷)، اما عملکرد روی دامنه هدف نه‌تنها بهبود قابل‌توجهی نداشته، بلکه اندکی افت نیز نشان می‌دهد (از ۰.۱۶۹۲ به ۰.۱۶۷۶) و همچنان در سطح بسیار پایینی باقی مانده است. این موضوع نشان می‌دهد که اگرچه DANN توانسته نمایش‌هایی پایدار برای دامنه مرجع یاد بگیرد، اما این هم‌ترازی برای غلبه بر یک تغییر دامنه‌ی شدید کافی نبوده است.

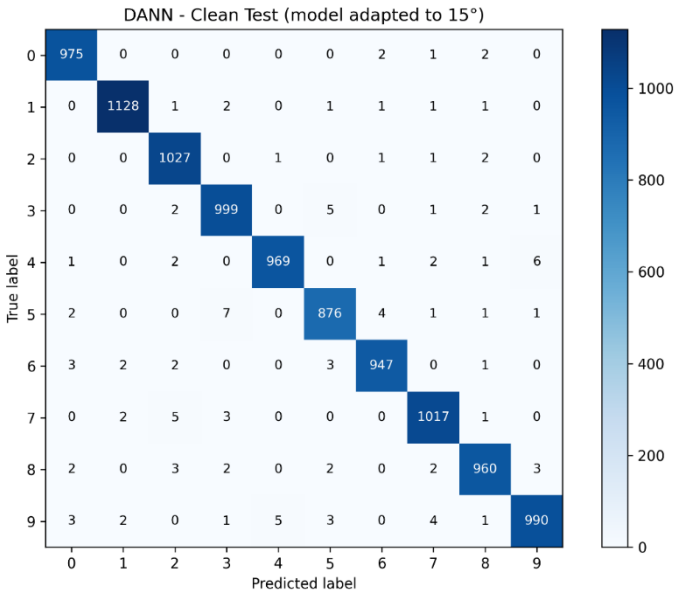
از منظر علمی، یکی از دلایل این رفتار آن است که DANN مبتنی بر هم‌ترازسازی توزیع ویژگی‌ها در سطح کلی[۲۵] است و فرض می‌کند که با نزدیک کردن توزیع ویژگی‌ها، مرزهای تصمیم نیز قابل تعمیم خواهند بود. اما در شیفت‌های شدید مانند چرخش ۹۰ درجه، نگاشت بین ویژگی‌ها و برچسب‌ها به‌صورت مؤثری دچار تغییر ساختاری می‌شود و کلاس‌ها در فضای ویژگی با یکدیگر هم‌پوشانی پیدا می‌کنند. در چنین شرایطی، هم‌ترازسازی صرف توزیع‌ها بدون در نظر گرفتن ساختار کلاس‌ها می‌تواند حتی منجر به تداخل بیشتر کلاس‌ها شود.

در مجموع، این نتایج تأیید می‌کنند که عملکرد مناسب روی داده‌های تمیز -حتی اگر بهبود هم پیدا کند- لزوماً به معنای تطبیق موفق با دامنه هدف نیست. معیار اصلی، بهبود روی دامنه هدف است، و در اینجا DANN تنها در سناریوهای با تغییر دامنه‌ی ملایم عملکرد مؤثری نشان داده، در حالی که در مواجهه با تغییرات شدید، همچنان با محدودیت جدی روبه‌رو است..

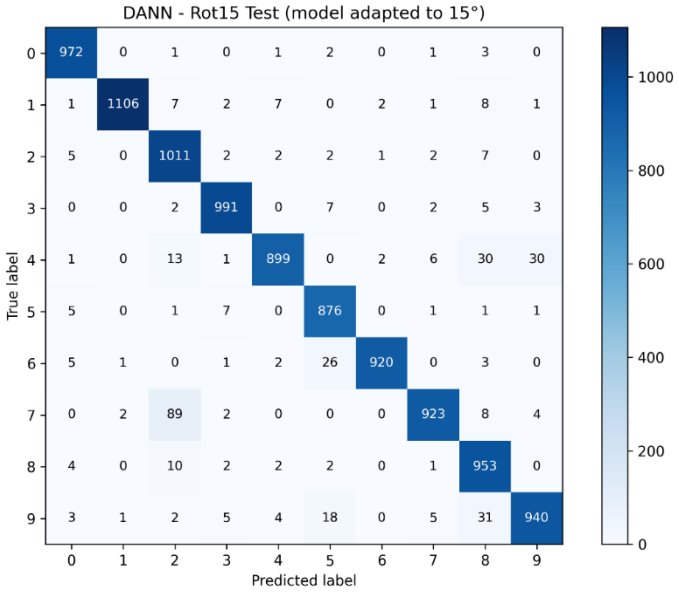
۳-۳-۵-۲- بررسی ماتریس درهم‌ریختگی و دقت در هر کلاس در روش هم‌ترازی ویژگی‌ها (DANN)

در این زیربخش، برای تحلیل دقیق‌تر رفتار مدل، از ماتریس درهم‌ریختگی و نمودار دقت به‌ازای هر کلاس استفاده می‌شود. این بررسی کمک می‌کند مشخص شود مدل در کدام کلاس‌ها عملکرد بهتری دارد و کدام ارقام بیشتر دچار اشتباه می‌شوند. همان‌طور که انتظار می‌رود، این تحلیل نسبت به دقت کلی دید عمیق‌تری از کیفیت تطبیق دامنه ارائه می‌دهد.

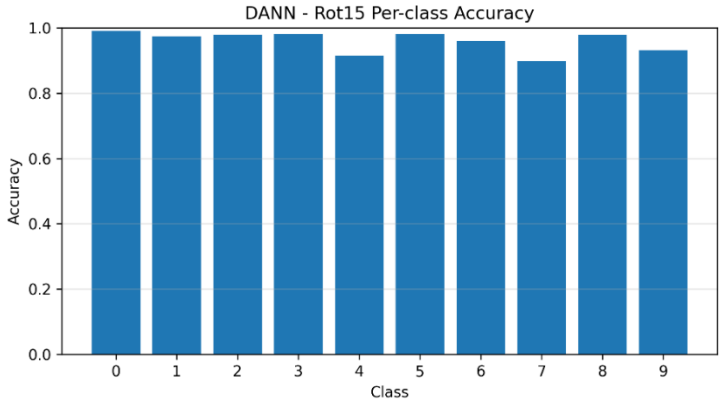
[25] Marginal feature alignment



*شکل ۳–۳۲: ماتریس درهم‌ریختگی داده‌های **Clean** برای مدل **DANN** در حالت تطبیق با چرخش ۱۵ درجه*



*شکل ۳–۳۳: ماتریس درهم‌ریختگی داده‌های **Rot15** برای مدل **DANN** (مدل تطبیق داده‌شده روی ۱۵ درجه)*



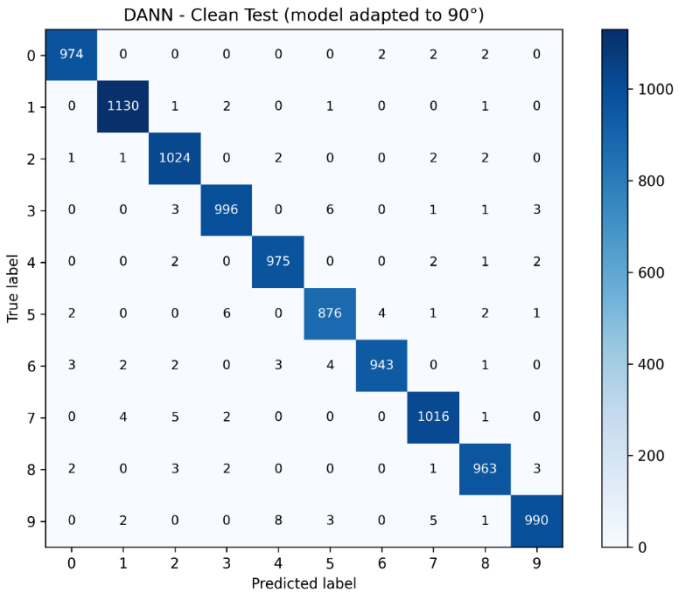
*شکل ۳–۳۴: دقت به‌ازای هر کلاس برای داده‌های **Rot15** برای مدل **DANN***

در حالت چرخش ۱۵ درجه، ماتریس درهم‌ریختگی روی داده‌های Clean نشان می‌دهد که مدل همچنان ساختار تصمیم‌گیری بسیار دقیقی دارد و تقریباً تمام پیش‌بینی‌ها روی قطر اصلی متمرکز هستند. خطاهای خارج از قطر بسیار محدود و پراکنده‌اند؛ به‌عنوان مثال، در کلاس ۳ تعداد کمی خطا به سمت ۵ (حدود ۵ نمونه) و در کلاس ۵ به سمت ۳ (حدود ۷ نمونه) دیده می‌شود. همچنین در کلاس ۴ مقدار کمی اشتباه با ۹ (حدود ۶ نمونه) وجود دارد. این الگو نشان می‌دهد که هم‌ترازی ویژگی‌ها در DANN باعث حفظ توانایی تفکیک کلاس‌ها در دامنه مرجع شده است.

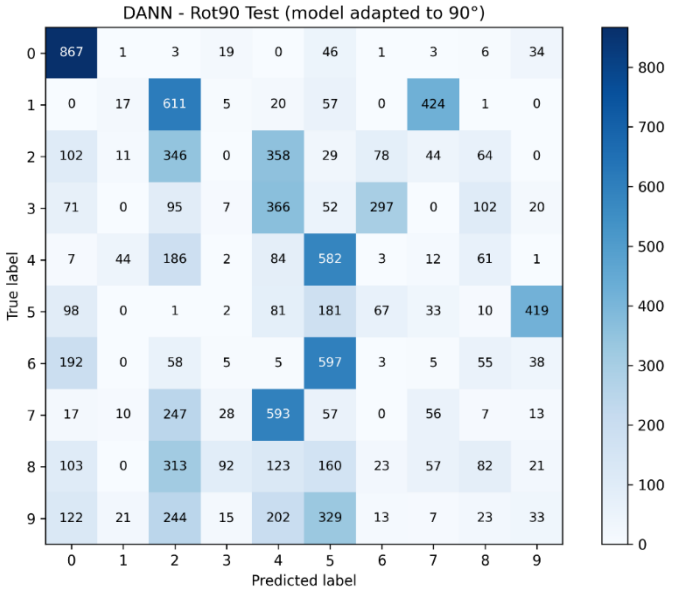
در داده‌های Rot15 نیز اگرچه میزان خطا نسبت به حالت Clean افزایش یافته، اما ماتریس همچنان تا حد زیادی قطرمحور باقی مانده است. با این حال، چند الگوی خطای مشخص و نسبتاً پررنگ دیده می‌شود. مهم‌ترین مورد، اشتباه کلاس ۷ با ۲ است (حدود ۸۹ نمونه) که نشان‌دهنده‌ی ابهام جدی بین این دو کلاس پس از چرخش است. همچنین در کلاس ۴، خطاهای قابل توجهی به سمت ۸ و ۹ (هرکدام حدود ۳۰ نمونه) دیده می‌شود. در کلاس ۶ نیز تعداد قابل توجهی از نمونه‌ها به ۵ نگاشت شده‌اند (حدود ۲۶ نمونه). علاوه بر این، در کلاس ۹ اشتباهاتی به سمت ۸ (حدود ۳۱ نمونه) و ۵ (حدود ۱۸ نمونه) مشاهده می‌شود. در کلاس ۱ نیز مقداری ابهام با کلاس‌های ۲ و ۴ (هرکدام حدود ۷ نمونه) وجود دارد. این الگوها نشان می‌دهند که چرخش باعث افزایش شباهت ساختاری میان برخی ارقام شده و مدل در این نواحی دچار خطای سیستماتیک می‌شود.

نمودار دقت به‌ازای هر کلاس این تحلیل را تأیید می‌کند. پایین‌ترین دقت مربوط به کلاس ۷ است که تحت تأثیر خطای شدید با کلاس ۲ قرار دارد. پس از آن کلاس‌های ۴ و ۶ نیز به دلیل خطاهای مشخص با ۸، ۹ و ۵ عملکرد ضعیف‌تری دارند. در مقابل، کلاس‌هایی مانند ۰، ۱، ۲ و ۵ همچنان دقت بالایی را حفظ کرده‌اند و نسبت به تغییر دامنه مقاوم‌تر هستند.

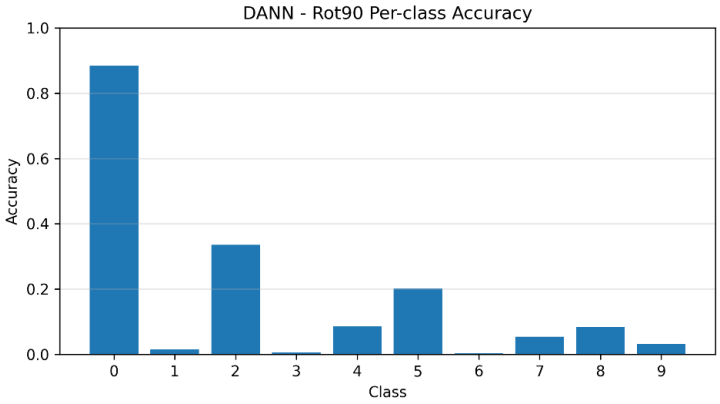
در مجموع، می‌توان گفت که در تغییر دامنه‌ی ملایم (چرخش ۱۵ درجه)، DANN توانسته تعادل مناسبی بین حفظ عملکرد در دامنه مرجع و بهبود تعمیم به دامنه هدف برقرار کند. خطاها عمدتاً متمرکز بر چند جفت‌کلاس خاص هستند که پس از چرخش، شباهت بصری بیشتری پیدا می‌کنند، در حالی که در سایر کلاس‌ها عملکرد مدل همچنان پایدار و نزدیک به حالت ایده‌آل باقی مانده است..



*شکل ۳–۳۵: ماتریس درهم‌ریختگی داده‌های **Clean** برای مدل **DANN** در حالت تطبیق با چرخش ۹۰ درجه*



*شکل ۳–۳۶: ماتریس درهم‌ریختگی داده‌های **Rot90** برای مدل **DANN** (مدل تطبیق داده‌شده روی ۹۰ درجه)*



*شکل ۳–۳۷: دقت به‌ازای هر کلاس برای داده‌های **Rot90** برای مدل **DANN***

در حالت چرخش ۹۰ درجه، ماتریس درهم‌ریختگی روی داده‌های Clean همچنان عملکرد بسیار دقیقی را نشان می‌دهد و پیش‌بینی‌ها تقریباً به‌طور کامل روی قطر اصلی متمرکز هستند. خطاهای خارج از قطر بسیار محدود و پراکنده‌اند؛ بنابراین DANN در این حالت نیز باعث تخریب عملکرد مدل روی دامنه مرجع نشده است. با این حال، وضعیت روی داده‌های Rot90 کاملاً متفاوت است و ماتریس درهم‌ریختگی ساختار قطرمحور خود را تا حد زیادی از دست داده است.

به‌طور مشخص، کلاس ۱ عمدتاً به کلاس‌های ۲ و ۷ پیش‌بینی می‌شود و سهم پیش‌بینی صحیح آن بسیار پایین است. کلاس ۲ علاوه بر پیش‌بینی صحیح محدود، به‌طور قابل توجهی با کلاس‌های ۴ و ۰ اشتباه گرفته می‌شود. کلاس ۳ بیشتر به کلاس‌های ۴، ۶، ۸ و ۲ نگاشت شده و تقریباً به‌ندرت درست تشخیص داده می‌شود. کلاس ۴ عمدتاً با ۵ و سپس ۲ اشتباه می‌شود. کلاس ۵ نیز بیشترین خطا را با ۹ دارد و بخشی از نمونه‌های آن با ۰ و ۴ اشتباه گرفته می‌شوند. کلاس ۶ به‌شدت به ۵ و سپس ۰ نگاشت شده است. کلاس ۷ عمدتاً با ۴ و ۲ دچار ابهام می‌شود. کلاس ۸ نیز خطاهای گسترده‌ای به سمت ۲، ۵، ۴ و ۰ دارد. در نهایت، کلاس ۹ بیشتر با ۵، ۲ و ۴ اشتباه گرفته می‌شود.

نمودار دقت کلاس‌به‌کلاس این تحلیل را تأیید می‌کند. تنها کلاس ۰ دقت نسبتاً بالایی دارد و پس از آن کلاس‌های ۲ و ۵ عملکرد نسبی بهتری نشان می‌دهند، اما همچنان فاصله زیادی با عملکرد مطلوب دارند. در مقابل، کلاس‌های ۱، ۳ و ۶ بسیار ضعیف عمل کرده‌اند و کلاس‌های ۴، ۷، ۸ و ۹ نیز در سطح پایینی از دقت قرار دارند.

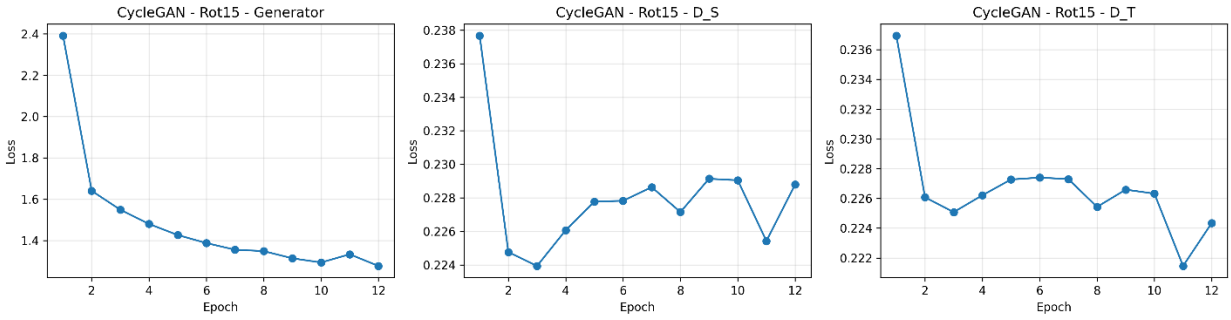
در نتیجه، برخلاف حالت چرخش ۱۵ درجه، در اینجا خطاها هم شدیدتر و هم ساختاریافته‌تر هستند. این الگو نشان می‌دهد که چرخش ۹۰ درجه باعث تغییر جدی در نمایش هندسی ارقام شده و ویژگی‌های یادگرفته‌شده، حتی با وجود استفاده از DANN، برای تفکیک مناسب کلاس‌ها کافی نیستند. بنابراین، در تغییرات دامنه‌ی شدید، هم‌ترازی کلی ویژگی‌ها به‌تنهایی نمی‌تواند شکاف میان دو دامنه را به‌طور مؤثر پر کند.

۳-۳-۶- تطبیق دامنه با استفاده از ترجمه دامنه[۲۶] مبتنی بر CycleGAN

در این بخش، رویکرد Domain Translation بررسی می‌شود. ایده اصلی این رویکرد آن است که به‌جای هم‌تراز کردن مستقیم وزن نمونه‌ها یا ویژگی‌ها، داده‌های یک دامنه به دامنه دیگر ترجمه شوند تا شکاف ظاهری میان دو دامنه کاهش یابد. در این تمرین، برای این منظور از CycleGAN استفاده شده است. CycleGAN با یادگیری دو نگاشت در دو جهت منبع ⟵ هدف و هدف ⟵ منبع تلاش می‌کند تصاویری تولید کند که از نظر ظاهر به دامنه مقصد نزدیک باشند، در حالی که محتوای اصلی آن‌ها تا حد ممکن حفظ شود. در نتیجه، انتظار می‌رود داده‌های دامنه هدف پس از ترجمه به دامنه منبع، برای طبقه‌بند آموزش‌دیده روی MNIST اصلی قابل‌فهم‌تر شوند.

۳-۳-۶-۱- بررسی روند آموزش اجزای CycleGAN، نمونه‌های تولیدشده و عملکرد نهایی

در این زیربخش، ابتدا روند تغییرات خطای اجزای مختلف CycleGAN بررسی می‌شود، سپس نمونه‌های ترجمه‌شده تحلیل می‌گردند و در نهایت عملکرد نهایی این رویکرد نسبت به مدل پایه ارزیابی می‌شود. باید توجه داشت که در این روش، دقت روی داده‌های تمیز همان مرجع baseline است، زیرا طبقه‌بند اصلی مستقیماً تغییر نکرده و تنها ورودی دامنه هدف پیش از طبقه‌بندی ترجمه می‌شود.



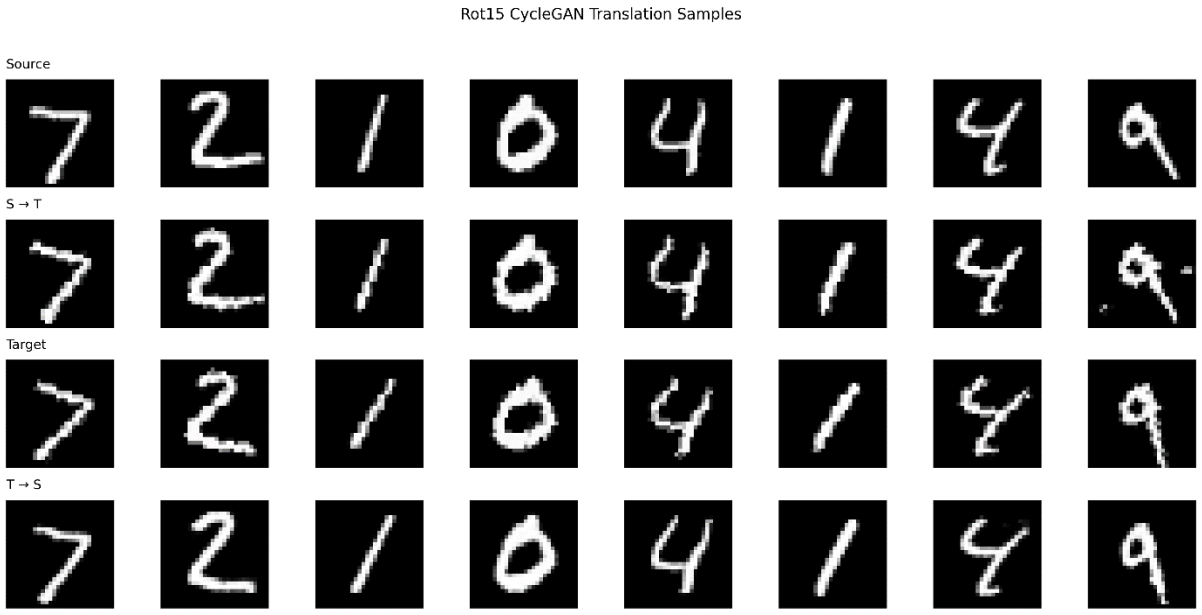
*شکل ۳–۳۸: روند تغییرات خطای **Generator** و دو **Discriminator** در **CycleGAN** برای دامنه ۱۵ درجه*

در حالت چرخش ۱۵ درجه، خطای Generator روندی کاهشی و نسبتاً یکنواخت دارد؛ به‌طوری‌که کاهش آن در ایپاک‌های ابتدایی شدیدتر است (از حدود ۲.۴ به حدود ۱.۶) و سپس با شیب ملایم‌تری تا حدود ۱.۳ ادامه پیدا می‌کند. این رفتار نشان می‌دهد که Generator به‌تدریج در حال یادگیری نگاشت بین دو دامنه بوده و فرآیند آموزش از نظر آن پایدار پیش می‌رود.

در مقابل، خطای Discriminator مربوط به دامنه مبدأ (D_S) پس از یک افت اولیه (از حدود ۰.۲۳۸ به حدود ۰.۲۲۴)، در ادامه در یک بازه‌ی محدود نوسان می‌کند و روند افزایشی مشخصی ندارد. این نوسانات خفیف نشان می‌دهد که D_S و Generator به نوعی تعادل نسبی رسیده‌اند و هیچ‌کدام به‌صورت کامل بر دیگری غلبه نکرده است. از سوی دیگر، خطای Discriminator مربوط به دامنه هدف (D_T) نیز پس از کاهش اولیه، در محدوده‌ای نزدیک به ۰.۲۲۵ تا ۰.۲۲۸ نوسان می‌کند و در ایپاک‌های پایانی حتی اندکی کاهش بیشتری نشان می‌دهد.

[26] Domain Translation

در مجموع، این الگوها نشان می‌دهند که بین Generator و Discriminatorها تعادل پایداری برقرار شده و آموزش دچار واگرایی یا نوسانات شدید نشده است. رفتار همزمان کاهش خطای Generator و نوسان محدود Discriminatorها بیانگر یک فرآیند یادگیری پایدار است، هرچند این معیارها به‌تنهایی برای قضاوت درباره کیفیت نهایی نگاشت کافی نیستند و باید در کنار ارزیابی‌های کیفی تفسیر شوند.



*شکل ۳–۳۹: نمونه‌های ترجمه‌شده توسط **CycleGAN** برای دامنه ۱۵ درجه، شامل تصاویر منبع، نگاشت منبع به هدف، تصاویر هدف و نگاشت هدف به منبع*

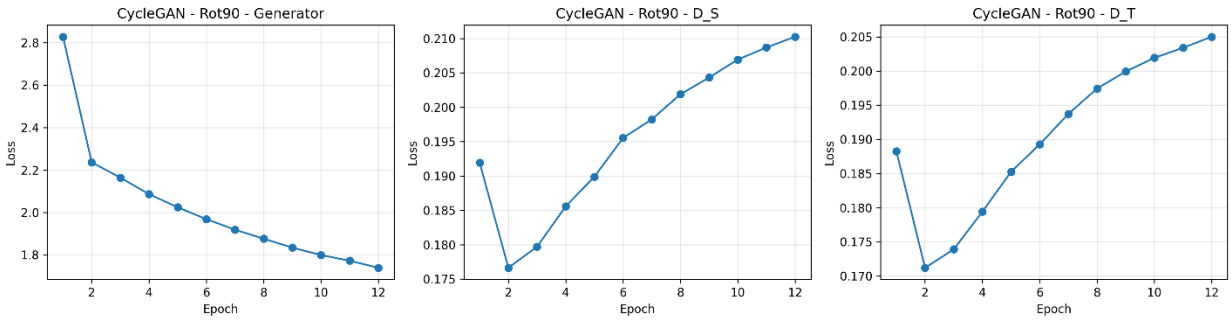
نمونه‌های تولیدشده در حالت ۱۵ درجه نشان می‌دهند که CycleGAN تا حد زیادی توانسته ساختار کلی ارقام را حفظ کند و نگاشت معناداری بین دو دامنه برقرار کند. در تبدیل منبع به هدف $(S \to T)$، تغییرات ظاهری ایجادشده عمدتاً در راستای چرخش ملایم هستند و شکل کلی ارقام مانند ۷، ۲، ۱، ۰ و ۴ به‌خوبی حفظ شده است. همچنین در تبدیل معکوس $(T \to S)$، تصاویر تقریباً به فرم اولیه بازمی‌گردند که نشان‌دهنده حفظ سازگاری چرخه‌ای[۲۷] است.

با این حال، با دقت بیشتر می‌توان مشاهده کرد که در برخی نمونه‌ها تغییرات جزئی اما قابل توجهی ایجاد شده است. برای مثال، در برخی ارقام مانند ۹، نویزهای کوچک یا اعوجاج‌های موضعی در اطراف خطوط دیده می‌شود. همچنین در برخی موارد، ضخامت خطوط نوشتاری اندکی تغییر کرده یا لبه‌ها نسبت به تصویر اصلی کمی ناصاف‌تر شده‌اند. این تغییرات اگرچه از نظر بصری شدید نیستند، اما نشان می‌دهند که نگاشت به‌طور کامل ایده‌آل نیست.

نکته مهم این است که این اعوجاج‌ها به‌صورت یکنواخت در همه نمونه‌ها رخ نمی‌دهند و برای برخی ارقام (به‌ویژه ارقامی با ساختار پیچیده‌تر مانند ۹) بیشتر قابل مشاهده‌اند. در مقابل، ارقامی با ساختار ساده‌تر مانند ۱ و ۰ تقریباً بدون تغییر قابل توجه منتقل شده‌اند. این موضوع نشان می‌دهد که پایداری نگاشت به ساختار هندسی رقم وابسته است.

[27] Cycle consistency

در مجموع، در چرخش ۱۵ درجه، CycleGAN توانسته ظاهر داده‌ها را به دامنه هدف نزدیک کند و ساختار کلی ارقام را حفظ نماید، اما همچنان تغییرات ظریف و نویزهای جزئی در برخی نمونه‌ها وجود دارد که می‌توانند بر کیفیت نهایی نمایش‌ها تأثیر بگذارند.

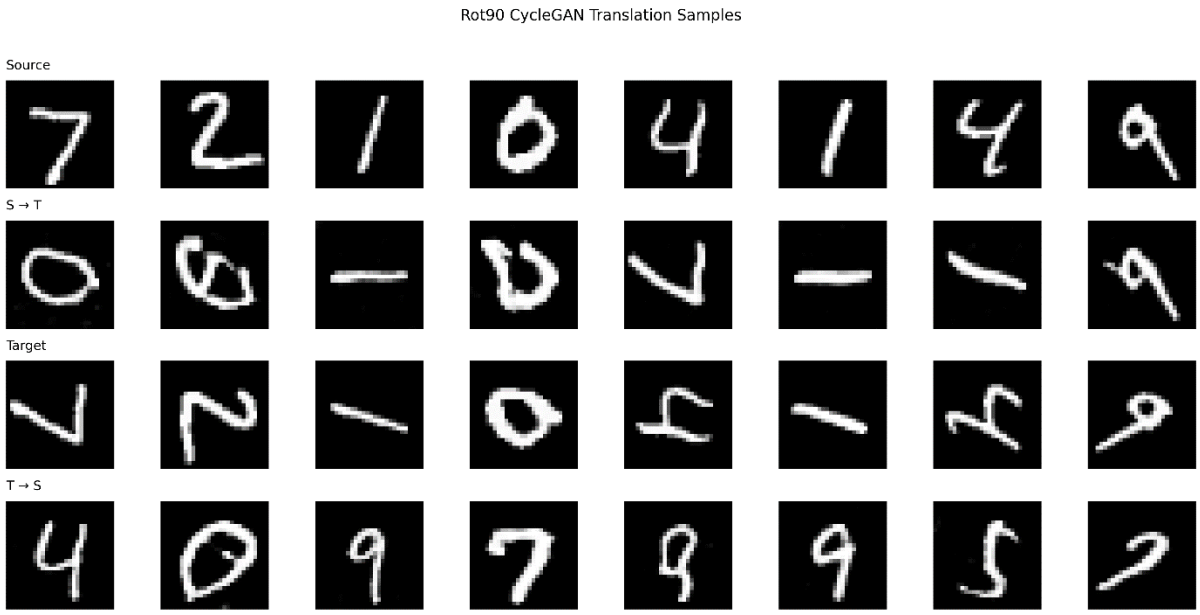


*شکل ۳–۴۰: روند تغییرات خطای **Generator** و دو **Discriminator** در **CycleGAN** برای دامنه ۹۰ درجه*

در حالت چرخش ۹۰ درجه، خطای Generator روندی کاهشی و نسبتاً یکنواخت دارد؛ به‌طوری‌که از مقدار نسبتاً بالایی (حدود ۲.۸) در اپیاک‌های ابتدایی شروع شده و به‌تدریج تا حدود ۱.۷ کاهش می‌یابد. این رفتار نشان‌دهنده آن است که Generator به‌مرور در حال یادگیری نگاشت بین دو دامنه است، هرچند مقدار نهایی loss همچنان نسبتاً بالا باقی می‌ماند که می‌تواند نشانه‌ای از دشواری این سناریو باشد.

در مقابل، خطای هر دو Discriminator پس از یک افت اولیه (در حدود اپیاک دوم)، روندی افزایشی و نسبتاً یکنواخت پیدا می‌کند. به‌طور مشخص، خطای D_S از حدود ۰.۱۷۶ به بیش از ۰.۲۱ و خطای D_T از حدود ۰.۱۷۱ به حدود ۰.۲۰۵ افزایش می‌یابد. این افزایش تدریجی نشان می‌دهد که Generator به‌مرور در فریب دادن Discriminatorها موفق‌تر شده و تمایز بین نمونه‌های واقعی و تولیدی برای آن‌ها دشوارتر شده است.

در مجموع، این الگوها نشان می‌دهند که فرآیند آموزش از نظر عددی پایدار است و بین Generator و Discriminatorها نوعی تعادل نسبی برقرار شده است، بدون آن‌که نشانه‌ای از واگرایی یا نوسانات شدید دیده شود. با این حال، همان‌طور که در مدل‌های GAN رایج است، کاهش یا افزایش loss به‌تنهایی معیار کافی برای ارزیابی کیفیت ترجمه نیست و برای قضاوت دقیق‌تر باید خروجی‌های تولیدشده نیز مورد بررسی قرار گیرند.



*شکل ۳–۴۱: نمونه‌های ترجمه‌شده توسط **CycleGAN** برای دامنه ۹۰ درجه، شامل تصاویر منبع، نگاشت منبع به هدف، تصاویر هدف و نگاشت هدف به منبع*

در نمونه‌های تولیدشده برای چرخش ۹۰ درجه، ضعف CycleGAN به‌وضوح قابل مشاهده است. در نگاشت منبع به هدف ($S \to T$)، اگرچه در برخی نمونه‌ها اثری از چرخش دیده می‌شود، اما در بسیاری از موارد ساختار رقم به‌شدت دچار تغییر شده و خروجی‌ها به اشکالی غیرطبیعی یا مبهم تبدیل شده‌اند. برای مثال، برخی ارقام به فرم‌های حلقه‌ای، خطوط افقی ساده یا اشکال ناقص تبدیل شده‌اند که شباهت دقیقی به رقم اولیه ندارند و در برخی موارد حتی قابل تشخیص نیستند.

این مشکل در نگاشت معکوس ($T \to S$) نیز مشهود است. تصاویر بازتولیدشده در بسیاری از موارد به رقم اولیه بازنمی‌گردند و تغییر هویت کلاس به‌وضوح دیده می‌شود؛ به‌عنوان مثال، برخی نمونه‌ها به ارقامی شبیه ۹، ۵، ۲ یا ۷ تبدیل شده‌اند، در حالی که با ورودی اولیه هم‌خوانی ندارند. این موضوع نشان می‌دهد که خاصیت cycle consistency در این سناریوی شدید به‌طور کامل حفظ نشده است.

در مجموع، برخلاف حالت ۱۵ درجه، در اینجا مسئله صرفاً اعمال یک تبدیل هندسی نیست، بلکه حفظ هویت کلاس به چالش اصلی تبدیل شده است. CycleGAN در مواجهه با این تغییر دامنه‌ی شدید نتوانسته نگاشتی پایدار و معنادار برقرار کند و در نتیجه، خروجی‌ها از نظر ساختاری و معنایی دچار اختلال قابل توجهی شده‌اند.

*جدول ۳–۵: عملکرد نهایی روش **Domain Translation** مبتنی بر **CycleGAN** در مقایسه با مدل پایه*

حالت	Clean Accuracy (Baseline Reference)	Target Accuracy after Translation
CycleGAN (Rot15)	۰.۹۸۶۲	۰.۹۵۷۷
CycleGAN (Rot90)	۰.۹۸۶۲	۰.۱۷۲۹

تحلیل عملکرد نهایی نشان می‌دهد که در این رویکرد، مقدار Clean Accuracy در واقع همان عملکرد مدل پایه است (۰.۹۸۶۲) و تغییری در آن ایجاد نشده، زیرا در روش Domain Translation مبتنی بر CycleGAN، مدل طبقه‌بند ثابت باقی می‌ماند و تنها داده‌های ورودی ترجمه می‌شوند. بنابراین، هرگونه تغییر در عملکرد صرفاً به کیفیت نگاشت یادگرفته‌شده توسط CycleGAN وابسته است.

در چرخش ۱۵ درجه، این روش توانسته بهبود محسوسی روی دامنه هدف ایجاد کند؛ به‌طوری‌که دقت از ۰.۹۴۸۱ به ۰.۹۵۷۷ افزایش یافته است. این نتیجه نشان می‌دهد که در تغییرات دامنه‌ی ملایم، CycleGAN قادر است توزیع داده‌ها را تا حد زیادی به دامنه مرجع نزدیک کند، در حالی که ساختار کلی و هویت کلاس‌ها نیز تا حد زیادی حفظ می‌شود. با این حال، فاصله با عملکرد ایده‌آل همچنان وجود دارد که می‌تواند ناشی از اعوجاج‌های ظریف در تصاویر ترجمه‌شده باشد.

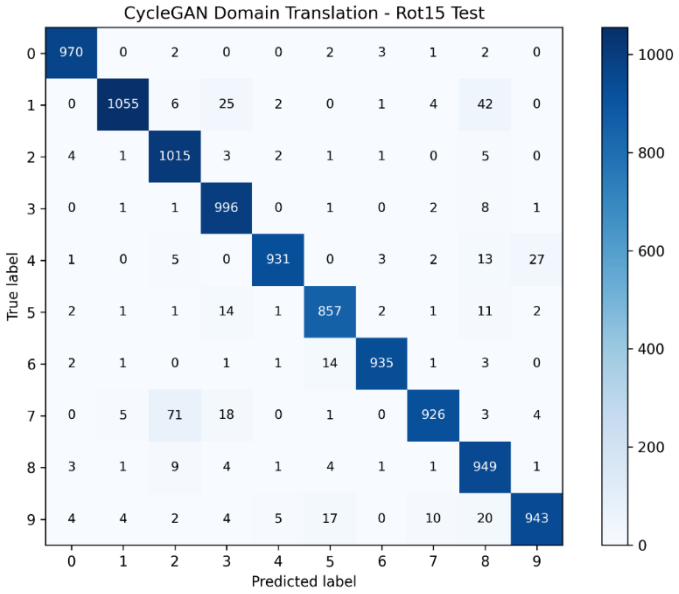
در مقابل، در چرخش ۹۰ درجه، اگرچه بهبود نسبت به مدل پایه مشاهده می‌شود (از ۰.۱۶۹۲ به ۰.۱۷۲۹)، اما این بهبود بسیار محدود است و عملکرد نهایی همچنان در سطح پایینی باقی می‌ماند. این موضوع نشان می‌دهد که در تغییرات دامنه‌ی شدید، CycleGAN نمی‌تواند نگاشتی بیاموزد که هم ظاهر داده را به دامنه مرجع نزدیک کند و هم هویت کلاس را به‌طور کامل حفظ نماید. در نتیجه، تصاویر ترجمه‌شده اگرچه تغییر ظاهری پیدا می‌کنند، اما الزاماً از دید مدل طبقه‌بند معادل نمونه‌های واقعی آن کلاس نیستند.

در عین حال، می‌توان انتظار داشت که با افزایش ظرفیت مدل، زمان آموزش بیشتر و تنظیم دقیق‌تر هایپرپارامترها، کیفیت نگاشت بهبود یابد و عملکرد بهتری حتی در شیفت‌های شدیدتر حاصل شود. با این حال، از منظر علمی، بعید است که این رویکرد به‌تنهایی بتواند به‌طور کامل شکاف ناشی از تغییرات دامنه‌ی بسیار شدید را از بین ببرد، زیرا محدودیت اصلی به حفظ همزمان ساختار بصری و هویت معنایی کلاس‌ها بازمی‌گردد، نه صرفاً ظرفیت مدل.

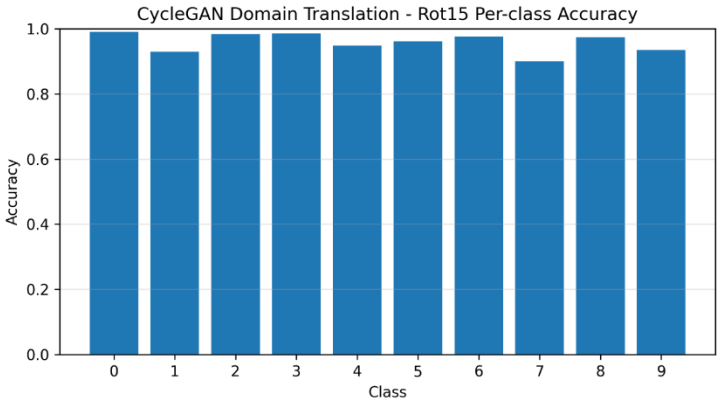
در مجموع، این نتایج نشان می‌دهند که Domain Translation در این تنظیم، در تغییرات دامنه‌ی ملایم عملکرد مؤثری دارد و می‌تواند به بهبود تعمیم کمک کند، اما در مواجهه با تغییرات شدید، حتی با وجود بهبود نسبی، همچنان در حفظ اطلاعات معنایی مورد نیاز برای طبقه‌بندی با محدودیت جدی مواجه است..

۳-۳-۶-۲- تحلیل ماتریس درهم‌ریختگی و دقت در هر کلاس

در این زیربخش، عملکرد روش Domain Translation مبتنی بر CycleGAN از دید جزئی‌تر و در سطح کلاس‌ها بررسی می‌شود. برای این منظور، ابتدا ماتریس درهم‌ریختگی و سپس نمودار دقت هر کلاس ارائه شده و در ادامه، تحلیل کلی رفتار مدل در هر سناریو مطرح می‌گردد. هدف اصلی این تحلیل، شناسایی کلاس‌هایی است که مدل در آن‌ها دچار بیشترین خطا شده و درک الگوی اشتباهات مدل پس از ترجمه دامنه است.



شکل ۳–۴۲: ماتریس درهم‌ریختگی مدل پس از ***Domain Translation*** *برای داده‌های چرخیده ۱۵ درجه*

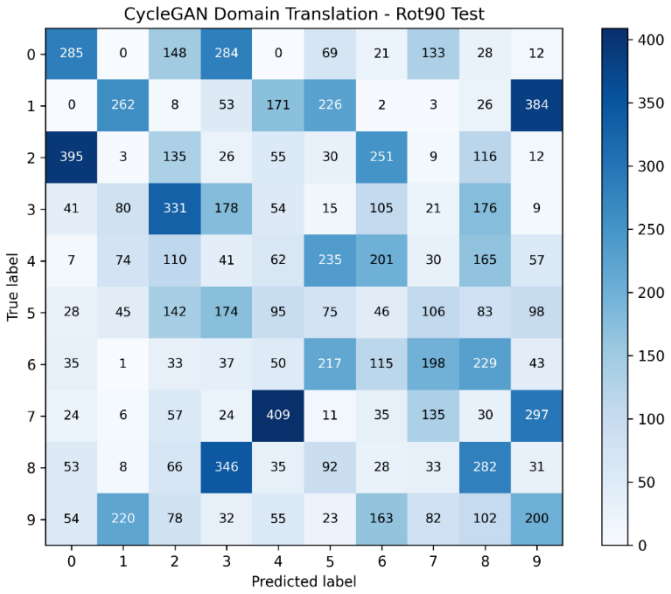


شکل ۳–۴۳: دقت در هر کلاس برای داده‌های چرخیده ۱۵ درجه پس از ***Domain Translation***

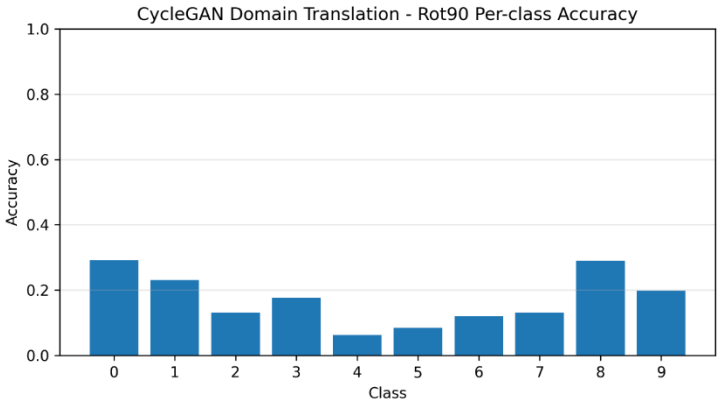
در حالت چرخش ۱۵ درجه، عملکرد کلی مدل پس از ترجمه‌ی دامنه همچنان قابل‌قبول است و ماتریس درهم‌ریختگی تا حد زیادی قطرمحور باقی مانده است. با این حال، برخی خطاهای مشخص دیده می‌شود. مهم‌ترین خطا مربوط به کلاس ۷ است که تعداد قابل توجهی از نمونه‌های آن به‌صورت ۲ پیش‌بینی شده‌اند. این موضوع در نمودار دقت کلاس‌به‌کلاس نیز دیده می‌شود، زیرا کلاس ۷ پایین‌ترین دقت را در میان کلاس‌ها دارد.

در کلاس ۱ نیز خطاهای قابل توجهی به سمت کلاس‌های ۸ و ۳ دیده می‌شود. کلاس ۴ بخشی از نمونه‌های خود را با ۹ و تا حدی ۸ اشتباه می‌گیرد، و در کلاس ۹ نیز خطاها بیشتر به سمت ۸، ۵ و ۷ پخش شده‌اند. همچنین کلاس ۵ تعدادی خطا به سمت ۳ دارد و کلاس ۶ نیز در برخی موارد با ۵ اشتباه گرفته شده است. این الگوها نشان می‌دهند که ترجمه‌ی دامنه در برخی ارقام باعث تشدید شباهت‌های هندسی، تغییر ضخامت خطوط یا اعوجاج جزئی در فرم رقم شده است.

در مقابل، کلاس‌هایی مانند ۰، ۲، ۳ و ۸ دقت بسیار بالایی دارند و پایداری بیشتری در برابر ترجمه نشان می‌دهند. در مجموع، خطاها عمدتاً در کلاس‌هایی دیده می‌شوند که پس از چرخش و ترجمه، از نظر ساختار نوشتاری به ارقام دیگر نزدیک‌تر می‌شوند؛ با این حال، شدت خطاها محدود است و عملکرد کلی مدل در دامنه‌ی ۱۵ درجه همچنان پایدار باقی مانده است.



*شکل ۳–۴۴: ماتریس درهم‌ریختگی مدل پس از **Domain Translation** برای داده‌های چرخیده ۹۰ درجه*



*شکل ۳–۴۵: دقت در هر کلاس برای داده‌های چرخیده ۹۰ درجه پس از **Domain Translation***

در حالت چرخش ۹۰ درجه، الگوی خطا بسیار گسترده و ساختاریافته است و ماتریس درهم‌ریختگی نشان می‌دهد که ترجمه‌ی دامنه نتوانسته هویت کلاس‌ها را به‌خوبی حفظ کند. در کلاس ۰، پیش‌بینی‌ها تقریباً بین خودِ کلاس ۰ و کلاس ۳ تقسیم شده‌اند و خطاهای قابل‌توجهی نیز به سمت کلاس‌های ۲ و ۷ دیده می‌شود. کلاس ۱ بیشتر به کلاس ۹ نگاشت شده و سپس با کلاس‌های ۵ و ۴ اشتباه گرفته می‌شود، در حالی که پیش‌بینی صحیح آن کمتر از خطای اصلی آن است.

در کلاس ۲، بیشترین خطا مربوط به کلاس ۰ است و پس از آن اشتباه با کلاس‌های ۶ و ۸ دیده می‌شود. در کلاس ۳، بیشترین پیش‌بینی به‌اشتباه مربوط به کلاس ۲ است و خطاهای قابل‌توجه دیگری به سمت کلاس‌های ۸، ۶ و ۱ وجود دارد؛ مقدار صحیح این کلاس نیز نسبتاً پایین است. کلاس ۴ یکی از ضعیف‌ترین عملکردها را دارد و بیشتر به کلاس‌های ۵، ۶، ۸ و ۲ نگاشت شده است. کلاس ۵ نیز توزیع خطای پراکنده‌ای دارد و بیشتر با کلاس‌های ۳، ۲، ۷، ۹ و ۴ اشتباه گرفته می‌شود.

در کلاس ۶، بیشترین پیش‌بینی مربوط به کلاس ۸ است و سپس خطاهای زیادی به سمت کلاس‌های ۵ و ۷ دیده می‌شود؛ بنابراین مقدار صحیح آن غالب نیست. کلاس ۷ عمدتاً به کلاس‌های ۴ و ۹ نگاشت شده و پیش‌بینی صحیح آن پایین است. در کلاس ۸، بیشترین پیش‌بینی به‌اشتباه مربوط به کلاس ۳ است، هرچند مقدار قابل‌توجهی نیز روی خود کلاس ۸ باقی مانده است. در نهایت، کلاس ۹ بیشترین خطا را با کلاس ۱ دارد و سپس با کلاس‌های ۶، ۸ و ۷ دچار ابهام می‌شود.

نمودار دقت کلاس‌به‌کلاس نیز همین وضعیت را تأیید می‌کند. کلاس‌های ۰ و ۸ بهترین عملکرد نسبی را دارند، اما دقت آن‌ها نیز حدود ۰.۳ است و فاصله زیادی با عملکرد مطلوب دارد. کلاس‌های ۱، ۹ و ۳ در سطح بعدی قرار می‌گیرند، در حالی که کلاس‌های ۴، ۵، ۶، ۷ و ۲ عملکرد بسیار ضعیفی نشان می‌دهند. در مجموع، مشکل اصلی این سناریو فقط افزایش تعداد خطاها نیست؛ بلکه ترجمه‌ی CycleGAN باعث شده هویت بسیاری از ارقام حفظ نشود و پیش‌بینی‌ها به‌صورت سیستماتیک میان چند کلاس مختلف پخش شوند.

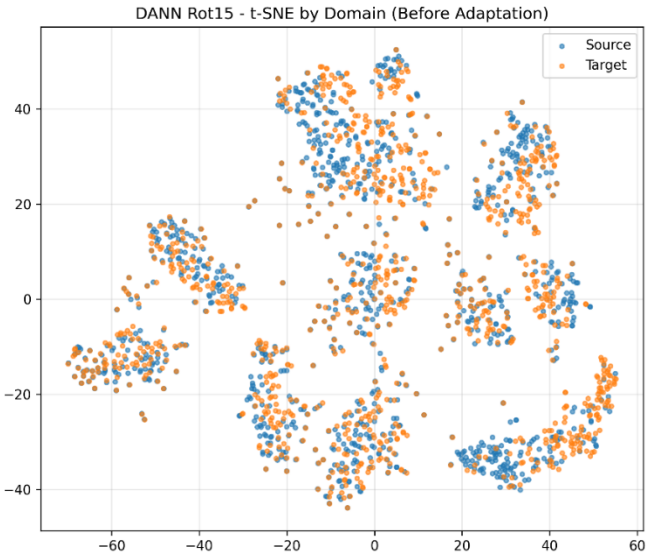
۳-۳-۷- تبیین نقش Domain Classifier Loss در هم‌ترازسازی ویژگی‌ها

در میان روش‌های مختلف تطبیق دامنه، تنها رویکرد مبتنی بر هم‌ترازسازی ویژگی‌ها است که مستقیماً بر نمایش‌های میانی[۲۸] اثر می‌گذارد. در روش Reweighting، domain classifier صرفاً ابزاری برای تخمین وزن نمونه‌ها بوده و در فرآیند استخراج ویژگی دخالتی ندارد. در مقابل، در روش Domain Translation نیز تطبیق در سطح داده انجام می‌شود و هیچ سازوکار مستقیمی برای اعمال قیود دامنه‌ای بر فضای ویژگی وجود ندارد. از این‌رو، تحلیل نقش domain classifier loss به‌طور خاص در چارچوب DANN معنا پیدا می‌کند.

در این روش، هدف اصلی یادگیری نمایش‌هایی است که در عین حفظ قدرت تفکیک‌پذیری برای کلاس‌ها، نسبت به دامنه (source/target) ناوردا باشند. این هدف از طریق یک یادگیری رقابتی بین encoder و domain classifier محقق می‌شود. به‌طور مشخص، encoder به‌گونه‌ای به‌روزرسانی می‌شود که domain classifier نتواند بین نمونه‌های دو دامنه تمایز قائل شود، در حالی که label classifier همچنان بر تفکیک کلاس‌ها تأکید دارد. در نتیجه، فضای ویژگی به سمت حالتی همگرا می‌شود که در آن توزیع‌های $p_s(f(x))$ و $p_t(f(x))$ به یکدیگر نزدیک شده و در عین حال ساختار کلاس‌ها حفظ می‌گردد.

برای درک اثر این سازوکار، بازنمایی‌های *t-SNE* قبل و بعد از اعمال تطبیق دامنه بررسی می‌شوند.

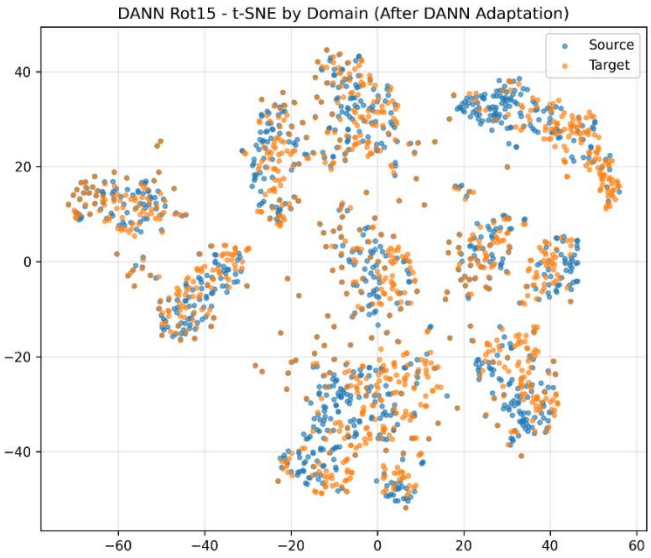
[28] Feature representations



شکل ۳–۴۶: بازنمایی ***t-SNE*** *ویژگی‌ها بر حسب دامنه (قبل از تطبیق) برای* ***Rot15***

در این نمودار t-SNE پیش از اعمال تطبیق دامنه، اگرچه برخی هم‌پوشانی‌ها بین نمونه‌های source و target دیده می‌شود، اما همچنان در بسیاری از نواحی، خوشه‌ها تمایل دارند بر اساس دامنه (رنگ) از یکدیگر جدا شوند. به‌عبارت دیگر، درون هر خوشه‌ی معنایی، نقاط source و target به‌طور کامل بر هم منطبق نیستند و نوعی جابه‌جایی یا شیفت میان آن‌ها مشاهده می‌شود.

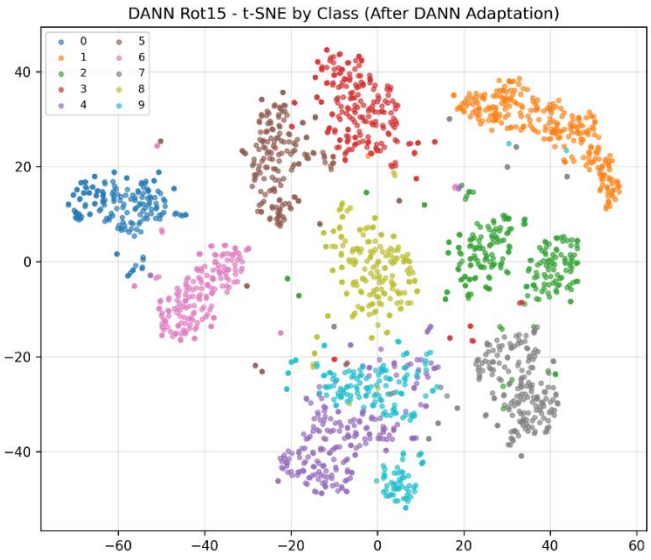
این الگو نشان می‌دهد که encoder ویژگی‌هایی را استخراج کرده که تا حدی وابسته به دامنه هستند؛ به همین دلیل، حتی زمانی که ساختار کلی کلاس‌ها حفظ شده، توزیع داده‌های target نسبت به source در فضای ویژگی جابه‌جا شده است. چنین ناهماهنگی‌ای باعث می‌شود که classifier که بر اساس توزیع source آموزش دیده، نتواند مرزهای تصمیم‌گیری مناسبی برای داده‌های target ایجاد کند و در نتیجه عملکرد روی دامنه هدف کاهش یابد.



شکل ۳–۴۷: بازنمایی ***t-SNE*** *ویژگی‌ها بر حسب دامنه (پس از تطبیق) برای* ***Rot15***

پس از اعمال DANN، هم‌پوشانی بین نمونه‌های source و target به‌طور محسوسی افزایش یافته و در اکثر خوشه‌ها نقاط دو دامنه تا حد زیادی در هم آمیخته‌اند. برخلاف حالت قبل، دیگر جدایی مشخصی بر اساس دامنه در بسیاری از نواحی دیده نمی‌شود و توزیع دو دامنه به یکدیگر نزدیک‌تر شده است.

این رفتار نشان می‌دهد که encoder تحت تأثیر domain classifier به سمت یادگیری نمایش‌هایی حرکت کرده که وابستگی کمتری به دامنه دارند. در نتیجه، ویژگی‌های استخراج‌شده بیشتر بر جنبه‌های معنایی (مرتبط با کلاس) متمرکز شده‌اند و مؤلفه‌های متمایزکننده‌ی دامنه تا حد زیادی تضعیف شده‌اند. چنین هم‌ترازی‌ای باعث می‌شود که classifier بتواند مرزهای تصمیم‌گیری مشابهی را برای هر دو دامنه به کار ببرد و در نتیجه، تعمیم‌پذیری مدل به دامنه هدف بهبود یابد.

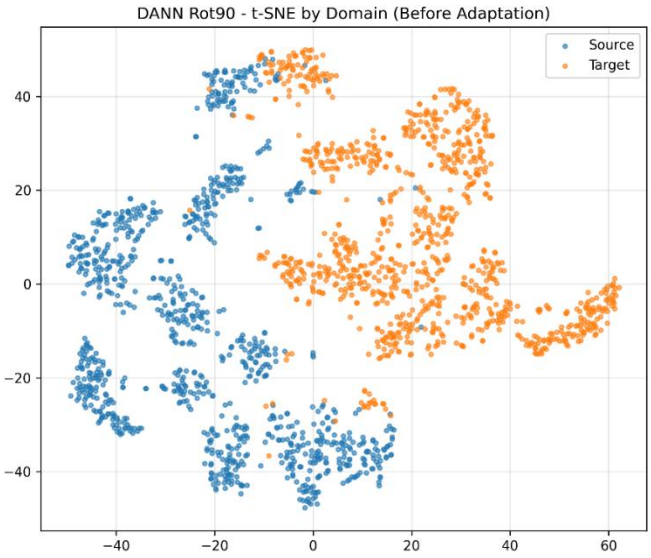


*شکل ۳–۴۸: بازنمایی **t-SNE** بر حسب کلاس (پس از تطبیق) برای **Rot15***

در این نمایش t-SNE، پس از اعمال DANN مشاهده می‌شود که ساختار خوشه‌بندی کلاس‌ها به‌خوبی شکل گرفته و نمونه‌های هر کلاس به‌صورت فشرده و متمایز در نواحی مشخصی از فضای ویژگی قرار گرفته‌اند. خوشه‌ها عمدتاً از یکدیگر جدا هستند و هم‌پوشانی بین کلاس‌های مختلف محدود است، که نشان‌دهنده حفظ قدرت تفکیک‌پذیری ویژگی‌هاست.

این الگو بیانگر آن است که فرآیند یادگیری به تعادل مناسبی میان دو هدف اصلی رسیده است: از یک سو، اطلاعات وابسته به دامنه کاهش یافته و نمونه‌ها صرف‌نظر از منبع خود در فضای مشترکی قرار گرفته‌اند، و از سوی دیگر، ساختار معنایی کلاس‌ها همچنان حفظ شده و حتی به‌صورت خوشه‌های منسجم‌تری سازمان‌دهی شده است. در نتیجه، ویژگی‌های استخراج‌شده هم دامنه‌ناوابسته هستند و هم توانایی بالایی در تفکیک کلاس‌ها دارند، که هدف اصلی روش DANN محسوب می‌شود.

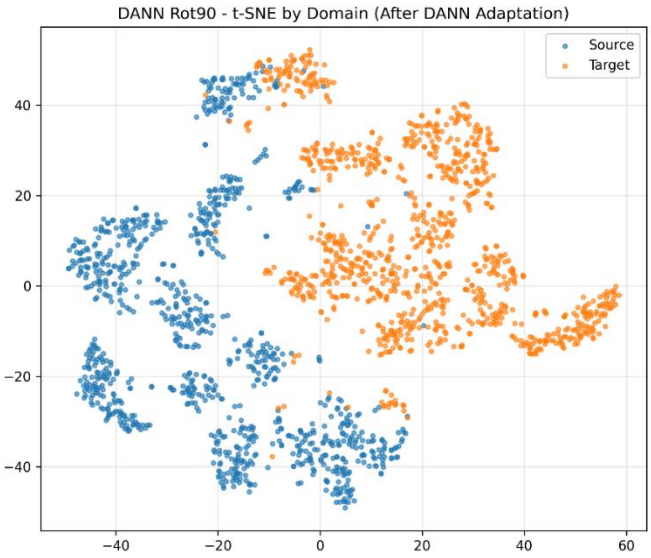
در سناریوی چرخش ۹۰ درجه، رفتار متفاوتی مشاهده می‌شود:



*شکل ۳–۴۹: بازنمایی **t-SNE** ویژگی‌ها بر حسب دامنه (قبل از تطبیق) برای **Rot90***

در این نمایش t-SNE پیش از اعمال تطبیق برای چرخش ۹۰ درجه، جدایی میان نمونه‌های source و target بسیار واضح و شدیدتر از حالت Rot15 است. نقاط مربوط به دو دامنه تقریباً در نواحی مجزا از فضای ویژگی قرار گرفته‌اند و هم‌پوشانی میان آن‌ها بسیار محدود است، به‌طوری که می‌توان مرز نسبتاً مشخصی بین دو توزیع مشاهده کرد.

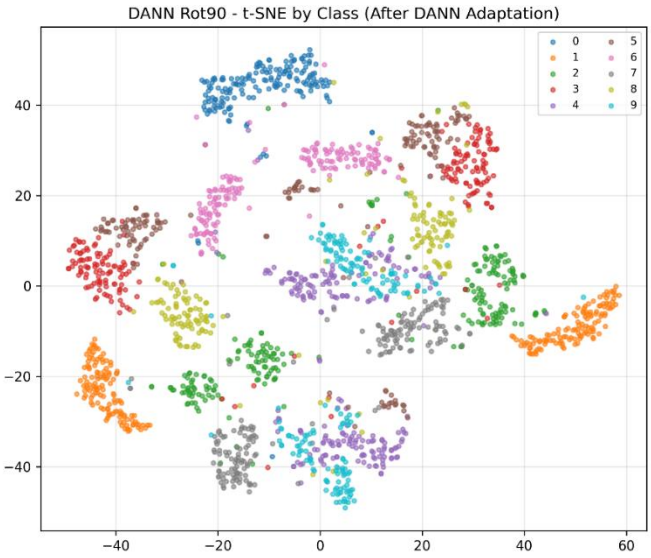
این الگو نشان می‌دهد که ویژگی‌های استخراج‌شده به‌شدت به جهت‌گیری تصویر وابسته هستند و تغییر ۹۰ درجه باعث ایجاد یک جابه‌جایی بزرگ در فضای ویژگی شده است. در نتیجه، encoder نمایش‌هایی تولید می‌کند که برای دو دامنه عملاً ناسازگار هستند، و همین اختلاف توزیع شدید باعث می‌شود classifier که بر روی داده‌های source آموزش دیده، در مواجهه با داده‌های target عملکرد بسیار ضعیفی داشته باشد.



*شکل ۳–۵۰: بازنمایی **t-SNE** ویژگی‌ها بر حسب دامنه (پس از تطبیق) برای **Rot90***

پس از اعمال DANN در حالت چرخش ۹۰ درجه، اگرچه فاصله میان دو دامنه تا حدی کاهش یافته، اما همچنان جدایی قابل‌توجهی بین نمونه‌های source و target در فضای ویژگی مشاهده می‌شود. برخلاف حالت Rot15، در اینجا هم‌پوشانی بین دو دامنه محدود است و بسیاری از خوشه‌ها همچنان به‌صورت مجزا بر اساس دامنه باقی مانده‌اند، به‌طوری که نواحی مشخصی عمدتاً توسط یکی از دو دامنه اشغال شده‌اند.

این الگو نشان می‌دهد که DANN تنها توانسته بخشی از اختلاف توزیع را جبران کند، اما در هم‌ترازسازی کامل دو دامنه موفق نبوده است. دلیل این موضوع را می‌توان در ماهیت تغییر دامنه دانست؛ چرخش ۹۰ درجه یک تغییر ساختاری عمیق ایجاد می‌کند که باعث می‌شود ویژگی‌های مفید برای تفکیک کلاس‌ها نیز به‌شدت تغییر کنند. در چنین شرایطی، تلاش برای حذف اطلاعات دامنه‌ای از طریق domain classifier می‌تواند با حفظ اطلاعات کلاس‌محور در تضاد قرار گیرد و در نتیجه، هم‌پوشانی کامل بین دو دامنه حاصل نشود.



*شکل ۳–۵۱: بازنمایی **t-SNE** بر حسب کلاس (پس از تطبیق) برای **Rot90***

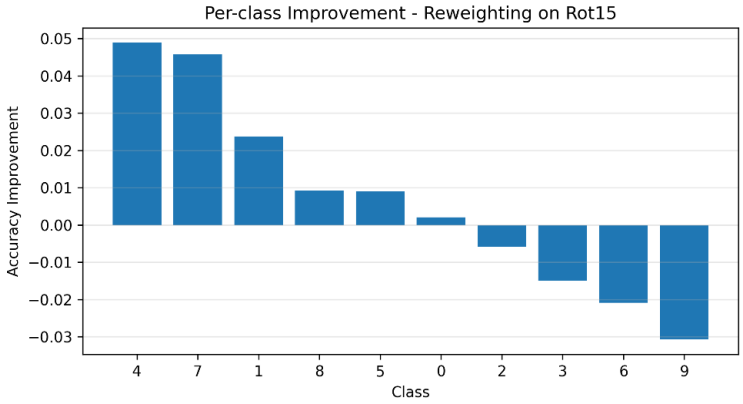
در این نمایش t-SNE بر اساس کلاس پس از اعمال DANN در حالت چرخش ۹۰ درجه، مشاهده می‌شود که اگرچه برخی خوشه‌های کلاسی همچنان قابل تشخیص هستند، اما میزان هم‌پوشانی بین کلاس‌ها نسبت به حالت Rot15 به‌طور محسوسی افزایش یافته است. چندین کلاس در نواحی مرکزی و حتی در برخی خوشه‌های پیرامونی با یکدیگر تداخل دارند و مرزهای بین آن‌ها شفاف نیست؛ به‌عبارت دیگر، ساختار خوشه‌بندی تضعیف شده و فشردگی و جدایی کلاس‌ها کاهش یافته است.

این الگو بیانگر یک trade-off میان domain invariance و class separability است. تلاش برای نزدیک‌سازی دو دامنه در شرایط یک شیفت شدید، باعث شده بخشی از اطلاعات متمایزکننده‌ی کلاس‌ها نیز تضعیف شود. در نتیجه، اگرچه encoder تا حدی ویژگی‌های دامنه‌ناوابسته استخراج کرده، اما این فرآیند به کاهش قدرت تفکیک کلاس‌ها منجر شده است.

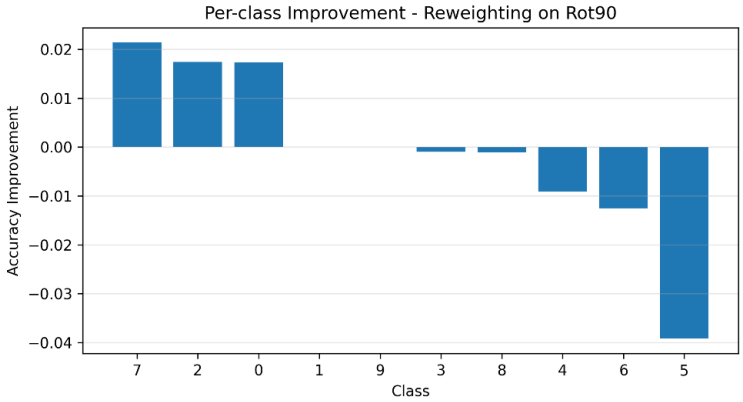
در مجموع، domain classifier loss از طریق یک یادگیری رقابتی، encoder را به سمت حذف مؤلفه‌های دامنه‌ای هدایت می‌کند. این سازوکار در شیفت‌های ملایم مؤثر است، اما در تغییرات شدید مانند چرخش ۹۰ درجه، ایجاد تعادل میان حذف اطلاعات دامنه‌ای و حفظ ساختار کلاسی دشوار می‌شود و در نهایت به هم‌پوشانی بیشتر کلاس‌ها و افت کیفیت خوشه‌بندی منجر می‌گردد.

۳-۳-۸- تحلیل الگوی خطا و بهبود کلاس‌ها پس از تطبیق دامنه

در این بخش، الگوی اشتباه‌گیری کلاس‌ها و میزان بهبود آن‌ها پس از اعمال روش‌های مختلف تطبیق دامنه بررسی می‌شود. تحلیل حالت پایه (بدون تطبیق) برای چرخش‌های 15° و 90° پیش‌تر در بخش ۳-۳-۲-۲- ارائه شد و در اینجا تمرکز بر چرایی تغییرات مشاهده‌شده در هر روش است.



شکل ۳–۵۲: بهبود دقت به‌ازای هر کلاس در روش ***Reweighting*** *برای چرخش* **15°**



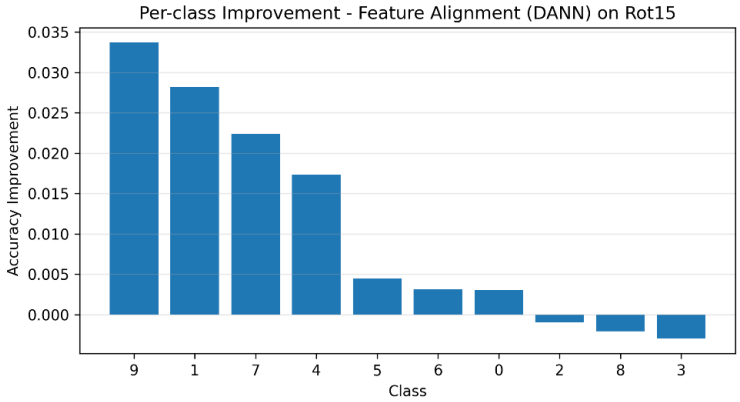
شکل ۳–۵۳: بهبود دقت به‌ازای هر کلاس در روش ***Reweighting*** *برای چرخش* **90°**

در چرخش 15°، بیشترین بهبود مربوط به کلاس‌های ۴ و ۷ است و پس از آن کلاس ۱ نیز بهبود قابل‌توجهی نشان می‌دهد، در حالی که کلاس‌های ۸ و ۵ بهبود ملایم‌تری دارند و کلاس ۰ تقریباً بدون تغییر باقی مانده است. در مقابل، افت عملکرد عمدتاً در کلاس‌های ۹ (بیشترین افت)، سپس ۶ و ۳ مشاهده می‌شود و کلاس ۲ نیز کاهش جزئی دارد. این الگو را می‌توان با ماهیت covariate shift ملایم توضیح داد؛ در این حالت، ساختار کلی ویژگی‌ها حفظ می‌شود و Reweighting با تقریب نسبت $\frac{p_{target}(x)}{p_{source}(x)}$ می‌تواند وزن نمونه‌های مرزی را اصلاح کرده و مرز تصمیم‌گیری را بهبود دهد. با این حال، در کلاس‌هایی که حتی با یک چرخش کوچک دچار هم‌پوشانی ساختاری با سایر کلاس‌ها می‌شوند (مانند ۹ یا ۶)، چون نمایش ویژگی‌ها تغییری نمی‌کند، این روش قادر به رفع ابهام نبوده و حتی با تأکید بیشتر بر نمونه‌های گمراه‌کننده، خطا را تقویت می‌کند.

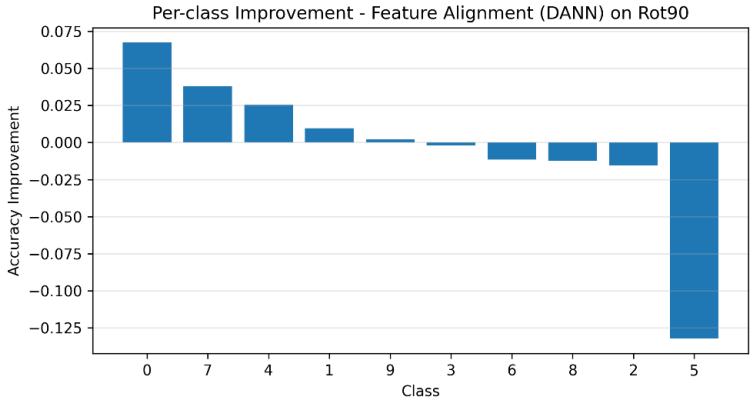
در چرخش 90°، بیشترین بهبود مربوط به کلاس ۷ و سپس کلاس‌های ۲ و ۰ است، در حالی که کلاس ۱ و ۹ تقریباً بدون تغییر باقی می‌مانند. در مقابل، افت عملکرد قابل‌توجهی در کلاس ۵ (بیشترین افت)، سپس ۶ و ۴ مشاهده می‌شود و کلاس‌های ۳ و ۸ تغییرات بسیار محدودی دارند. این رفتار ناشی از آن است که در یک شیفت ساختاری شدید، فرض اصلی Reweighting یعنی ثابت بودن $p(y|x)$ عملاً نقض می‌شود؛ زیرا چرخش ۹۰ درجه می‌تواند نمایش ویژگی‌ها را به‌گونه‌ای تغییر دهد که یک

نمونه در فضای جدید به نواحی مربوط به کلاس‌های دیگر منتقل شود. در این شرایط، وزن‌دهی مجدد تنها توزیع نمونه‌ها را تغییر می‌دهد، اما چون نمایش ویژگی‌ها اصلاح نشده، هم‌پوشانی بین کلاس‌ها باقی مانده و حتی می‌تواند تشدید شود. بنابراین تنها کلاس‌هایی که پس از چرخش همچنان الگوهای متمایزتری دارند (مانند ۰ یا تا حدی ۲ و ۷) از این روش سود می‌برند، در حالی که سایر کلاس‌ها دچار افت می‌شوند.

در مجموع، این نتایج نشان می‌دهند که Reweighting زمانی از نظر تئوری و عملی مؤثر است که اختلاف بین دامنه‌ها در حد یک covariate shift واقعی و ملایم باشد؛ اما در مواجهه با تغییرات شدید و ساختاری، که باعث تغییر در خود توزیع شرطی $p(y|x)$ می‌شوند، این روش به‌تنهایی کافی نیست و حتی می‌تواند به تقویت خطاهای موجود منجر شود.



شکل ۳–۵۴: بهبود دقت به‌ازای هر کلاس در روش ***Feature Alignment (DANN)*** *برای چرخش* **15°**



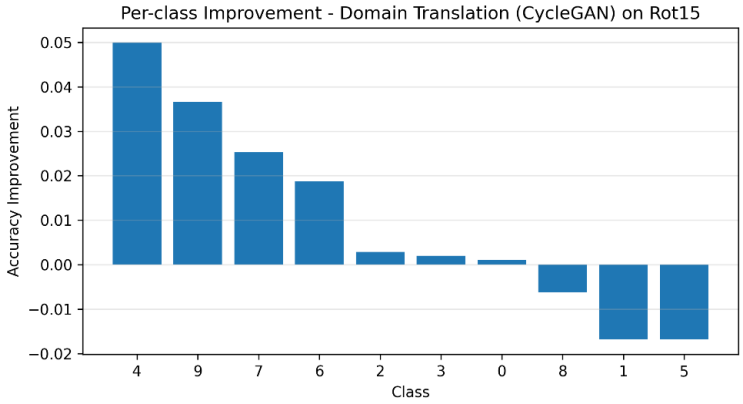
شکل ۳–۵۵: بهبود دقت به‌ازای هر کلاس در روش ***Feature Alignment (DANN)*** *برای چرخش* **90°**

در روش Feature Alignment (DANN)، در چرخش ۱۵ درجه بیشترین بهبود مربوط به کلاس ۹ است و پس از آن کلاس‌های ۱، ۷ و ۴ قرار می‌گیرند. کلاس‌های ۵، ۶ و ۰ بهبودهای بسیار جزئی دارند، در حالی که کلاس‌های ۲، ۸ و به‌ویژه ۳ دچار افت عملکرد شده‌اند. این الگو نشان می‌دهد که در یک شیفت ملایم، DANN می‌تواند با هم‌ترازسازی توزیع ویژگی‌ها، برای کلاس‌هایی که ساختار نسبتاً پایدار و متمایزی دارند، نمایش‌های مشترک مؤثری بیاموزد. در مقابل، برای کلاس‌هایی که مرز

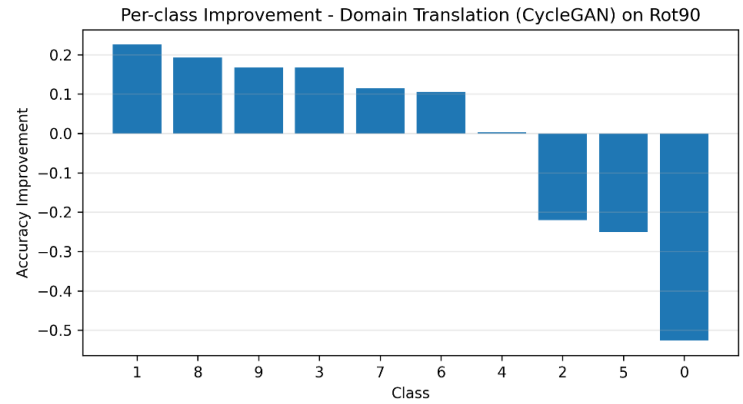
تصمیم‌گیری آن‌ها به‌صورت ذاتی به جزئیات ظریف یا جهت‌گیری حساس است (مانند ۳ یا ۸)، اعمال فشار برای domain invariance باعث کاهش class separability می‌شود؛ به این معنا که ویژگی‌های متمایزکننده تضعیف شده و در نتیجه دقت کاهش می‌یابد.

در چرخش ۹۰ درجه، این trade-off شدیدتر می‌شود. بیشترین بهبود مربوط به کلاس ۰ است و پس از آن کلاس‌های ۷، ۴ و ۱ قرار می‌گیرند، در حالی که کلاس‌های ۳ و ۹ تقریباً بدون تغییر می‌باشند. در مقابل، افت عملکرد در کلاس‌های ۶، ۸ و ۲ مشاهده می‌شود و کلاس ۵ با اختلاف زیاد بیشترین کاهش دقت را دارد. این رفتار از نظر علمی ناشی از آن است که در یک شیفت ساختاری شدید، نگاشت بین دامنه‌ها دیگر صرفاً یک جابه‌جایی در توزیع $p(x)$ نیست، بلکه نمایش‌های ویژگی برای برخی کلاس‌ها به‌طور اساسی تغییر می‌کنند. در چنین شرایطی، DANN با مجبور کردن encoder به یادگیری ویژگی‌های domain-invariant، ناگزیر بخشی از اطلاعات تمایزبخش کلاس‌ها را نیز حذف می‌کند، به‌ویژه برای کلاس‌هایی که پس از چرخش، شباهت زیادی با چند کلاس دیگر پیدا می‌کنند (مانند ۵).

در مجموع، این نمودارها نشان می‌دهند که Feature Alignment زمانی مؤثر است که بتوان یک نمایش مشترک و نسبتاً پایدار بین دامنه‌ها تعریف کرد. اما با افزایش شدت تغییر دامنه، تعادل بین domain invariance و حفظ اطلاعات کلاس‌محور به چالش کشیده می‌شود و در برخی کلاس‌ها، به‌جای بهبود، افت عملکرد رخ می‌دهد.



شکل ۳–۵۶: بهبود دقت به‌ازای هر کلاس در روش ***Domain Translation (CycleGAN)*** *برای چرخش* **15°**



شکل ۳–۵۷: بهبود دقت به‌ازای هر کلاس در روش ***Domain Translation (CycleGAN)*** *برای چرخش* **90°**

در روش Domain Translation (CycleGAN)، رفتار مدل به‌شدت به کیفیت نگاشت تصویری وابسته است و این موضوع در تحلیل per-class به‌خوبی قابل مشاهده است.

در چرخش 15°، بیشترین بهبود مربوط به کلاس ۴ است و پس از آن کلاس‌های ۹، ۷ و ۶ قرار می‌گیرند. کلاس‌های ۲، ۳ و ۰ بهبودهای بسیار جزئی دارند، در حالی که کلاس‌های ۸، ۱ و به‌ویژه ۵ دچار افت عملکرد شده‌اند. این الگو نشان می‌دهد که در یک شیفت ملایم، CycleGAN می‌تواند برای کلاس‌هایی با ساختار پایدارتر (مانند ۴ یا ۹) نگاشت نسبتاً دقیقی ایجاد کند، اما در کلاس‌هایی که به جزئیات ظریف وابسته‌اند (مانند ۵ یا ۱)، حتی اعوجاج‌های کوچک در فرآیند ترجمه می‌تواند ویژگی‌های تمایزبخش را تخریب کند. از آنجا که این روش مستقیماً روی فضای تصویر عمل می‌کند، هرگونه خطای بازسازی به‌طور مستقیم به فضای ویژگی منتقل شده و بر دقت طبقه‌بندی اثر می‌گذارد.

در چرخش 90°، الگو بسیار ناهمگن‌تر و شدیدتر می‌شود. بیشترین بهبود مربوط به کلاس ۱ است و پس از آن کلاس‌های ۸، ۹، ۳، ۷ و ۶ نیز بهبود قابل‌توجهی دارند، در حالی که کلاس ۴ تقریباً بدون تغییر باقی مانده است. در مقابل، افت شدید در کلاس ۰ (بیشترین افت) و سپس کلاس‌های ۵ و ۲ مشاهده می‌شود. این رفتار از نظر علمی ناشی از آن است که در یک شیفت ساختاری

شدید، نگاشت بین دو دامنه دیگر یک تبدیل ساده هندسی نیست و CycleGAN باید هم‌زمان هم ظاهر را تغییر دهد و هم هویت کلاس را حفظ کند. در بسیاری از موارد، این دو هدف در تضاد قرار می‌گیرند و مدل ممکن است برای واقعی‌تر کردن تصویر، ویژگی‌های کلیدی کلاس را تغییر دهد؛ به‌ویژه برای کلاس‌هایی مانند ۰ که ساختار آن‌ها نسبت به چرخش بسیار حساس است.

در مجموع، این نتایج نشان می‌دهند که Domain Translation به‌شدت به توانایی مدل در حفظ معنای واقعی هر نمونه وابسته است؛ به این معنا که تصویر ترجمه‌شده باید همچنان همان کلاس اولیه را نمایش دهد، حتی اگر ظاهر آن تغییر کند. به‌عبارت دیگر، تغییرات اعمال‌شده نباید ویژگی‌های کلیدی و تمایزبخش کلاس (مانند شکل کلی، اتصالات خطوط یا الگوی هندسی عدد) را تخریب کند. در تغییرات ملایم، این تعادل تا حد زیادی حفظ می‌شود و مدل می‌تواند بدون تغییر ماهیت نمونه‌ها، آن‌ها را به دامنه جدید نزدیک کند، در نتیجه بهبود عملکرد حاصل می‌شود. اما در شیفت‌های شدید، حفظ هم‌زمان ظاهر دامنه هدف و هویت کلاس دشوار می‌شود؛ در این حالت، مدل ممکن است برای واقعی‌تر کردن تصویر، بخش‌هایی از ساختار اصلی را تغییر دهد و همین موضوع باعث می‌شود برخی نمونه‌ها به کلاس‌های دیگر شبیه شوند. در نتیجه، عملکرد به‌صورت نابرابر بین کلاس‌ها توزیع شده و در برخی موارد حتی افت شدیدی مشاهده می‌شود.

به‌طور کلی، تحلیل نتایج نشان می‌دهد که الگوی بهبود یا افت عملکرد در سطح کلاس‌ها، نه‌تنها به ماهیت هندسی ارقام بلکه به نوع تغییر دامنه و نحوه‌ی تعامل آن با نمایش‌های یادگرفته‌شده وابسته است. در شیفت‌های ملایم (مانند چرخش ۱۵ درجه)، اغلب روش‌ها قادرند برای بخش قابل‌توجهی از کلاس‌ها بهبود ایجاد کنند، زیرا ساختار کلی ویژگی‌ها حفظ شده و فرض‌هایی مانند ثبات نسبی $p(y|x)$ تا حد زیادی برقرار می‌ماند. در این شرایط، روش‌هایی مانند Reweighting یا DANN می‌توانند با اصلاح توزیع یا هم‌ترازسازی ویژگی‌ها، مرزهای تصمیم‌گیری را بهبود دهند. با این حال، حتی در این حالت نیز کلاس‌هایی که به جزئیات ظریف یا جهت‌گیری حساس‌ترند (و در تحلیل‌ها مشاهده شد که دچار هم‌پوشانی بیشتری می‌شوند)، همچنان ممکن است دچار افت عملکرد شوند.

با افزایش شدت شیفت دامنه (چرخش ۹۰ درجه)، مسئله ماهیتی متفاوت پیدا می‌کند و در بسیاری از کلاس‌ها، نمایش‌های ویژگی به‌طور اساسی تغییر می‌کنند. در این حالت، روش Reweighting به‌دلیل عدم تغییر در فضای ویژگی عملاً ناکارآمد می‌شود، زیرا فرض covariate shift نقض می‌شود. از سوی دیگر، روش Feature Alignment (DANN) اگرچه می‌تواند فاصله بین دامنه‌ها را کاهش دهد، اما به‌دلیل وجود یک trade-off ذاتی میان domain invariance و class separability، در برخی کلاس‌ها منجر به تضعیف ویژگی‌های تمایزبخش می‌شود. در روش Domain Translation نیز مشاهده شد که موفقیت به‌شدت به توانایی مدل در حفظ هویت کلاس وابسته است؛ به‌طوری‌که در بسیاری از موارد، تغییرات تصویری باعث تخریب ویژگی‌های کلیدی شده و برخی کلاس‌ها حتی دچار افت شدید می‌شوند، در حالی که برخی دیگر بهبود قابل‌توجهی دارند.

در مجموع، این نتایج نشان می‌دهند که هیچ‌یک از روش‌های بررسی‌شده قادر به ارائه‌ی بهبود یکنواخت برای تمامی کلاس‌ها نیستند و عملکرد آن‌ها به‌شدت به ویژگی‌های ساختاری هر کلاس و نوع شیفت دامنه وابسته است. به‌عبارت دیگر، تطبیق دامنه یک مسئله ناهمگن در سطح کلاس‌هاست و موفقیت هر روش، به میزان سازگاری بین تغییر دامنه و فرض‌های ضمنی آن روش بستگی دارد.

۳-۴- نکات پیاده‌سازی و ملاحظات عملی

در این بخش، برخی از بخش‌های مهم پیاده‌سازی بررسی می‌شوند که درک آن‌ها برای تفسیر درست نتایج و ارتباط میان پیاده‌سازی و مبانی نظری مسئله ضروری است. تمرکز این بخش بر قطعاتی از کد است که یا در رفتار مدل‌ها نقش تعیین‌کننده دارند، یا بدون توضیح آن‌ها ممکن است منطق کلی آزمایش به‌درستی روشن نباشد.

۳-۴-۱- تفاوت پیش‌پردازش در مدل‌های طبقه‌بندی و مدل ترجمه دامنه

پیش از ورود به جزئیات آموزش، لازم بود داده‌ها متناسب با نوع مدل مورد استفاده پیش‌پردازش شوند. در این پیاده‌سازی، برای مدل‌های طبقه‌بندی شامل مدل پایه، Reweighting و DANN از یک نوع نرمال‌سازی استفاده شده و برای CycleGAN از نوع دیگری. این تفاوت صرفاً یک انتخاب سلیقه‌ای نیست، بلکه مستقیماً به ماهیت خروجی و نحوه‌ی آموزش این مدل‌ها مربوط است.

```python
mean, std = 0.1307, 0.3081

# For classification models: baseline / reweighting / DANN
transform_cls = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((mean,), (std,))
])

# For GAN-based domain translation
transform_gan = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

*شکل ۳–۵۸: قطعه‌کد مربوط به تعریف **transform**های مورد استفاده برای مدل‌های طبقه‌بندی و **CycleGAN***

در مدل‌های طبقه‌بندی، هدف اصلی آن است که ورودی‌ها در مقیاسی قرار گیرند که آموزش شبکه پایدارتر و همگرایی سریع‌تر شود. به همین دلیل، تصاویر MNIST با استفاده از میانگین و انحراف معیار استاندارد همین مجموعه‌داده نرمال‌سازی شده‌اند. در این حالت، مدل ورودی‌هایی دریافت می‌کند که از نظر آماری centered و scaled هستند و این موضوع معمولاً برای شبکه‌های دسته‌بندی‌کننده استاندارد مفید است.

اما در CycleGAN، شرایط متفاوت است. در اینجا شبکه‌ی مولد در لایه‌ی خروجی از تابع Tanh استفاده می‌کند و بنابراین خروجی آن ذاتاً در بازه‌ی $[-1,1]$ قرار می‌گیرد. به همین دلیل، ورودی‌های GAN نیز باید در همین بازه نرمال‌سازی شوند تا بین مقیاس داده‌ی واقعی و داده‌ی تولیدشده سازگاری وجود داشته باشد. اگر همان نرمال‌سازی مربوط به مدل‌های طبقه‌بندی برای GAN استفاده می‌شد، توزیع ورودی و خروجی مولد با یکدیگر هم‌خوانی نداشتند و آموزش مولد و متمایزکننده ناپایدارتر می‌شد.

بنابراین، تفاوت میان transform_cls و transform_gan از تفاوت در هدف مدل‌ها ناشی می‌شود: در مدل‌های طبقه‌بندی، نرمال‌سازی برای بهبود یادگیری ویژگی‌ها و پایداری آموزش انجام می‌شود، در حالی که در CycleGAN نرمال‌سازی باید با بازه‌ی خروجی مولد هم‌راستا باشد. این نکته از نظر عملی نیز مهم است، زیرا نشان می‌دهد که حتی وقتی همه‌ی مدل‌ها روی یک مجموعه‌داده آموزش می‌بینند، لزوماً نباید از یک پیش‌پردازش مشترک برای همه‌ی آن‌ها استفاده کرد.

۳-۴-۲- برآورد وزن نمونه‌ها در روش Reweighting

در روش Reweighting، ایده‌ی اصلی آن است که همه‌ی نمونه‌های دامنه‌ی مبدأ به یک اندازه در آموزش نقش نداشته باشند، بلکه هر نمونه متناسب با شباهتش به دامنه‌ی هدف وزن بگیرد. به این ترتیب، مدل در زمان آموزش بیشتر تحت تأثیر نمونه‌هایی قرار می‌گیرد که از نظر توزیعی به داده‌های هدف نزدیک‌ترند.

```python
@torch.no_grad()
def estimate_source_weights(domain_model, source_dataset, batch_size=256, clip_max=5.0):
    """
    weight(x) = p_t(x) / p_s(x) ~= (1 - c(source|x)) / c(source|x)
    """
    domain_model.eval()
    loader = DataLoader(source_dataset, batch_size=batch_size, shuffle=False)

    all_weights = []

    for x, _ in loader:
        x = x.to(device)
        logits = domain_model(x)
        c_source = torch.sigmoid(logits).clamp(1e-4, 1 - 1e-4)
        w = ((1.0 - c_source) / c_source).clamp(0.0, clip_max)
        all_weights.extend(w.cpu().numpy().tolist())
    return np.array(all_weights, dtype=np.float32)
```

شکل ۳–۵۹: قطعه‌کد مربوط به برآورد وزن نمونه‌های دامنه مبدأ با استفاده از ***domain classifier***

اگر $p_S(x)$ و $p_T(x)$ به‌ترتیب چگالی داده در دامنه‌ی مبدأ و هدف باشند، وزن ایده‌آل هر نمونه‌ی x به‌صورت زیر تعریف می‌شود:

$$w(x) = \frac{p_T(x)}{p_S(x)} \quad (۸)$$

این نسبت بیان می‌کند که هر نمونه از دامنه‌ی مبدأ تا چه اندازه نماینده‌ی داده‌های دامنه‌ی هدف است. چون این نسبت معمولاً به‌صورت مستقیم در دسترس نیست، از یک domain classifier برای تقریب آن استفاده می‌شود. اگر $c(x)$ احتمال این باشد که نمونه‌ی x متعلق به دامنه‌ی مبدأ است، می‌توان نشان داد که این وزن را می‌توان به‌صورت تقریبی از رابطه‌ی زیر به‌دست آورد:

$$w(x) \approx \frac{1 - c(x)}{c(x)} \quad (۹)$$

بنابراین، هرچه یک نمونه از دید domain classifier کمتر شبیه داده‌های مبدأ و بیشتر شبیه داده‌های هدف باشد، وزن بزرگ‌تری دریافت می‌کند.

در کد، ابتدا مدل domain classifier در حالت ارزیابی قرار می‌گیرد و کل داده‌های دامنه‌ی مبدأ به‌صورت batchی پردازش می‌شوند. خروجی مدل به‌صورت logits محاسبه می‌شود و سپس با اعمال تابع sigmoid، مقدار

$$c(x) = \sigma(logit) \quad (۱۰)$$

به‌دست می‌آید که تفسیر احتمالی دارد. از این مقدار، وزن هر نمونه طبق رابطه‌ی بالا محاسبه می‌شود:

$$w(x) = \frac{1 - c(x)}{c(x)} \quad (۱۱)$$

در پیاده‌سازی، دو نکته‌ی عملی مهم نیز در نظر گرفته شده است. نخست آن‌که مقدار $c(x)$ در بازه‌ای محدود نگه داشته می‌شود تا از بروز ناپایداری عددی جلوگیری شود؛ زیرا اگر $c(x)$ به صفر بسیار نزدیک شود، وزن به‌صورت غیرواقعی بسیار بزرگ خواهد شد. دوم آن‌که خود وزن نهایی نیز با یک حد بالایی clip_max بریده می‌شود تا تعداد کمی نمونه با وزن بسیار بزرگ، کل فرآیند آموزش را تحت سلطه قرار ندهند. این محدودسازی از نظر عملی اهمیت زیادی دارد، زیرا در غیر این صورت آموزش مدل ممکن است ناپایدار یا بیش از حد حساس به چند نمونه‌ی خاص شود.

در نهایت، وزن‌های محاسبه‌شده برای همه‌ی نمونه‌های دامنه‌ی مبدأ در یک آرایه جمع‌آوری می‌شوند تا در مرحله‌ی بعد، هنگام محاسبه‌ی loss طبقه‌بندی، هر نمونه با وزن متناظر خود در یادگیری مدل اثر بگذارد. به این ترتیب، این تابع حلقه‌ی اتصال میان ایده‌ی نظری Importance Sampling و پیاده‌سازی عملی آن در روش Reweighting را تشکیل می‌دهد.

۳-۴-۳- لایه معکوس‌سازی گرادیان در روش DANN

در روش DANN، ایده‌ی اصلی آن است که انکدر در عین یادگیری ویژگی‌های مفید برای طبقه‌بندی، نمایش‌هایی تولید کند که از نظر دامنه نیز تا حد ممکن غیرقابل تفکیک باشند. تحقق هم‌زمان این دو هدف نیازمند سازوکاری است که سیگنال حاصل از

طبقه‌بند دامنه را به‌صورت معکوس به انکدر منتقل کند. دو قطعه‌کد زیر دقیقاً برای همین منظور به کار می‌روند و به همین دلیل باید در کنار یکدیگر تفسیر شوند.

```python
class GradientReversalFunction(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lambd):
        ctx.lambd = lambd
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output):
        return -ctx.lambd * grad_output, None
```

شکل ۳–۶۰: پیاده‌سازی کلاس ***GradientReversalFunction*** *برای معکوس‌سازی گرادیان در مسیر بازانتشار*

```python
class GRL(nn.Module):
    def __init__(self, lambd=1.0):
        super().__init__()
        self.lambd = lambd

    def forward(self, x):
        return GradientReversalFunction.apply(x, self.lambd)
```

شکل ۳–۶۱: تعریف ماژول ***GRL*** *به‌عنوان یک لایه‌ی قابل استفاده در معماری* ***DANN***

در قطعه‌کد اول، یک تابع سفارشی برای بازتعریف رفتار گرادیان در مرحله‌ی backward نوشته شده است. رفتار این تابع در مرحله‌ی forward کاملاً خنثی است؛ یعنی ورودی x را بدون تغییر عبور می‌دهد:

$$GRL(x) = x \tag{۱۲}$$

در نتیجه، از دید مسیر پیشرو، این لایه هیچ تغییری در نمایش ویژگی‌ها ایجاد نمی‌کند. اما نکته‌ی اصلی در مرحله‌ی backward ظاهر می‌شود، جایی که گرادیان دریافتی به‌صورت زیر بازگردانده می‌شود:

$$\frac{\partial \mathcal{L}}{\partial x} = -\frac{\lambda(\partial \mathcal{L})}{\partial y} \tag{۱۳}$$

علامت منفی نشان می‌دهد که جهت گرادیان معکوس شده و ضریب λ شدت این اثر را کنترل می‌کند.

در قطعه‌کد دوم، همین مکانیزم در قالب یک ماژول استاندارد PyTorch بسته‌بندی شده است تا بتوان آن را مانند یک لایه‌ی معمولی در مدل استفاده کرد. به این ترتیب، در معماری DANN خروجی انکدر ابتدا از این لایه عبور می‌کند و سپس به طبقه‌بند

دامنه داده می‌شود. از آنجا که در مسیر forward چیزی تغییر نمی‌کند، طبقه‌بند دامنه همان ویژگی‌های استخراج‌شده را دریافت می‌کند. اما هنگام بازانتشار، گرادیان حاصل از domain loss با علامت معکوس به انکدر برمی‌گردد.

اثر این کار از نظر مفهومی بسیار مهم است. طبقه‌بند دامنه تلاش می‌کند از روی ویژگی‌ها تشخیص دهد که نمونه متعلق به دامنه‌ی مبدأ است یا هدف؛ بنابراین پارامترهای خودِ این طبقه‌بند در جهت کاهش domain loss به‌روزرسانی می‌شوند. در مقابل، انکدر به‌واسطه‌ی GRL گرادیانی در جهت مخالف دریافت می‌کند؛ یعنی به جای آن‌که ویژگی‌هایی تولید کند که تشخیص دامنه را آسان‌تر کنند، به سمت تولید ویژگی‌هایی حرکت می‌کند که تشخیص دامنه را دشوارتر سازند. به بیان دیگر، GRL همان سازوکاری است که یک هدف adversarial را بدون نیاز به تعریف دو مرحله‌ی بهینه‌سازی مجزا، وارد آموزش انکدر می‌کند.

نکته‌ی مهم دیگر آن است که این لایه به‌تنهایی چیزی را «یاد» نمی‌گیرد؛ بلکه فقط نحوه‌ی انتقال سیگنال گرادیان را تغییر می‌دهد. بنابراین، نقش آن نه در استخراج ویژگی، بلکه در جهت‌دهی به فرایند یادگیری ویژگی‌ها است. همین ویژگی باعث می‌شود که انکدر در DANN به‌جای یادگیری نمایش‌های وابسته به دامنه، به سمت ویژگی‌هایی حرکت کند که برای طبقه‌بندی کلاس مناسب‌اند اما اطلاعات دامنه را تا حد امکان پنهان می‌کنند.

۳-۴-۴- طراحی Discriminator به‌صورت PatchGAN در CycleGAN

در معماری CycleGAN، نقش Discriminator این است که تشخیص دهد یک تصویر واقعی است یا توسط Generator تولید شده است. در این پیاده‌سازی، به‌جای استفاده از یک Discriminator کلاسیک که برای کل تصویر یک تصمیم می‌گیرد، از ساختاری به نام PatchGAN استفاده شده است.

```python
class PatchDiscriminator(nn.Module):
    """
    PatchGAN-like discriminator.
    """
    def __init__(self, in_ch=1, base_ch=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, base_ch, 4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(base_ch, base_ch * 2, 4, stride=2, padding=1),
            nn.InstanceNorm2d(base_ch * 2),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(base_ch * 2, base_ch * 4, 3, stride=1, padding=1),
            nn.InstanceNorm2d(base_ch * 4),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(base_ch * 4, 1, 3, stride=1, padding=1)
        )

    def forward(self, x):
        return self.net(x)
```

شکل ۳–۶۲: ساختار ***PatchGAN Discriminator*** *مورد استفاده در* ***CycleGAN***

در این ساختار، تصویر ورودی از چند لایه‌ی کانولوشن عبور می‌کند و در نهایت به یک خروجی با ابعاد فضایی (و نه یک اسکالر) تبدیل می‌شود. به این معنی که Discriminator به‌جای اینکه بگوید «کل تصویر واقعی است یا جعلی»، برای هر ناحیه‌ی کوچک[۲۹] از تصویر یک تصمیم جداگانه می‌گیرد. در نتیجه، خروجی نهایی یک نقشه[۳۰] است که هر مقدار آن نشان‌دهنده‌ی واقعی یا جعلی بودن یک بخش از تصویر است.

این طراحی چند مزیت مهم دارد. اول اینکه Discriminator به‌جای تمرکز بر ساختار کلی تصویر، روی جزئیات محلی مانند لبه‌ها، ضخامت خطوط و بافت تمرکز می‌کند. این موضوع در مسئله‌ای مانند MNIST اهمیت زیادی دارد، زیرا تمایز بین ارقام تا حد زیادی به همین ویژگی‌های محلی وابسته است. دوم اینکه این رویکرد باعث می‌شود Generator مجبور شود در تمام نقاط تصویر کیفیت مناسبی ارائه دهد، نه فقط در سطح کلی.

در کد، این رفتار از طریق چند لایه‌ی کانولوشن با stride بزرگ‌تر از ۱ پیاده‌سازی شده که به‌تدریج ابعاد تصویر را کاهش می‌دهند و هم‌زمان تعداد کانال‌ها را افزایش می‌دهند. استفاده از LeakyReLU به پایداری آموزش کمک می‌کند و InstanceNorm نیز باعث می‌شود نرمال‌سازی به‌صورت مستقل برای هر نمونه انجام شود، که در مدل‌های مولد معمولاً انتخاب مناسب‌تری نسبت به BatchNorm است.

[29] Patch
[30] Feature map

در نهایت، لایه‌ی خروجی یک نگاشت تک‌کاناله تولید می‌کند که هر مقدار آن بیانگر قضاوت Discriminator درباره‌ی یک patch از تصویر است. این نوع طراحی باعث می‌شود Discriminator حساسیت بالایی به خطاهای موضعی داشته باشد و در نتیجه Generator را وادار کند که ساختارهای محلی تصویر را با دقت بیشتری بازسازی کند.

۳-۵- نتیجه‌گیری

در این فصل، عملکرد مدل پایه و سه رویکرد اصلی تطبیق دامنه شامل Reweighting، هم‌ترازسازی ویژگی‌ها (DANN) و تبدیل دامنه (CycleGAN) در دو سناریوی با شدت متفاوت از تغییر دامنه (چرخش ۱۵ و ۹۰ درجه) بررسی شد. نتایج به‌روشنی نشان می‌دهند که انتخاب روش مناسب به میزان اختلاف بین دو دامنه و نوع این اختلاف وابسته است و نمی‌توان یک راه‌حل واحد را برای همه شرایط پیشنهاد داد.

جدول ۳–۶: مقایسه نهایی عملکرد روش‌های مختلف در دامنه‌های مختلف

Method	Evaluation Domain	Accuracy
No Adaptation	Clean	۰.۹۸۶۲
No Adaptation	Rot15	۰.۹۴۸۱
No Adaptation	Rot90	۰.۱۶۹۲
Supervised Target (Upper Bound)	Clean (trained on Rot15 labeled)	۰.۹۷۳۷
Supervised Target (Upper Bound)	Rot15 (trained on Rot15 labeled)	۰.۹۸۶۳
Supervised Target (Upper Bound)	Clean (trained on Rot90 labeled)	۰.۳۳۲۶
Supervised Target (Upper Bound)	Rot90 (trained on Rot90 labeled)	۰.۹۸۵۳
Reweighting	Clean (model adapted to 15)	۰.۹۸۵۶
Reweighting	Rot15 (model adapted to 15)	۰.۹۵۵۰
Reweighting	Clean (model adapted to 90)	۰.۹۸۴۹
Reweighting	Rot90 (model adapted to 90)	۰.۱۶۹۱
Feature Alignment (DANN)	Clean (model adapted to 15)	۰.۹۸۸۸
Feature Alignment (DANN)	Rot15 (model adapted to 15)	۰.۹۵۹۱
Feature Alignment (DANN)	Clean (model adapted to 90)	۰.۹۸۸۷
Feature Alignment (DANN)	Rot90 (model adapted to 90)	۰.۱۶۷۶
Domain Translation (CycleGAN)	Clean	۰.۹۸۶۲
Domain Translation (CycleGAN)	Rot15	۰.۹۵۷۷
Domain Translation (CycleGAN)	Rot90	۰.۱۷۲۹

در سناریوی تغییر دامنه‌ی ملایم (Rot15)، مدل پایه خود عملکرد نسبتاً بالایی دارد (۰.۹۴۸۱)، که نشان‌دهنده‌ی هم‌پوشانی قابل‌توجه بین توزیع‌های مبدأ و هدف است. در این شرایط، روش‌های مختلف تنها بهبود محدودی ایجاد می‌کنند، اما در میان آن‌ها Feature Alignment (DANN) بهترین عملکرد را نشان داده (۰.۹۵۹۱). از نظر مفهومی، این نتیجه قابل انتظار است؛ زیرا در شیفت‌های ملایم، می‌توان فرض کرد که ساختار شرطی $p(y|x)$ تقریباً ثابت مانده و تنها توزیع ویژگی‌ها تغییر کرده است. بنابراین، هم‌ترازسازی فضای ویژگی‌ها بدون از دست رفتن اطلاعات تمایزبخش امکان‌پذیر است. به همین دلیل، در چنین شرایطی روش‌های مبتنی بر هم‌ترازسازی ویژگی‌ها انتخاب مناسب‌تری هستند؛ چرا که با هزینه‌ی محاسباتی نسبتاً پایین، می‌توانند نمایش‌های مشترک و پایدار بین دو دامنه ایجاد کنند، بدون اینکه به ساختار کلاس‌ها آسیب بزنند.

در مقابل، در سناریوی تغییر دامنه‌ی شدید (Rot90)، مدل پایه عملکرد بسیار ضعیفی دارد (۰.۱۶۹۲)، که نشان‌دهنده‌ی شکاف عمیق بین دو دامنه است. در این حالت، Reweighting (۰.۱۶۹۱) عملاً هیچ تأثیری ندارد، زیرا این روش تنها اختلاف در $p(x)$ را اصلاح می‌کند و قادر به جبران تغییرات ساختاری در داده نیست. روش DANN نیز (۰.۱۶۷۶) بهبودی ایجاد نمی‌کند و حتی اندکی افت دارد. دلیل این موضوع به محدودیت ذاتی هم‌ترازسازی سراسری ویژگی‌ها برمی‌گردد: وقتی ساختار داده‌ها به‌طور اساسی تغییر می‌کند، اعمال فشار برای یکسان‌سازی توزیع‌ها می‌تواند باعث هم‌پوشانی نامطلوب بین کلاس‌ها و کاهش تمایزپذیری شود.

در این میان، Domain Translation (CycleGAN) با رسیدن به ۰.۱۷۲۹ بهترین عملکرد نسبی را در این سناریو دارد. این نتیجه نشان می‌دهد که در شیفت‌های شدید، رویکردی که در سطح داده عمل می‌کند و تلاش می‌کند نمونه‌های دامنه هدف را به دامنه مبدأ تبدیل کند، نسبت به روش‌های مبتنی بر ویژگی انتخاب مناسب‌تری است. با این حال، این برتری نسبی است و فاصله‌ی زیادی با حالت ایده‌آل (Upper Bound ≈ 0.9853) دارد. دلیل اصلی این محدودیت آن است که CycleGAN الزاماً نمی‌تواند نگاشتی بیاموزد که هم‌زمان ظاهر تصویر را اصلاح کند و اطلاعات معنایی مرتبط با کلاس (semantic consistency) را نیز به‌طور کامل حفظ نماید؛ در نتیجه، برخی نمونه‌ها پس از ترجمه، از نظر طبقه‌بند معادل نمونه‌های واقعی آن کلاس نیستند.

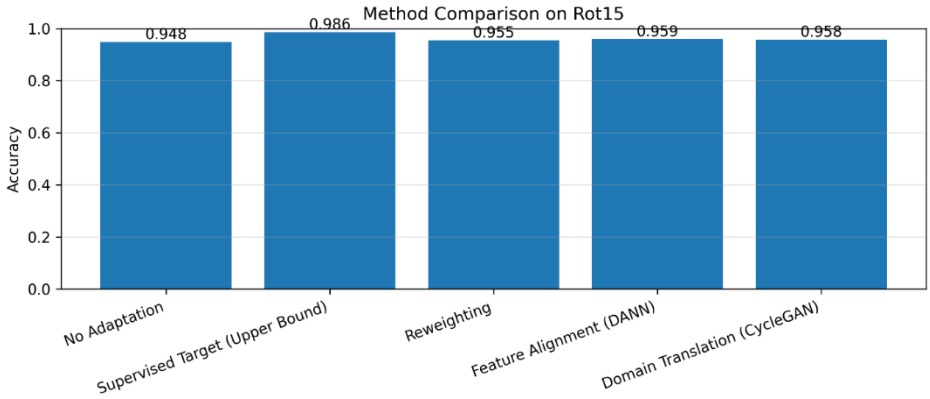
نکته‌ی مهم دیگر در مورد Domain Translation، هزینه محاسباتی و پیچیدگی آموزش آن است. برخلاف Reweighting یا DANN که نسبتاً ساده‌تر و پایدارتر هستند، آموزش CycleGAN شامل یک بازی adversarial بین generator و discriminator است که:

- به تنظیم دقیق hyperparameterها نیاز دارد،
- ممکن است دچار ناپایداری (mode collapse یا نوسان) شود،
- و معمولاً به زمان و منابع محاسباتی بیشتری نیاز دارد.

در نتیجه، حتی اگر این روش در شیفت‌های شدید عملکرد بهتری داشته باشد، استفاده از آن همیشه مقرون‌به‌صرفه نیست و باید بین کیفیت تطبیق و هزینه‌ی محاسباتی یک trade-off در نظر گرفته شود. همچنین افزایش ظرفیت مدل یا زمان آموزش می‌تواند کیفیت ترجمه را بهبود دهد، اما به‌طور کلی انتظار نمی‌رود که این رویکرد به‌تنهایی بتواند شکاف‌های بسیار شدید دامنه را به‌طور کامل از بین ببرد.

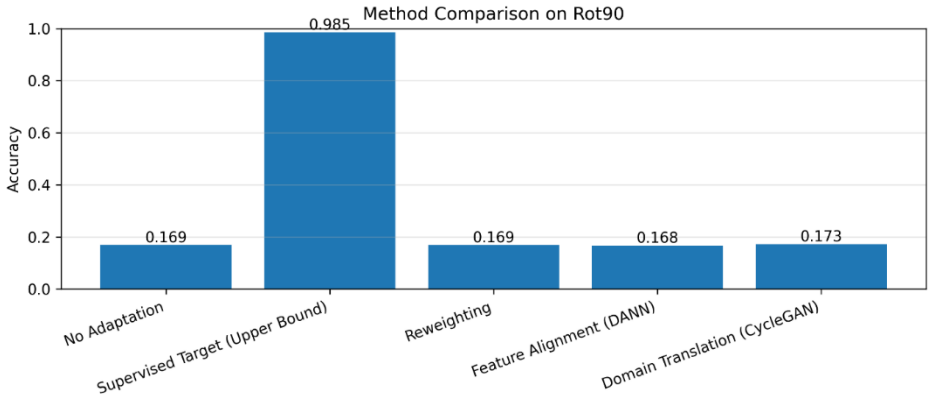
در جمع‌بندی، نتایج این فصل نشان می‌دهند که:

- در شیفت‌های ملایم: روش‌های مبتنی بر هم‌ترازسازی ویژگی‌ها مانند DANN انتخاب مناسب‌تری هستند، زیرا می‌توانند با حفظ ساختار کلاس‌ها، توزیع‌ها را به‌خوبی هم‌تراز کنند.
- در شیفت‌های شدید: روش‌های مبتنی بر تبدیل دامنه مانند CycleGAN عملکرد بهتری دارند، زیرا مستقیماً در سطح داده عمل می‌کنند؛ هرچند با هزینه محاسباتی بالاتر و محدودیت در حفظ کامل اطلاعات معنایی.
- هیچ‌یک از این روش‌ها به‌تنهایی قادر به حل کامل مسئله در شیفت‌های شدید نیستند، و موفقیت آن‌ها به میزان زیادی به ماهیت تغییر دامنه و ساختار داده‌ها وابسته است..



شکل ۳–۶۳: مقایسه عملکرد روش‌ها در شرایط با هم‌پوشانی بالا بین دامنه‌ها

در این شکل مشاهده می‌شود که در شرایط با هم‌پوشانی بالا (چرخش ۱۵ درجه)، تفاوت عملکرد میان روش‌ها محدود است و همگی در بازه‌ی دقت بالایی قرار دارند. با این حال، روش‌های تطبیق دامنه -به‌ویژه هم‌ترازسازی ویژگی‌ها (DANN)- اندکی بهبود نسبت به مدل پایه ایجاد کرده‌اند، که نشان می‌دهد در شیفت‌های ملایم، اصلاح فضای ویژگی می‌تواند به‌صورت تدریجی عملکرد را بهبود دهد.



شکل ۳–۶۴: مقایسه عملکرد روش‌ها در شرایط با هم‌پوشانی پایین بین دامنه‌ها

در مقابل، در شرایط با هم‌پوشانی پایین (چرخش ۹۰ درجه)، تفاوت میان روش‌ها به‌وضوح نمایان می‌شود و تقریباً تمامی روش‌های مبتنی بر ویژگی و توزیع عملکردی نزدیک به مدل پایه و در سطحی پایین دارند. در این میان، تنها روش تبدیل دامنه (CycleGAN) بهبود نسبی -هرچند محدود- ایجاد کرده است، که نشان می‌دهد در شیفت‌های شدید، تغییر مستقیم داده‌ها می‌تواند مؤثرتر از هم‌ترازسازی در فضای ویژگی باشد، هرچند این بهبود همچنان فاصله زیادی با حالت ایده‌آل دارد..

در نهایت، لازم به تأکید است که استفاده از حالت Supervised Fine-Tuning در این فصل صرفاً با هدف تعیین یک کران بالا انجام شده است. این روش به دلیل استفاده از برچسب‌های دامنه هدف، در چارچوب مسئله تطبیق دامنه قرار نمی‌گیرد، زیرا در

این مسئله فرض می‌شود که به برچسب‌های دامنه هدف دسترسی نداریم. بنابراین، نتایج این حالت تنها به‌عنوان مرجعی برای سنجش فاصله عملکرد روش‌های بدون نظارت از حالت ایده‌آل در نظر گرفته می‌شوند.

در مجموع، نتایج نشان می‌دهند که انتخاب روش تطبیق دامنه باید متناسب با شدت و ماهیت شیفت انجام شود. در شیفت‌های ملایم، روش‌های مبتنی بر هم‌ترازسازی ویژگی‌ها (مانند DANN) مؤثرتر هستند، زیرا می‌توانند با حفظ تمایز کلاس‌ها، نمایش‌های مشترک ایجاد کنند. در مقابل، در شیفت‌های شدید، روش‌های مبتنی بر تبدیل دامنه (مانند CycleGAN) عملکرد بهتری دارند، هرچند با هزینه محاسباتی بالاتر و محدودیت در حفظ کامل اطلاعات معنایی. در نهایت، در اختلاف‌های بسیار زیاد، هیچ‌یک از این رویکردها به‌تنهایی کافی نیستند و حل کامل مسئله نیازمند ترکیب روش‌ها یا استفاده از اطلاعات نظارتی است.

فصل چهارم

یادگیری انتقالی

در این فصل، به بررسی یادگیری انتقالی به‌عنوان یکی از رویکردهای مؤثر در بهبود تعمیم‌پذیری مدل‌ها، به‌ویژه در شرایط کمبود داده، پرداخته می‌شود. ابتدا تأثیر این روش بر مؤلفه‌های خطا شامل بایاس و واریانس تحلیل شده و سپس در قالب یک آزمایش عملی با استفاده از معماری LeNet، عملکرد آن در مقایسه با آموزش از صفر بررسی می‌گردد. در ادامه، با تحلیل رفتار لایه‌های مختلف شبکه، میزان انتقال‌پذیری آن‌ها و شرایط برتری یادگیری انتقالی نسبت به سایر رویکردها مورد ارزیابی قرار می‌گیرد.

۴-۱- تأثیر یادگیری انتقالی بر تعمیم‌پذیری و تحلیل آن در چارچوب Bias–Variance

یادگیری انتقالی رویکردی است که در آن دانش حاصل از آموزش یک مدل بر روی یک مسئله یا مجموعه‌داده (دامنه مبدأ) به مسئله‌ای دیگر (دامنه هدف) که معمولاً دارای داده‌های محدودتری است منتقل می‌شود. هدف اصلی این روش، بهبود تعمیم‌پذیری مدل در شرایطی است که آموزش از صفر به دلیل کمبود داده یا پیچیدگی مسئله، منجر به عملکرد ضعیف می‌شود.

برای تحلیل دقیق‌تر اثر این روش، می‌توان از چارچوب Bias–Variance Tradeoff استفاده کرد. خطای کلی مدل را می‌توان به دو مؤلفه اصلی بایاس و واریانس تجزیه کرد. بایاس ناشی از ساده‌سازی بیش از حد مدل و ناتوانی آن در تقریب تابع واقعی است، در حالی که واریانس ناشی از حساسیت بیش از حد مدل به داده‌های آموزشی و یادگیری نویز موجود در آن‌ها است.

در شرایطی که داده‌های آموزشی محدود باشند، آموزش یک مدل از ابتدا معمولاً منجر به واریانس بالا می‌شود، زیرا مدل برای تنظیم پارامترهای خود به شدت به نمونه‌های موجود وابسته خواهد بود. در چنین حالتی، مدل ممکن است الگوهای تصادفی و نویزی را نیز به‌عنوان ساختار واقعی یاد بگیرد و در نتیجه روی داده‌های جدید عملکرد ضعیفی داشته باشد.

یادگیری انتقالی با فراهم کردن یک مقداردهی اولیه مبتنی بر دانش قبلی، این مشکل را تا حد زیادی کاهش می‌دهد. در این روش، پارامترهای مدل به‌صورت تصادفی مقداردهی اولیه نمی‌شوند، بلکه از مدلی که قبلاً روی یک مجموعه‌داده بزرگ آموزش دیده است، به ارث می‌رسند. این امر باعث می‌شود که فضای جستجوی مدل محدودتر شده و فرآیند یادگیری به سمت نواحی معنادارتر از فضای پارامترها هدایت شود. در نتیجه، حساسیت مدل به داده‌های محدود کاهش یافته و واریانس به‌طور قابل توجهی کاهش می‌یابد.

از سوی دیگر، تأثیر یادگیری انتقالی بر بایاس وابسته به میزان شباهت بین دامنه مبدأ و دامنه هدف است. اگر این دو دامنه به یکدیگر نزدیک باشند، ویژگی‌های آموخته‌شده در دامنه مبدأ می‌توانند به‌عنوان نمایش‌های مناسبی از داده‌های دامنه هدف نیز

عمل کنند. در این حالت، مدل نه‌تنها دچار افزایش بایاس نمی‌شود، بلکه به دلیل شروع از یک نمایش بهتر، سریع‌تر به تقریب مناسبی از تابع هدف می‌رسد و بایاس آن کاهش می‌یابد یا حداقل افزایش پیدا نمی‌کند. در مقابل، اگر دامنه‌ها تفاوت زیادی داشته باشند، دانش منتقل‌شده ممکن است نامرتبط باشد و مدل را به سمت الگوهای نادرست هدایت کند که در این صورت بایاس افزایش می‌یابد؛ این پدیده به‌عنوان انتقال منفی[۳۱] شناخته می‌شود.

برای درک بهتر این موضوع، فرض کنید مدلی برای تشخیص اشیاء در تصاویر طبیعی (مانند حیوانات و وسایل نقلیه) آموزش داده شده است. این مدل در لایه‌های اولیه خود قادر به استخراج ویژگی‌های پایه‌ای مانند لبه‌ها، بافت‌ها و الگوهای ساده است. حال اگر بخواهیم همین مدل را برای یک مسئله مشابه، مانند تشخیص انواع حیوانات خاص، به کار ببریم، این ویژگی‌های پایه همچنان معتبر و مفید خواهند بود. در نتیجه، تنها با تنظیم لایه‌های نهایی، مدل می‌تواند به عملکرد خوبی دست یابد، در حالی که اگر همین مدل را از ابتدا و با داده‌های محدود آموزش دهیم، احتمالاً به دلیل کمبود داده دچار بیش‌برازش خواهد شد. در این مثال، یادگیری انتقالی با استفاده از ویژگی‌های عمومی آموخته‌شده، باعث کاهش واریانس و در عین حال عدم افزایش یا کاهش بایاس می‌شود.

در مجموع، یادگیری انتقالی را می‌توان به‌عنوان نوعی regularization ضمنی در نظر گرفت که با کاهش واریانس و، در صورت شباهت دامنه‌ها، کاهش یا کنترل بایاس، منجر به بهبود تعمیم‌پذیری مدل می‌شود. این ویژگی به‌ویژه در سناریوهایی که داده‌های هدف محدود هستند، اهمیت بیشتری پیدا می‌کند و یکی از دلایل اصلی محبوبیت این روش در کاربردهای عملی یادگیری عمیق محسوب می‌شود.

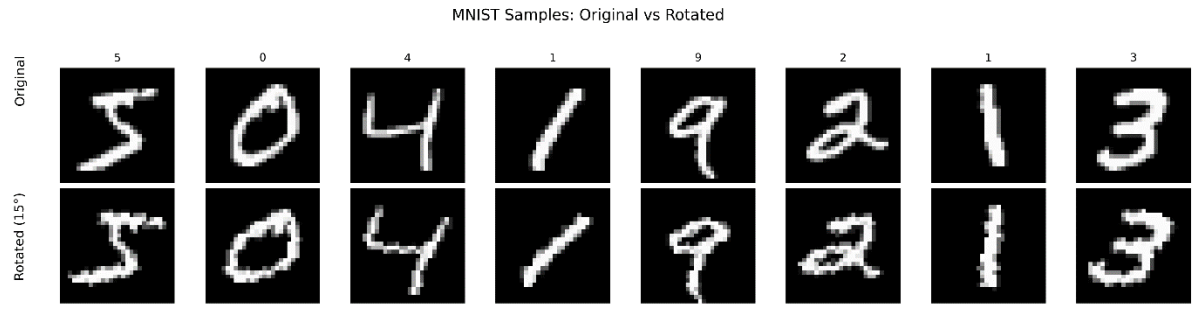
۴-۲- ارزیابی تجربی یادگیری انتقالی با استفاده از معماری LeNet

در این بخش، عملکرد یادگیری انتقالی در قالب یک آزمایش عملی مورد بررسی قرار می‌گیرد. ابتدا یک مدل LeNet بر روی مجموعه کامل MNIST آموزش داده می‌شود تا به‌عنوان مدل مبدأ عمل کند. سپس با ثابت نگه داشتن لایه‌های اولیه و تنظیم لایه‌های نهایی، مدل بر روی یک زیرمجموعه کوچک از داده‌های چرخیده fine-tune می‌شود. در ادامه، عملکرد این روش با حالتی که مدل از ابتدا و تنها با داده‌های محدود آموزش داده می‌شود مقایسه شده و رفتار آن‌ها از منظر تعمیم‌پذیری و بیش‌برازش تحلیل می‌گردد.

۴-۲-۱- بررسی داده‌ها و تحلیل تغییرات ناشی از چرخش

در این آزمایش، داده‌ها شامل تصاویر مجموعه MNIST در دو حالت دامنه مبدأ (تصاویر اصلی) و دامنه هدف (تصاویر چرخیده با زاویه ۱۵ درجه) هستند. اعمال این چرخش یک نمونه از جابجایی دامنه محسوب می‌شود که بدون تغییر در برچسب‌ها، توزیع داده‌های ورودی را تغییر می‌دهد.

[31] Negative Transfer



*شکل ۴–۱: نمونه‌هایی از داده‌های **MNIST** در حالت اصلی (ردیف بالا) و نسخه‌ی چرخیده با زاویه ۱۵ درجه (ردیف پایین)*

همان‌طور که در شکل مشاهده می‌شود، چرخش اعمال‌شده اگرچه ساختار کلی ارقام را حفظ کرده است، اما باعث تغییر در جهت‌گیری و نحوه‌ی قرارگیری اجزای تصویر شده است. این تغییر به‌ویژه در ارقامی که دارای ساختار جهت‌دار هستند، محسوس‌تر بوده و منجر به تغییر در الگوهای فضایی می‌شود که مدل در حالت عادی از آن‌ها استفاده می‌کند.

از دید مدل، این مسئله به معنای تغییر در توزیع ویژگی‌های ورودی است. در نتیجه، مدلی که صرفاً روی داده‌های اصلی آموزش دیده باشد، ممکن است در مواجهه با این تصاویر دچار افت عملکرد شود، زیرا نمایش ویژگی‌ها نسبت به حالت آموزش اولیه متفاوت شده است.

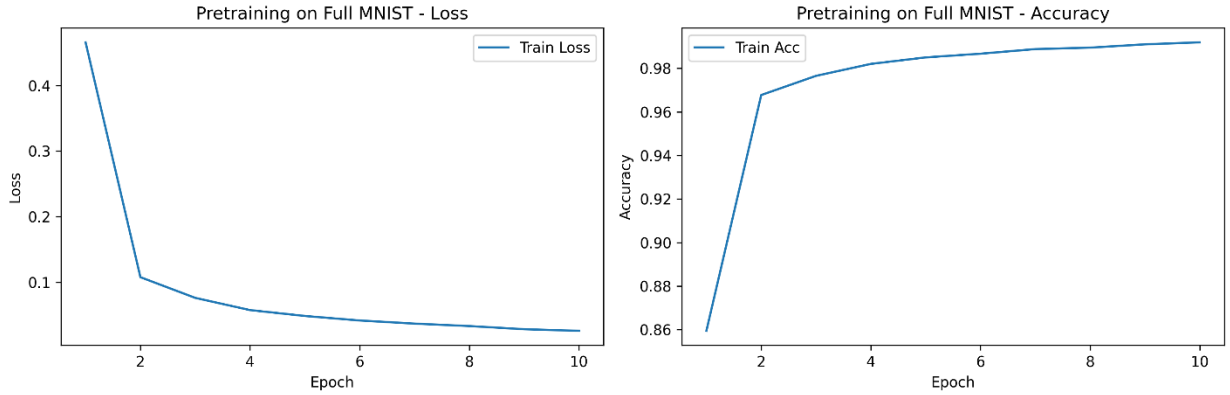
در عین حال، ویژگی‌های پایه‌ای مانند لبه‌ها و ساختار کلی ارقام همچنان حفظ شده‌اند و تنها دچار تغییر در جهت‌گیری شده‌اند. این موضوع نشان می‌دهد که اطلاعات اصلی مسئله از بین نرفته، بلکه نحوه‌ی نمایش آن تغییر کرده است. بنابراین انتظار می‌رود لایه‌های اولیه شبکه که ویژگی‌های عمومی را استخراج می‌کنند، همچنان قابل استفاده باشند، در حالی که لایه‌های نهایی نیاز به تطبیق با این تغییر دارند.

این ویژگی، این سناریو را به نمونه‌ای مناسب برای بررسی یادگیری انتقالی تبدیل می‌کند، زیرا تغییر ایجادشده بیشتر بر نحوه‌ی تفسیر ویژگی‌ها اثر می‌گذارد تا ماهیت خود آن‌ها.

۴-۲-۲- پیش‌آموزش مدل LeNet بر روی مجموعه MNIST

در این مرحله، مدل LeNet بر روی کل مجموعه آموزشی MNIST آموزش داده شد تا به‌عنوان مدل مبدأ در فرآیند یادگیری انتقالی مورد استفاده قرار گیرد. با توجه به هدف این بخش که استخراج یک نمایش مناسب از داده‌ها است، از کل داده‌های آموزشی برای پیش‌آموزش استفاده شد و مجموعه اعتبارسنجی جداگانه‌ای در نظر گرفته نشد. این تصمیم با هدف بهره‌گیری حداکثری از داده‌ها و دستیابی به یک مدل پایه قوی اتخاذ شده است، زیرا در این مرحله تمرکز بر تنظیم دقیق ابرپارامترها نیست، بلکه هدف، یادگیری ویژگی‌های عمومی و پایدار از داده‌ها است.

روند آموزش مدل در شکل زیر نشان داده شده است.



شکل ۴–۲: روند تغییرات ***loss*** *و دقت مدل* ***LeNet*** *در مرحله پیش‌آموزش بر روی کل مجموعه* ***MNIST***

مطابق شکل، کاهش یکنواخت مقدار تابع هزینه در کنار افزایش تدریجی دقت نشان‌دهنده‌ی همگرایی مناسب مدل در طول فرآیند آموزش است. رفتار نرم و بدون نوسان شدید منحنی‌ها بیانگر پایداری فرآیند یادگیری و انتخاب مناسب نرخ یادگیری است. همچنین، رسیدن دقت آموزش به مقادیر بالا نشان می‌دهد که مدل توانسته ساختارهای اصلی داده را به‌خوبی فرا بگیرد.

برای ارزیابی کلی مدل، عملکرد آن بر روی مجموعه تست MNIST نیز اندازه‌گیری شد که دقتی برابر با ۰.۹۸۸۱ به دست آمد. این مقدار نشان می‌دهد که مدل نه‌تنها داده‌های آموزشی را به‌خوبی یاد گرفته، بلکه توانایی تعمیم به داده‌های دیده‌نشده در همان دامنه را نیز داراست.

نکته مهم در این مرحله آن است که هدف اصلی از پیش‌آموزش، دستیابی به یک مدل با عملکرد بهینه روی MNIST نیست، بلکه فراهم کردن یک نمایش ویژگی مناسب برای انتقال به دامنه هدف است. بنابراین، کیفیت این مرحله زمانی اهمیت پیدا می‌کند که مدل بتواند ویژگی‌های عمومی و قابل انتقال را در لایه‌های اولیه خود یاد بگیرد، به‌گونه‌ای که در مراحل بعدی، تنها با تنظیم بخش کوچکی از شبکه بتوان آن را با داده‌های جدید تطبیق داد.

۴-۲-۳- پیکربندی آزمایش‌ها و تنظیمات انتقال دانش

در این بخش، به‌منظور بررسی اثر یادگیری انتقالی، چندین پیکربندی مختلف از آموزش مدل طراحی شده است تا امکان مقایسه‌ی مستقیم بین حالت‌های مختلف فراهم شود. در تمامی این پیکربندی‌ها، داده‌های مورد استفاده شامل یک زیرمجموعه‌ی ۲۰۰ نمونه‌ای از تصاویر MNIST با چرخش ۱۵ درجه است تا شرایط کمبود داده در دامنه هدف شبیه‌سازی شود.

مدل پایه در تمامی حالت‌های مبتنی بر یادگیری انتقالی، همان مدلی است که در مرحله پیش‌آموزش بر روی کل مجموعه MNIST آموزش داده شده است. تفاوت اصلی بین این حالت‌ها در میزان استفاده از دانش قبلی و نحوه‌ی به‌روزرسانی پارامترهای مدل در مرحله fine-tuning است.

در ساده‌ترین حالت یادگیری انتقالی، لایه‌های کانولوشنی اولیه مدل شامل conv1 و conv2 ثابت نگه داشته شده و تنها لایه‌های fully connected (fc1، fc2 و fc3) بر روی داده‌های جدید آموزش داده می‌شوند. در این حالت، فرض بر این است که ویژگی‌های استخراج‌شده در لایه‌های اولیه کاملاً عمومی هستند و نیازی به به‌روزرسانی آن‌ها وجود ندارد.

در حالت دوم، تنها لایه‌ی اول کانولوشنی (conv1) ثابت نگه داشته شده و لایه‌ی دوم کانولوشنی به همراه لایه‌های fully connected قابل آموزش در نظر گرفته می‌شوند. این پیکربندی امکان تطبیق بخشی از ویژگی‌های میانی مدل با دامنه جدید را فراهم می‌کند، در حالی که همچنان از ویژگی‌های پایه استخراج‌شده در لایه‌های ابتدایی استفاده می‌شود.

در حالت سوم، تمامی پارامترهای مدل از مقداردهی اولیه‌ی حاصل از پیش‌آموزش استفاده کرده و کل شبکه در مرحله fine-tuning قابل آموزش است. این رویکرد بیشترین میزان انعطاف‌پذیری را برای سازگاری با دامنه هدف فراهم می‌کند، اما در عین حال ممکن است در شرایط داده محدود، مستعد بیش‌برازش باشد.

در نهایت، به‌عنوان خط پایه، یک مدل با همان معماری LeNet از ابتدا و تنها با استفاده از همین ۲۰۰ نمونه‌ی چرخیده آموزش داده می‌شود. این حالت نشان‌دهنده‌ی عملکرد مدل در غیاب هرگونه دانش قبلی بوده و به‌عنوان مرجعی برای ارزیابی میزان تأثیر یادگیری انتقالی مورد استفاده قرار می‌گیرد.

در تمامی این آزمایش‌ها، تنظیمات آموزش شامل نرخ یادگیری، تعداد epoch و اندازه batch به‌صورت یکسان در نظر گرفته شده است تا مقایسه‌ی بین حالت‌ها به‌صورت منصفانه انجام شود و تفاوت‌های مشاهده‌شده صرفاً ناشی از نحوه‌ی استفاده از دانش پیش‌آموخته باشد.

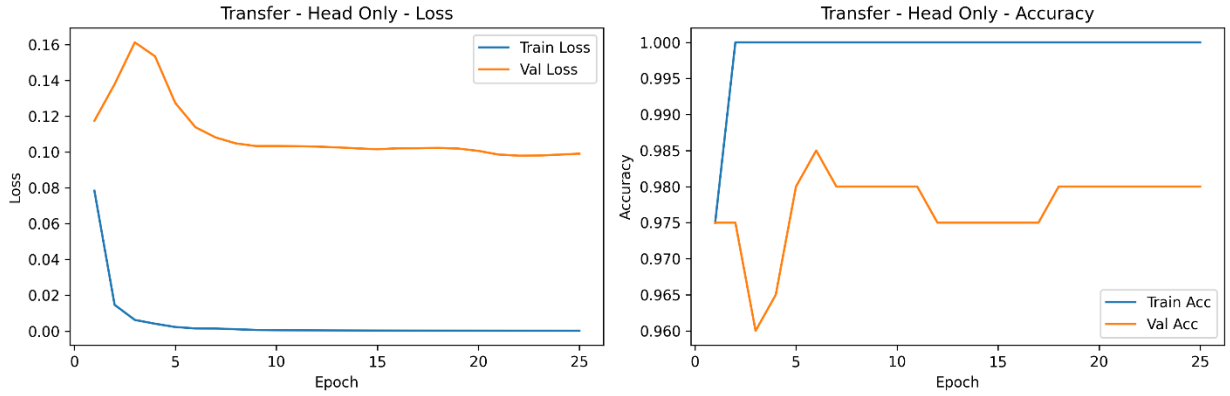
۴-۲-۴- مقایسه عملکرد حالت‌های مختلف یادگیری انتقالی

در این بخش، عملکرد چهار پیکربندی مختلف روی مجموعه‌ی ۲۰۰ نمونه‌ای از داده‌های چرخیده بررسی می‌شود. این پیکربندی‌ها از نظر تعداد پارامترهای قابل آموزش و میزان استفاده از دانش پیش‌آموخته تفاوت دارند که در جدول زیر خلاصه شده است.

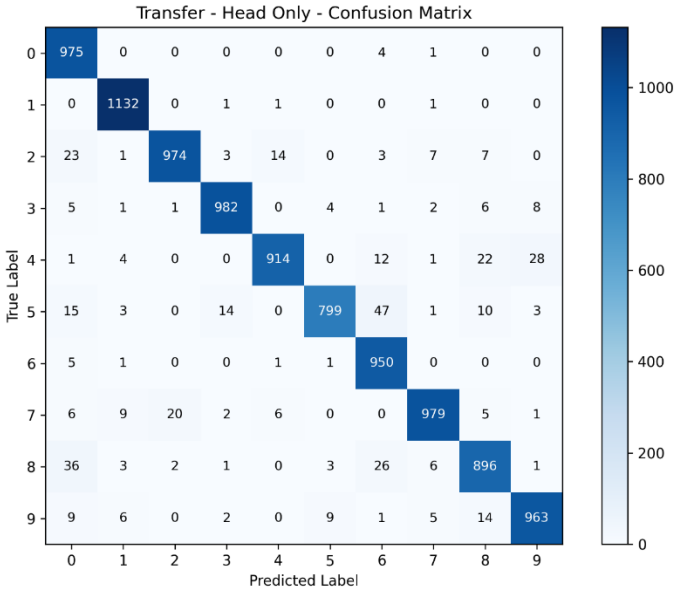
حالت آزمایش	پارامترهای قابل آموزش (کل: ۴۴۴۲۶)	Best Val Acc	Test Acc (Rot15)	Final Train Acc	Final Val Acc	Final Train Loss	Final Val Loss
Head Only	۴۱۸۵۴	۰.۹۸۵	۰.۹۵۶۴	۱.۰۰۰	۰.۹۸۰	۰.۰۰۰۱۶۵	۰.۰۹۹۰۹۶
Conv2 + Head	۴۴۲۷۰	۰.۹۸۰	۰.۹۵۲۷	۱.۰۰۰	۰.۹۷۵	۰.۰۰۰۰۸۰	۰.۱۲۲۸۲۷
Full Fine-tune	۴۴۴۲۶	۰.۹۸۵	۰.۹۶۱۰	۱.۰۰۰	۰.۹۸۵	۰.۰۰۰۰۹۰	۰.۱۲۴۷۵۹
From Scratch	۴۴۴۲۶	۰.۸۰۵	۰.۸۰۲۳	۰.۹۷۵	۰.۷۹۵	۰.۱۰۸۸۲۱	۰.۷۳۷۰۶۶

۴-۲-۴-۱- حالت Head Only

در این حالت تنها لایه‌های fully connected به‌روزرسانی می‌شوند و بخش استخراج ویژگی بدون تغییر باقی می‌ماند.



شکل ۴–۳: روند تغییرات ***loss*** *و* ***accuracy*** *در حالت* ***Transfer - Head Only***



شکل ۴–۴: ماتریس درهم‌ریختگی مدل در حالت ***Transfer - Head Only***

در نمودارهای آموزش، رفتار loss نشان می‌دهد که مدل در همان epochهای ابتدایی به‌سرعت روی داده‌های آموزشی همگرا شده و مقدار train loss تقریباً به صفر می‌رسد. این کاهش سریع بیانگر آن است که پارامترهای قابل آموزش (لایه‌های fully connected) به‌راحتی قادر به برازش کامل روی مجموعه‌ی کوچک ۲۰۰ نمونه‌ای هستند. در مقابل، منحنی validation loss در ابتدا افزایش یافته و سپس به‌تدریج کاهش یافته و در یک مقدار تقریباً ثابت تثبیت می‌شود. این رفتار اولیه (افزایش سپس کاهش) نشان‌دهنده‌ی فاز سازگاری اولیه classifier با نمایش‌های ازپیش‌آموخته‌شده است.

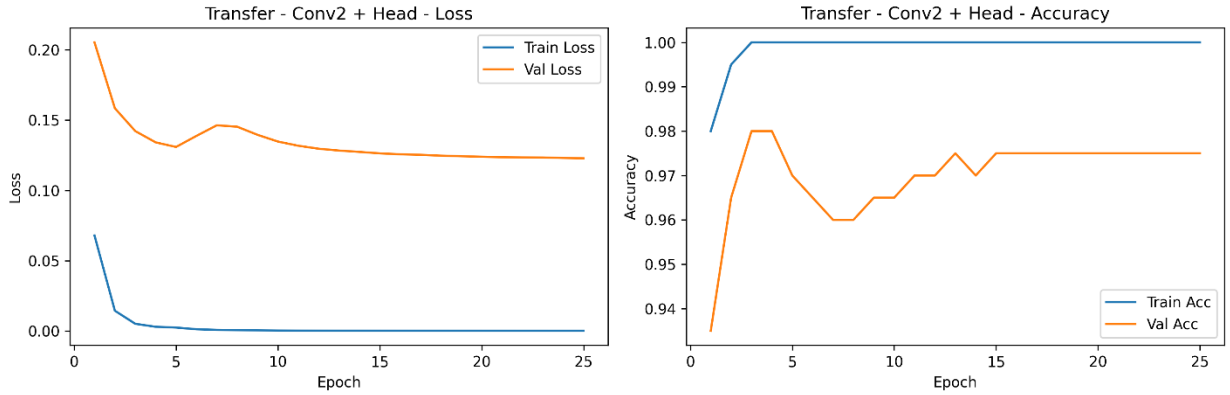
با این حال، فاصله‌ی قابل توجه بین train loss (نزدیک صفر) و validation loss (در سطحی بالاتر و تقریباً ثابت) نشان می‌دهد که مدل دچار بیش‌برازش شده است، اما این بیش‌برازش از نوع شدید و مخرب نیست. دلیل این موضوع آن است که بخش استخراج ویژگی ثابت باقی مانده و تنها بخش کوچکی از مدل قابلیت تغییر دارد، بنابراین ظرفیت مدل برای انطباق بیش‌ازحد با داده‌های محدود کنترل شده است. این موضوع در نمودار accuracy نیز دیده می‌شود؛ دقت آموزش به‌سرعت به مقدار کامل می‌رسد، در حالی که دقت validation در یک بازه محدود نوسان کرده و سپس پایدار می‌شود، که نشان‌دهنده‌ی تعمیم قابل قبول مدل است.

در ماتریس درهم‌ریختگی نیز الگوی خطاها ساختارمند و محدود است. بیشترین خطاها مربوط به کلاس‌هایی است که پس از چرخش، شباهت بصری بیشتری به یکدیگر پیدا می‌کنند، مانند اشتباه بین برخی ارقام خمیده یا دارای تقارن جزئی. با این حال، بخش عمده‌ای از جرم ماتریس روی قطر اصلی متمرکز است که نشان‌دهنده‌ی عملکرد پایدار مدل در تفکیک اکثر کلاس‌ها است. این موضوع تأیید می‌کند که ویژگی‌های استخراج‌شده توسط لایه‌های کانولوشنی از پیش‌آموزش‌دیده، بدون نیاز به تغییر، همچنان توانایی توصیف مناسب داده‌های چرخیده را دارند.

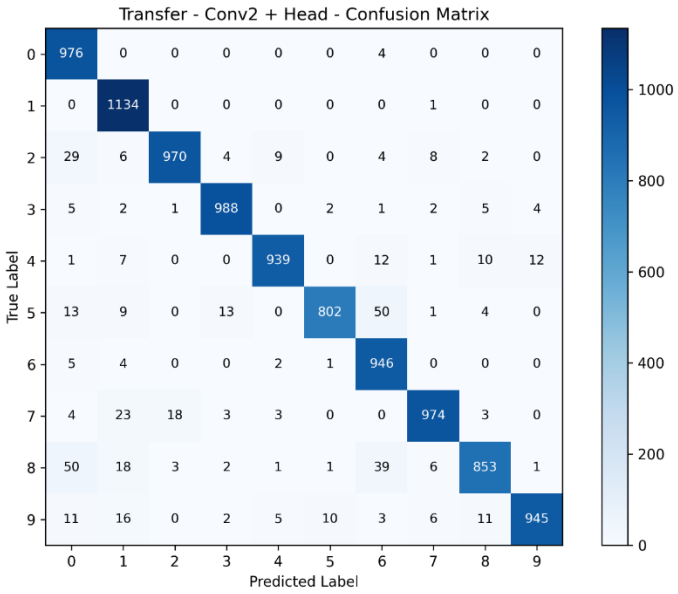
در مجموع، این حالت نشان می‌دهد که حتی با وجود بیش‌برازش در لایه‌های نهایی، استفاده از نمایش‌های ازپیش‌آموخته‌شده باعث می‌شود مدل همچنان تعمیم‌پذیری قابل قبولی داشته باشد و خطاها به‌صورت کنترل‌شده و قابل تفسیر باقی بمانند.

۴-۲-۴-۲- حالت Conv2 + Head

در این حالت علاوه بر classifier، لایه‌ی دوم کانولوشنی نیز قابل آموزش است.



شکل ۴–۵: روند تغییرات ***loss*** *و* ***accuracy*** *در حالت* ***Transfer - Conv2 + Head***



شکل ۴–۶: ماتریس درهم‌ریختگی مدل در حالت ***Transfer - Conv2 + Head***

در نمودارهای آموزش، مشاهده می‌شود که train loss همانند حالت قبل به‌سرعت به مقادیر نزدیک به صفر کاهش می‌یابد و دقت آموزش نیز خیلی زود به مقدار کامل می‌رسد. این موضوع نشان می‌دهد که با افزایش تعداد پارامترهای قابل آموزش (آزاد شدن conv2)، مدل همچنان توانایی برازش کامل داده‌های محدود را دارد. با این حال، رفتار validation متفاوت‌تر است؛ validation loss پس از کاهش اولیه، یک ناحیه‌ی نوسانی و سپس یک روند کاهشی بسیار کند را طی می‌کند و در سطحی بالاتر از حالت Head Only تثبیت می‌شود.

این فاصله‌ی مشخص بین train loss و validation loss، همراه با اشباع کامل دقت آموزش، نشان‌دهنده‌ی بیش‌برازش قوی‌تر نسبت به حالت قبل است. در نمودار accuracy نیز این موضوع دیده می‌شود؛ دقت validation پس از رسیدن به یک مقدار نسبتاً بالا، دچار افت شده و سپس با نوساناتی جزئی به یک سطح پایدار می‌رسد. این الگو بیانگر آن است که مدل در ابتدا سعی در

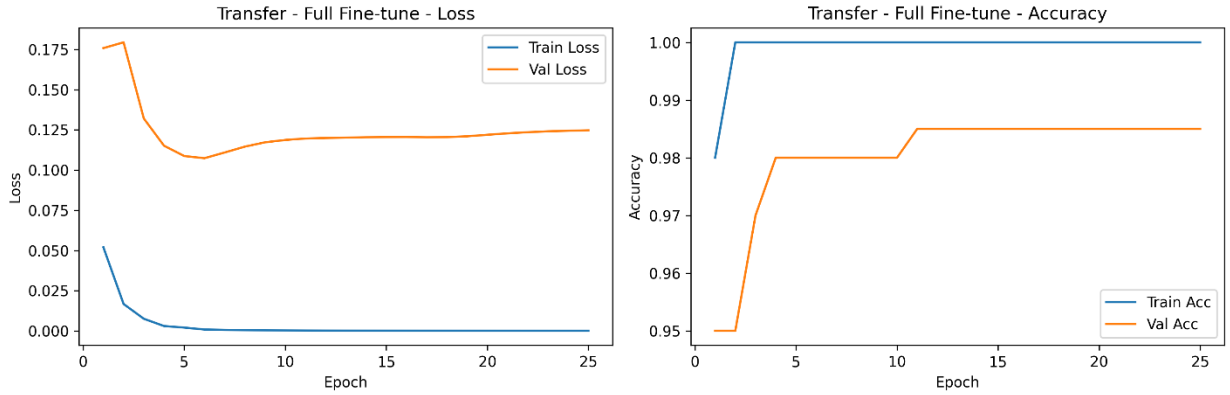
تطبیق سریع با داده‌ها دارد، اما به‌مرور بخشی از تعمیم‌پذیری خود را از دست می‌دهد و در یک تعادل ضعیف‌تر نسبت به حالت Head Only قرار می‌گیرد.

در ماتریس درهم‌ریختگی نیز این کاهش پایداری قابل مشاهده است. اگرچه همچنان جرم اصلی روی قطر قرار دارد، اما پراکندگی خطاها نسبت به حالت قبلی بیشتر شده است. برخی کلاس‌ها، به‌ویژه آن‌هایی که از نظر ساختار نوشتاری به یکدیگر شباهت دارند، دچار اختلاط بیشتری شده‌اند. این موضوع نشان می‌دهد که تغییر در لایه‌ی conv2 باعث شده بخشی از ویژگی‌های میانی که در پیش‌آموزش شکل گرفته بودند، دچار تغییر شوند و این تغییرات همیشه در جهت بهبود نبوده‌اند.

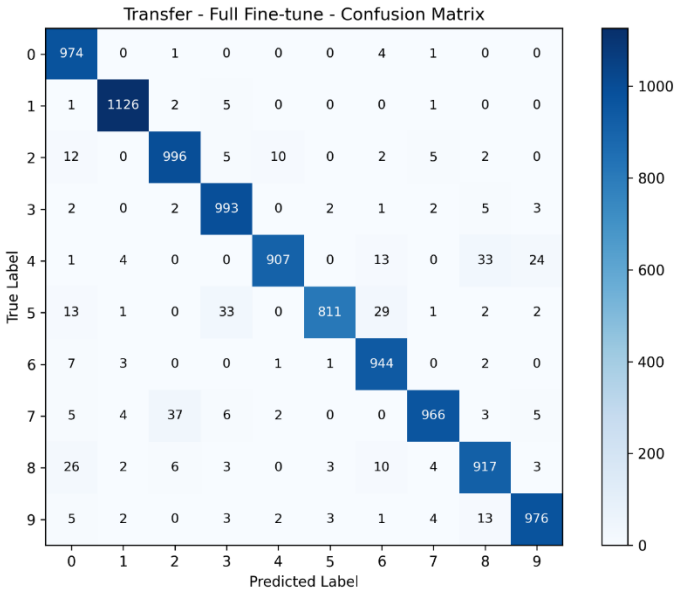
به‌طور کلی، این حالت نشان می‌دهد که افزایش ظرفیت مدل از طریق آزاد کردن لایه‌های عمیق‌تر، در حضور داده‌ی محدود می‌تواند اثر دوگانه داشته باشد: از یک سو امکان تطبیق بیشتر با دامنه هدف را فراهم می‌کند، اما از سوی دیگر باعث ناپایداری نمایش‌های آموخته‌شده و افزایش بیش‌برازش می‌شود. در اینجا، این اثر منفی غالب بوده و عملکرد نسبت به حالت ساده‌تر (Head Only) اندکی تضعیف شده است.

۴-۲-۴-۳- حالت Full Fine-tune

در این حالت تمام پارامترهای مدل قابل آموزش هستند.



شکل ۴–۷: روند تغییرات ***loss*** *و* ***accuracy*** *در حالت* ***Transfer - Full Fine-tune***



شکل ۴–۸: ماتریس درهم‌ریختگی مدل در حالت ***Transfer - Full Fine-tune***

در این حالت، با آزاد شدن تمامی پارامترهای مدل، بیشترین ظرفیت برای یادگیری فراهم می‌شود و این موضوع به‌وضوح در نمودارهای آموزش قابل مشاهده است. منحنی train loss به‌سرعت به مقادیر نزدیک به صفر همگرا می‌شود و دقت آموزش در همان epochهای ابتدایی به مقدار اشباع می‌رسد. این رفتار نشان‌دهنده‌ی توان بالای مدل در برازش کامل داده‌های آموزشی است. در مقابل، validation loss پس از کاهش اولیه، به‌تدریج افزایش یافته و در سطحی بالاتر نسبت به نقطه‌ی بهینه تثبیت می‌شود. این واگرایی بین train و validation loss، نشانه‌ی مشخصی از بروز بیش‌برازش است که نسبت به حالت‌های قبلی شدت بیشتری دارد.

در نمودار accuracy نیز این پدیده قابل مشاهده است. دقت validation در مراحل ابتدایی رشد کرده و به مقدار بالایی می‌رسد، اما پس از آن بهبود قابل توجهی نداشته و در یک سطح تقریباً ثابت باقی می‌ماند، در حالی که دقت آموزش در سطح

کامل حفظ می‌شود. این شکاف پایدار بین عملکرد آموزش و validation نشان می‌دهد که مدل، علی‌رغم یادگیری دقیق داده‌های آموزشی، در تعمیم به داده‌های جدید محدود شده است. با این حال، عدم افت محسوس در دقت validation بیانگر آن است که مقداردهی اولیه ناشی از پیش‌آموزش همچنان نقش مهمی در حفظ عملکرد ایفا می‌کند.

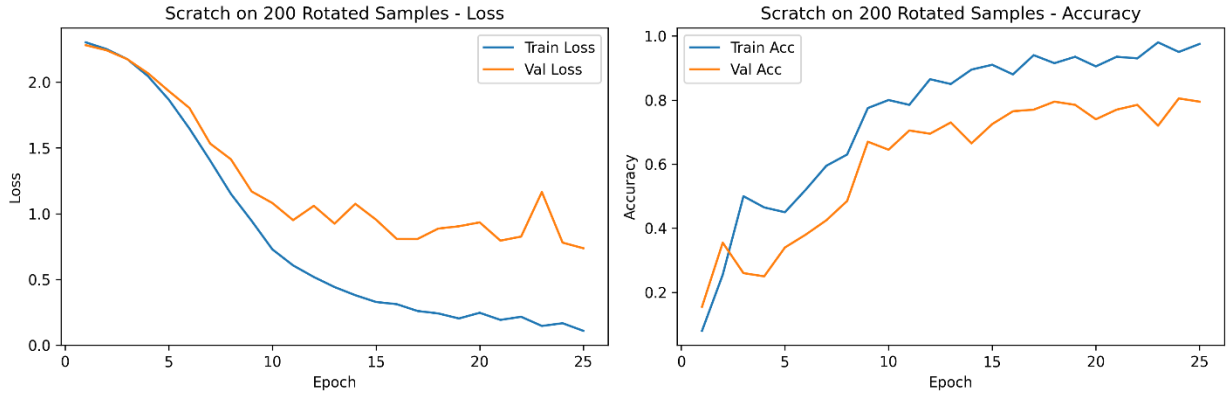
بررسی ماتریس درهم‌ریختگی نشان می‌دهد که این پیکربندی، دقیق‌ترین تفکیک کلاس‌ها را در میان حالت‌های انتقالی ارائه می‌دهد. نسبت به هر دو حالت Head Only و Conv2 + Head، تمرکز جرم روی قطر اصلی بیشتر شده و پراکندگی خطاها کاهش یافته است. این موضوع بیانگر آن است که مدل با در اختیار داشتن انعطاف کامل، توانسته خود را بهتر با تغییرات دامنه (چرخش) تطبیق دهد و نمایش‌های مناسب‌تری برای داده‌های هدف بیاموزد. با این حال، خطاها به‌طور کامل حذف نشده‌اند و همچنان در کلاس‌هایی با شباهت ساختاری بالا مشاهده می‌شوند، که نشان‌دهنده‌ی محدودیت‌های ذاتی مسئله است.

نکته قابل توجه آن است که با وجود بهبود در دقت و ساختار بهتر ماتریس درهم‌ریختگی، مقدار validation loss در این حالت نسبت به پیکربندی‌های ساده‌تر بیشتر است. این موضوع نشان می‌دهد که مدل در برخی نمونه‌های نادرست، با اطمینان بالاتری پیش‌بینی اشتباه انجام می‌دهد، که منجر به افزایش loss می‌شود. بنابراین، اگرچه مدل از نظر تعداد پیش‌بینی‌های صحیح عملکرد بهتری دارد، اما از نظر اطمینان در پیش‌بینی‌ها رفتار تیزتری از خود نشان می‌دهد.

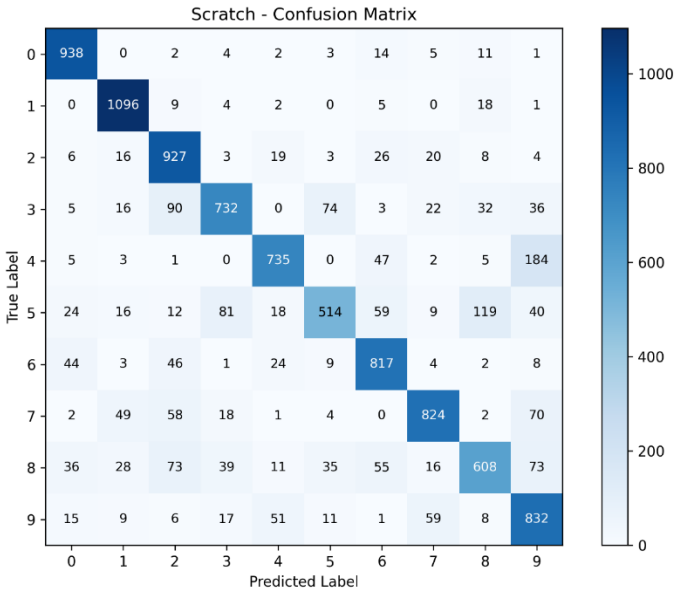
در مجموع، این حالت نشان می‌دهد که آزادسازی کامل پارامترها امکان تطبیق دقیق‌تر با دامنه هدف را فراهم می‌کند و منجر به بهبود تفکیک کلاس‌ها می‌شود، اما این افزایش ظرفیت با هزینه‌ی افزایش بیش‌برازش و کاهش پایداری در برآورد عدم قطعیت همراه است.

۴-۴-۲-۴- حالت From Scratch

در این حالت مدل بدون استفاده از دانش قبلی تنها با ۲۰۰ نمونه آموزش داده می‌شود.



شکل ۴–۹: روند تغییرات ***loss*** *و* ***accuracy*** *در حالت* ***Scratch***



شکل ۴–۱۰: ماتریس درهم‌ریختگی مدل در حالت ***Scratch***

در این حالت، مدل بدون استفاده از دانش پیش‌آموخته و تنها با ۲۰۰ نمونه‌ی آموزشی آموزش داده شده است، بنابراین رفتار نمودارها به‌طور قابل توجهی با حالات انتقالی متفاوت است. در نمودار loss مشاهده می‌شود که train loss به‌صورت یکنواخت و قابل توجهی کاهش می‌یابد، در حالی که validation loss اگرچه در ابتدا کاهش پیدا می‌کند، اما در ادامه در مقدار نسبتاً بالایی نوسان داشته و روند پایداری ندارد. این فاصله‌ی زیاد و پایدار بین train و validation loss، نشان‌دهنده‌ی بیش‌برازش شدید مدل است.

در نمودار accuracy نیز همین الگو دیده می‌شود. دقت آموزش به‌مرور افزایش یافته و به مقادیر بالا نزدیک می‌شود، اما دقت validation با فاصله‌ی قابل توجهی پایین‌تر باقی می‌ماند و نوسانات محسوسی دارد. این رفتار نشان می‌دهد که مدل در حال

حفظ الگوهای داده‌های آموزشی است، اما قادر به تعمیم آن‌ها به داده‌های جدید نیست. در مقایسه با تمامی حالت‌های مبتنی بر یادگیری انتقالی، این شکاف بین train و validation در اینجا به‌مراتب بزرگ‌تر است و بیانگر ضعف جدی در تعمیم‌پذیری است.

بررسی ماتریس درهم‌ریختگی نیز این موضوع را به‌خوبی تأیید می‌کند. برخلاف حالات انتقالی که جرم ماتریس عمدتاً روی قطر اصلی متمرکز بود، در اینجا پراکندگی خطاها در کل ماتریس به‌وضوح مشاهده می‌شود. بسیاری از کلاس‌ها به‌طور قابل توجهی با یکدیگر اشتباه گرفته شده‌اند، به‌ویژه در کلاس‌هایی که از نظر ساختاری شباهت بیشتری دارند. این پراکندگی نشان می‌دهد که مدل نتوانسته ویژگی‌های پایدار و متمایزکننده‌ای برای کلاس‌ها یاد بگیرد و در نتیجه مرزهای تصمیم آن به‌درستی شکل نگرفته‌اند.

نکته مهم دیگر آن است که با وجود ظرفیت کامل مدل (مشابه حالت Full Fine-tune)، عملکرد به‌مراتب ضعیف‌تری حاصل شده است. این موضوع نشان می‌دهد که مشکل اصلی در این سناریو، کمبود داده و نبود مقداردهی اولیه مناسب است، نه ظرفیت مدل. در واقع، در غیاب دانش قبلی، مدل مجبور است تمامی ویژگی‌ها را از ابتدا و بر اساس داده‌ی محدود یاد بگیرد که منجر به یادگیری ناپایدار و بیش‌برازش شدید می‌شود.

در مجموع، این حالت نشان می‌دهد که آموزش از صفر در شرایط داده‌ی محدود نه‌تنها منجر به بیش‌برازش شدید می‌شود، بلکه باعث می‌شود مدل حتی در سطح استخراج ویژگی نیز دچار ضعف باشد و نتواند نمایش‌های مناسبی برای تفکیک کلاس‌ها ایجاد کند.

۴-۲-۴-۵- جمع‌بندی مقایسه‌ای حالت‌های مختلف آموزش

نتایج به‌دست‌آمده نشان می‌دهد که نحوه‌ی انتخاب لایه‌های قابل آموزش در یادگیری انتقالی، تأثیر مستقیمی بر پایداری آموزش، میزان بیش‌برازش، دقت نهایی و هزینه‌ی محاسباتی دارد. در حالت Head Only، به دلیل محدود بودن پارامترهای قابل آموزش، فرآیند آموزش پایدار بوده و منحنی‌های آموزش و ولیدیشن نوسان کمی دارند. مدل با وجود بیش‌برازش روی داده‌های آموزش، تعمیم‌پذیری مناسبی روی داده‌های ولیدیشن و تست حفظ می‌کند. این رفتار نشان می‌دهد که ویژگی‌های استخراج‌شده در لایه‌های کانولوشنی، تا حد زیادی برای دامنه‌ی هدف (تصاویر چرخیده) قابل استفاده بوده‌اند.

با باز کردن لایه‌های بیشتر در حالت Conv2 + Head، ظرفیت مدل افزایش پیدا می‌کند، اما این افزایش ظرفیت لزوماً به بهبود عملکرد منجر نشده و حتی نوسانات بیشتری در روند آموزش و افت نسبی در عملکرد مشاهده می‌شود. این کاهش پایداری در منحنی‌های ولیدیشن نشان می‌دهد که مدل نسبت به داده‌های محدود حساس‌تر شده است. این موضوع بیانگر آن است که تنظیم بیش از حد لایه‌های میانی، در شرایط داده‌ی محدود، می‌تواند به تخریب جزئی نمایش‌های مفید از پیش‌آموزش‌دیده منجر شود.

در حالت Full Fine-tune، اگرچه مدل بیشترین تعداد پارامترهای قابل آموزش را دارد و بیش‌برازش همچنان مشاهده می‌شود، اما روند آموزش نسبت به حالت Conv2 + Head پایداری بهتری دارد و نوسانات کمتری در منحنی‌های ولیدیشن دیده می‌شود. این نشان می‌دهد که مقداردهی اولیه‌ی مناسب باعث شده فرآیند بهینه‌سازی در فضای پارامترها جهت‌دارتر باشد. در عین حال، عملکرد نهایی نسبت به سایر حالات کمی بهبود یافته است، که بیانگر توانایی مدل در تطبیق بهتر با دامنه‌ی هدف و تفکیک دقیق‌تر برخی کلاس‌ها است.

در مقابل، حالت آموزش از ابتدا (Scratch) به‌وضوح ضعیف‌ترین عملکرد را دارد. روند آموزش ناپایدارتر بوده، نوسانات قابل‌توجهی در منحنی‌های ولیدیشن مشاهده می‌شود و فاصله‌ی قابل‌توجهی بین دقت آموزش و ولیدیشن وجود دارد. همچنین، ماتریس درهم‌ریختگی نشان‌دهنده‌ی خطاهای گسترده‌تر در اکثر کلاس‌ها است. این نتیجه تأیید می‌کند که در حضور داده‌ی محدود، نبود دانش اولیه منجر به واریانس بالا، ناپایداری در یادگیری و تعمیم‌پذیری ضعیف می‌شود.

از نظر هزینه‌ی محاسباتی نیز، هرچه تعداد پارامترهای قابل آموزش بیشتر شود، هزینه‌ی آموزش افزایش می‌یابد؛ در حالی که حالت Head Only کم‌هزینه‌ترین و Full Fine-tune پرهزینه‌ترین سناریو است. با این حال، تفاوت عملکرد بین این دو حالت نسبت به اختلاف هزینه، چندان چشمگیر نیست.

نکته‌ی مهم این است که با افزایش تعداد پارامترهای قابل آموزش، به دلیل محدود بودن داده‌های هدف، ریسک بیش‌برازش افزایش می‌یابد، زیرا مدل آزادی بیشتری برای تطبیق با جزئیات داده‌های آموزشی پیدا می‌کند. با این حال، اگر حجم داده‌ی هدف بیشتر بود، این افزایش ظرفیت می‌توانست به بهبود تعمیم‌پذیری منجر شود. همچنین در شرایطی که شباهت بین دامنه‌ی مبدأ و هدف کمتر باشد، باز کردن لایه‌های بیشتر می‌تواند حتی به کاهش بایاس و بهبود عملکرد کمک کند، زیرا مدل فرصت اصلاح نمایش‌های اولیه را پیدا می‌کند.

در مجموع، نتایج نشان می‌دهد که انتخاب میزان fine-tuning یک trade-off میان ظرفیت مدل، حجم داده و میزان شباهت دامنه‌ها است؛ به‌طوری‌که در این مسئله، Full Fine-tune بهترین عملکرد را ارائه داده، اما Head Only نیز با هزینه‌ی کمتر، عملکردی رقابتی و پایدار داشته است.

۴-۳- تحلیل قابلیت انتقال لایه‌ها و مقایسه با Domain Adaptation

در این بخش، هدف بررسی این است که کدام لایه‌های مدل LeNet بیشترین قابلیت انتقال[۳۲] را دارند و این ویژگی تحت چه شرایطی باعث برتری یادگیری انتقالی نسبت به روش‌های Domain Adaptation می‌شود.

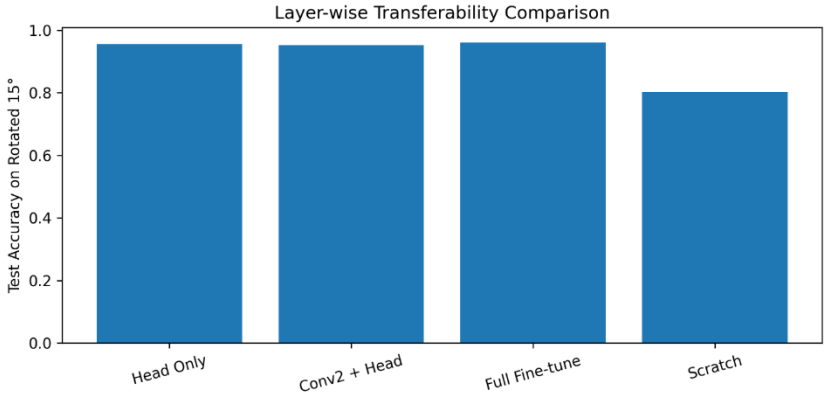
برای شفافیت، نتایج کمی در جدول زیر ارائه شده‌اند:

جدول ۴–۱: مقایسه عملکرد سناریوهای مختلف از نظر دقت و شکاف تعمیم‌پذیری

حالت	لایه‌های قابل آموزش	Rot15 Test Acc	Val Acc	Generalization Gap (Train Acc – Val Acc)
Head Only	fc1, fc2, fc3	۰.۹۵۶۴	۰.۹۸۵	۰.۰۲۰
Conv2 + Head	conv2, fc1, fc2, fc3	۰.۹۵۲۷	۰.۹۸۰	۰.۰۲۵
Full Fine-tune	all layers	۰.۹۶۱۰	۰.۹۸۵	۰.۰۱۵
Scratch	all layers (random init)	۰.۸۰۲۳	۰.۸۰۵	۰.۱۸۰

[32] Transferability

همچنین مقایسه‌ی دیداری دقت روی داده‌های چرخیده در شکل زیر آمده است:



شکل ۴–۱۱: مقایسه دقت تست روی داده‌های چرخیده برای سناریوهای مختلف

نتایج نشان می‌دهند که در حالت Head Only که تنها لایه‌های fc1، fc2 و fc3 آموزش داده شده‌اند و لایه‌های conv1 و conv2 فریز هستند، مدل همچنان به عملکرد بالایی دست پیدا می‌کند. این موضوع به‌وضوح نشان می‌دهد که ویژگی‌های استخراج‌شده توسط conv1 و conv2 به‌خوبی قابل انتقال هستند و تغییر دامنه از MNIST معمولی به نسخه چرخیده، تأثیر جدی بر کارایی این لایه‌ها ندارد. به بیان دیگر، این دو لایه الگوهایی مانند لبه‌ها، انحناها و ساختار کلی ارقام را استخراج می‌کنند که تحت چرخش‌های کوچک پایدار باقی می‌مانند.

در حالت Conv2 + Head که علاوه بر لایه‌های fully connected، لایه conv2 نیز قابل آموزش شده است، انتظار می‌رود مدل بتواند خود را بهتر با دامنه هدف تطبیق دهد. با این حال، نتایج نشان می‌دهند که این تغییر نه‌تنها بهبود قابل توجهی ایجاد نکرده، بلکه اندکی افت عملکرد نیز مشاهده می‌شود. این رفتار نشان می‌دهد که لایه conv2 نسبت به conv1 وابستگی بیشتری به دامنه مبدأ دارد و تغییر آن در شرایط داده محدود می‌تواند به تخریب بخشی از نمایش‌های مفید منجر شود. در واقع، conv2 ترکیب‌های پیچیده‌تری از ویژگی‌ها را مدل می‌کند که تا حدی به ساختار خاص داده‌های اولیه وابسته هستند.

در حالت Full Fine-tune که تمامی لایه‌ها (conv1, conv2, fc1, fc2, fc3) قابل آموزش هستند، بهترین عملکرد نهایی مشاهده می‌شود. این نشان می‌دهد که اگرچه conv1 و conv2 از ابتدا ویژگی‌های قابل انتقال خوبی ارائه می‌دهند، اما امکان تنظیم مجدد آن‌ها در کنار مقداردهی اولیه مناسب، می‌تواند به تطبیق دقیق‌تر مدل با دامنه هدف منجر شود. نکته مهم این است که بهبود عملکرد نسبت به حالت Head Only محدود است، که تأیید می‌کند بخش عمده‌ی دانش قابل انتقال از قبل در conv1 و conv2 وجود داشته است.

در مقابل، در حالت Scratch که تمام لایه‌ها از ابتدا آموزش داده می‌شوند، عملکرد به‌طور قابل توجهی ضعیف‌تر است. این نشان می‌دهد که بدون وجود ویژگی‌های از پیش آموخته‌شده در conv1 و conv2، مدل قادر به استخراج نمایش‌های عمومی مناسب از داده‌های محدود نیست و در نتیجه تعمیم‌پذیری آن کاهش می‌یابد.

بر اساس این تحلیل، می‌توان نتیجه گرفت که:

- لایه‌های conv1 و conv2 بیشترین قابلیت انتقال را دارند.
- لایه‌های fc1, fc2, fc3 وابسته به مسئله هدف هستند و باید مجدداً آموزش داده شوند.
- باز کردن conv2 بدون داده کافی می‌تواند به تخریب جزئی دانش منتقل‌شده منجر شود.

در ادامه، برای مقایسه با روش‌های Domain Adaptation، یک آزمایش مبتنی بر reweighting نیز انجام شد. در این آزمایش، تنها از همان ۲۰۰ نمونه‌ی دامنه هدف (بدون استفاده از برچسب‌ها) برای تخمین وزن‌ها استفاده شد تا شرایط داده‌ای با سناریوی یادگیری انتقالی یکسان باشد. به این ترتیب، تفاوت اصلی بین دو رویکرد صرفاً در نوع سیگنال مورد استفاده (برچسب‌دار در مقابل بدون برچسب) خواهد بود.

جدول ۴–۲: مقایسه یادگیری انتقالی با ***Reweighting Domain Adaptation*** *(با ۲۰۰ نمونه دامنه هدف)*

روش	**نوع داده هدف**	**نوع سیگنال**	**دقت تست (Rot15)**
Transfer (Full Fine-tune)	۲۰۰ نمونه چرخیده	برچسب‌دار	۰.۹۶۱۰
Reweighting DA	۲۰۰ نمونه چرخیده	برچسب‌دار	۰.۳۱۹۸

همان‌طور که در جدول مشاهده می‌شود، روش reweighting در این شرایط عملکرد بسیار ضعیف‌تری نسبت به یادگیری انتقالی دارد. دلیل اصلی این موضوع آن است که در reweighting، وزن‌دهی به نمونه‌ها بر اساس تخمین نسبت چگالی بین دامنه مبدأ و هدف انجام می‌شود، اما این تخمین در فضای داده‌های تصویری و با تعداد بسیار محدود نمونه (۲۰۰ نمونه) به‌صورت ناپایدار و نویزی صورت می‌گیرد. در نتیجه، وزن‌های محاسبه‌شده نمی‌توانند به‌درستی اهمیت نمونه‌ها را منعکس کنند و حتی ممکن است باعث شوند مدل بر روی نمونه‌های نامناسب تمرکز بیشتری داشته باشد.

در مقابل، در یادگیری انتقالی، مدل از همان تعداد داده هدف اما به‌صورت برچسب‌دار استفاده می‌کند و مستقیماً از سیگنال نظارتی برای تنظیم پارامترها بهره می‌برد. این موضوع باعث می‌شود فرآیند تطبیق با دامنه هدف به‌صورت هدفمندتر و پایدارتر انجام شود. علاوه بر این، در این مسئله خاص، تفاوت بین دامنه‌ها محدود و از نوع تغییرات هندسی ساده (چرخش) است، بنابراین ویژگی‌های استخراج‌شده توسط conv1 و conv2 همچنان معتبر باقی می‌مانند و نیازی به هم‌ترازسازی پیچیده توزیع‌ها وجود ندارد.

در مجموع، این نتایج نشان می‌دهد که یادگیری انتقالی در شرایطی که:

- حجم داده در دامنه هدف محدود باشد

در این حالت، اگر داده‌های هدف به‌صورت برچسب‌دار در دسترس باشند، یادگیری انتقالی می‌تواند بسیار مؤثرتر از روش‌های Domain Adaptation عمل کند. زیرا مدل مستقیماً از سیگنال نظارتی برای تنظیم پارامترها استفاده

می‌کند، در حالی‌که روش‌های تطبیق دامنه در داده‌های کم، به دلیل وابستگی به تخمین توزیع، معمولاً ناپایدار هستند.

- دانش به‌دست‌آمده از pretraining با دامنه هدف هم‌راستا باشد

در این مسئله، تغییر دامنه صرفاً یک چرخش ساده است و بنابراین ویژگی‌های استخراج‌شده توسط conv1 و conv2 همچنان معتبر باقی می‌مانند. در چنین شرایطی، مدل تنها نیاز به تنظیم جزئی دارد و یادگیری انتقالی می‌تواند به‌صورت مستقیم و مؤثر عمل کند.

می‌تواند عملکردی به‌مراتب بهتر از روش‌های Domain Adaptation داشته باشد. در مقابل، روش‌هایی مانند reweighting که به تخمین توزیع وابسته هستند، در داده‌های کم می‌توانند ناپایدار شده و حتی منجر به افت شدید عملکرد شوند.

۴-۴- نکات پیاده‌سازی و ملاحظات عملی

در این بخش، برخی از قسمت‌های کلیدی پیاده‌سازی که نقش مهمی در نتایج نهایی داشته‌اند بررسی می‌شوند. تمرکز این بخش بر روی قطعه‌کدهایی است که مستقیماً بر رفتار مدل در سناریوهای مختلف یادگیری انتقالی تأثیر گذاشته‌اند و درک صحیح آن‌ها برای تحلیل نتایج ضروری است.

۴-۴-۱- کنترل لایه‌های قابل آموزش در سناریوهای مختلف

در سناریوهای مختلف یادگیری انتقالی، یکی از مهم‌ترین تصمیمات، تعیین این است که کدام لایه‌ها باید آموزش داده شوند و کدام لایه‌ها باید ثابت باقی بمانند. قطعه‌کد زیر این منطق را پیاده‌سازی می‌کند:

```python
def set_trainable_layers(model, mode="head_only"):
    for p in model.parameters():
        p.requires_grad = False

    if mode == "head_only":
        for p in model.fc1.parameters():
            p.requires_grad = True
        for p in model.fc2.parameters():
            p.requires_grad = True
        for p in model.fc3.parameters():
            p.requires_grad = True

    elif mode == "conv2_plus_head":
        for p in model.conv2.parameters():
            p.requires_grad = True
        for p in model.fc1.parameters():
            p.requires_grad = True
        for p in model.fc2.parameters():
            p.requires_grad = True
        for p in model.fc3.parameters():
            p.requires_grad = True

    elif mode == "full_finetune":
        for p in model.parameters():
            p.requires_grad = True

    else:
        raise ValueError(f"Unknown mode: {mode}")
```

شکل ۴–۱۲: تابع تنظیم لایه‌های قابل آموزش برای سناریوهای مختلف ***fine-tuning***

این تابع ابتدا تمامی پارامترهای مدل را غیرفعال می‌کند (requires_grad=False) و سپس بر اساس سناریوی انتخاب‌شده، تنها زیرمجموعه‌ای از لایه‌ها را برای آموزش فعال می‌کند. این طراحی باعث می‌شود که کنترل دقیقی روی ظرفیت مدل و میزان تغییر در نمایش‌های آموخته‌شده داشته باشیم.

در حالت head_only، تنها لایه‌های fc1، fc2 و fc3 قابل آموزش هستند. این حالت معادل استفاده از مدل به‌عنوان یک feature extractor ثابت است که در آن نمایش‌های استخراج‌شده توسط conv1 و conv2 بدون تغییر باقی می‌مانند. این تنظیم معمولاً منجر به پایداری بیشتر در آموزش می‌شود، زیرا فضای پارامترهای قابل به‌روزرسانی محدود است.

در حالت conv2_plus_head، علاوه بر لایه‌های fully connected، لایه conv2 نیز قابل آموزش می‌شود. این کار به مدل اجازه می‌دهد تا بخشی از نمایش‌های میانی را با دامنه هدف تطبیق دهد، اما همان‌طور که در نتایج مشاهده شد، این افزایش انعطاف‌پذیری در شرایط داده محدود می‌تواند به ناپایداری و حتی افت عملکرد منجر شود.

در حالت full_finetune، تمامی پارامترهای مدل قابل آموزش هستند. این حالت بیشترین ظرفیت را در اختیار مدل قرار می‌دهد و امکان تطبیق کامل با دامنه هدف را فراهم می‌کند، اما در عین حال ریسک بیش‌برازش نیز در آن بیشتر است.

نکته مهم در این پیاده‌سازی این است که استفاده از requires_grad به‌عنوان مکانیزم کنترل آموزش، نه‌تنها ساده و شفاف است، بلکه به‌طور مستقیم در فرآیند backpropagation نیز اعمال می‌شود؛ به این معنا که گرادیان تنها برای پارامترهای فعال محاسبه و به‌روزرسانی می‌شود. این موضوع باعث می‌شود که تغییر رفتار مدل در سناریوهای مختلف، به‌صورت دقیق و قابل کنترل انجام شود.

۴-۵- نتیجه‌گیری

در این فصل، تأثیر یادگیری انتقالی در مقایسه با آموزش از ابتدا و همچنین نقش تنظیم لایه‌های مختلف مدل مورد بررسی قرار گرفت. نتایج نشان داد که استفاده از یادگیری انتقالی، به‌ویژه در شرایطی که داده‌های دامنه هدف محدود هستند، به‌طور قابل توجهی عملکرد بهتری نسبت به آموزش از صفر ارائه می‌دهد و موجب بهبود تعمیم‌پذیری مدل می‌شود.

همچنین مشاهده شد که میزان باز بودن لایه‌ها نقش مهمی در تعادل بین ظرفیت مدل و بیش‌برازش دارد. در حالی که باز کردن کامل لایه‌ها (Full Fine-tune) منجر به بهترین عملکرد نهایی شد، اما ریسک بیش‌برازش را نیز افزایش داد. در مقابل، محدود نگه داشتن لایه‌های قابل آموزش (Head Only) با وجود کاهش جزئی در دقت، پایداری بیشتری در فرآیند آموزش ایجاد کرد.

در مقایسه با روش‌های تطبیق دامنه، نتایج نشان داد که در شرایطی که داده‌ی هدف محدود اما برچسب‌دار باشد، یادگیری انتقالی می‌تواند عملکرد بهتری ارائه دهد. در مقابل، روش‌هایی مانند reweighting که به تخمین توزیع وابسته هستند، در داده‌های کم دچار ناپایداری شده و حتی ممکن است عملکرد ضعیف‌تری نسبت به روش‌های ساده‌تر داشته باشند.

در مجموع، این فصل نشان می‌دهد که انتخاب مناسب بین یادگیری انتقالی، میزان fine-tuning و روش‌های تطبیق دامنه، به عواملی مانند حجم داده، نوع شیفت دامنه و دسترسی به برچسب‌ها وابسته است و باید به‌صورت آگاهانه و متناسب با مسئله انجام شود.